

More Recent Advances in (Hyper)Graph Partitioning

ÜMIT V. ÇATALYÜREK, Georgia Institute of Technology, Atlanta, GA, United States of America
KAREN D. DEVINE, Sandia National Laboratories, ret., United States of America
MARCELO FONSECA FARAJ, Heidelberg University, Germany
LARS GOTTESBÜREN, Karlsruhe Institute of Technology, Germany
TOBIAS HEUER, Karlsruhe Institute of Technology, Germany
HENNING MEYERHENKE, Humboldt-Universität zu Berlin, Germany
PETER SANDERS, Karlsruhe Institute of Technology, Germany
SEBASTIAN SCHLAG, Apple Inc., United States of America
CHRISTIAN SCHULZ, Heidelberg University, Germany
DANIEL SEEMAIER, Karlsruhe Institute of Technology, Germany
DOROTHEA WAGNER, Karlsruhe Institute of Technology, Germany

In recent years, significant advances have been made in the design and evaluation of balanced (hyper)graph partitioning algorithms. We survey trends of the last decade in practical algorithms for balanced (hyper)graph partitioning together with future research directions. Our work serves as an update to a previous survey on the topic [34]. In particular, the survey extends the previous survey by also covering hypergraph partitioning and streaming algorithms, and has an additional focus on parallel algorithms.

CONTENTS

Abstract	1
Contents	1
1 Introduction	2
2 Overview/Classification	3
3 Preliminaries	3
3.1 Notation	3
3.2 The k -way (Hyper)Graph Partitioning Problem	4
4 Applications	4
5 Sequential Techniques	6
5.1 Classic (Hyper)Graph Partitioning Techniques	6
5.2 Recent Advances in Multilevel Partitioning	7
5.3 Evolutionary Computation	11
5.4 Streaming	12
5.5 Repartitioning	17
5.6 Objective Functions and Problem Variations	19
6 Parallel Algorithms	23
6.1 Shared Memory	23
6.2 Distributed-Memory	28
6.3 GPU	32
6.4 Approaches for Other Types of Hardware	33
7 Experimental Methodology	34
8 Future Challenges	35
References	36

1 INTRODUCTION

Graphs are a universal and widely used way to model relations between objects. They are useful in a wide range of applications from social networks, simulation grids, road networks/route planning, (graph) neural networks, and many more. With exploding data sets, the scalability of graph processing methods has become an increasing challenge. As huge problem instances in the area become abundant, there is a need for scalable algorithms to perform analysis. Often (hyper)graph partitioning is a key technology to make various algorithms scale in practice. More precisely, scalable algorithms for various applications often require a subroutine that partitions a (hyper)graph into k blocks of roughly equal size such that the number of edges that run between blocks is minimized. Balance is most often modeled using a *balancing constraint* that demands that all block weights are below a given upper bound. The number of applications that require such partitions of (hyper)graphs as key subroutine is huge. Social network operators use graph partitioning techniques to load balance their operation and make sure that site response times are low [110]; efficient route planning algorithms rely on a precomputation phase in which a road network needs to be partitioned [49]; scientific simulations that run on supercomputers use (hyper)graph partitioning to balance load and minimize communication between processors [41, 67, 187]; partitioning (graph) neural networks is helpful to speedup training times [234]; and (hyper)graph partitioning helps to improve chip placements in VLSI design [8, 158]. On the other side, the problem is NP-hard [71] even for unweighted trees of maximum degree three [11] and no constant-factor approximation algorithms exist [13]. Thus, heuristic algorithms are used in practice. The purpose of this paper is to give a structured overview of the rich literature, with a clear emphasis on explaining key ideas and discussing work that has been published in the last decade and is missing in other overviews. More precisely, our work serves as an update to a previous generic survey on the topic [34]; in particular we also extend the scope of the survey to hypergraph partitioning algorithms.

There have been other (older) surveys on the topic. The book by Bichot and Siarry [26] covers techniques for graph partitioning such as the multilevel method, metaheuristics, parallel methods, and hypergraph partitioning, as well as applications of graph partitioning. The survey by Schloegel et al. [185] was published around the turn of the millennium and has a focus on techniques for scientific computing, including algorithms for adaptive and dynamic simulations and process mapping algorithms. Static algorithms as well as formulations with multiple objectives and constraints are also discussed. Monien et al. [148] discuss multilevel algorithms. Their description has a focus on matching-based coarsening and local search that use vertex-swapping heuristics. Kim et al. [121] cover memetic algorithms. Abbas et al. [1] describe and categorize streaming graph partitioning techniques based on their assumptions, objectives and costs, and perform an experimental comparison of the algorithms for different applications and datasets. The last generic survey on the topic by Buluç et al. [34] covers a wide range of techniques and practical algorithms that have been published in or before 2013. For hypergraph partitioning there exist two older surveys [8, 158]. Alpert and Kahng [8] discuss min-cut and ratio cut bipartitioning formulations along with multi-way extensions, constraint-driven partitioning and partitioning with replication. Papa and Markov [158] discuss practical applications of hypergraph partitioning, exact algorithms, and various local search heuristics for the problem as well as software packages and benchmarks. Schlag [182] presents a recent overview of hypergraph techniques in his dissertation.

Our survey is structured as follows. We start by introducing the problems and other preliminaries in Section 3. Then in Section 4, we focus on new applications that emerged in the last decade. We continue with novel sequential techniques in Section 5. Then we continue with covering parallel algorithms in Section 6. We describe experimental methodology in Section 7. We conclude with future challenges in Section 8.

2 OVERVIEW/CLASSIFICATION

The field is currently very active. On the one side there are *streaming algorithms* that are very fast and consume little memory yet yield only low quality solutions. These algorithms assume that internal memory of a single machine proportional to the number of vertices is available and typically load nodes and their neighborhood one by one to directly make an assignment to a block. Hence, the edges of a graph do not have to fit into the memory of the machine. On the other side, there is a wide range of sequential internal-memory algorithms that tackle partitioning problems that fit into the main memory of a single machine without using parallelism. Recent advances in this area provide a rich set of algorithms that are able to compute partitions of very high quality. These high-quality algorithms often also have shared-memory parallel counterparts. While the sequential internal memory/parallel shared-memory non-metaheuristic algorithms in this area use a reasonable amount of time for most applications, there are also memetic or evolutionary algorithms that invest a lot of resources to achieve even higher quality. Note that memetic or evolutionary algorithms can be sequential internal-memory, shared-memory or even distributed memory parallel. Due to their high running time, such algorithms are typically used only on graphs having a few hundred thousand nodes. Exact solvers are currently able to solve only very small instances while also requiring a lot of time to solve an instance even for few blocks. For larger numbers of blocks, the exact solvers currently do not work well. Distributed parallel algorithms scale well to large instances, but if these tools do not implement multilevel strategies, they typically have much lower quality than sequential internal memory partitioning algorithms. Even when such algorithms use the multilevel scheme, solution quality of the algorithms operating in this model of computation is typically worse compared to their internal memory/shared-memory counterparts. However, this mode of computation has the advantage that huge instances can be partitioned very quickly and also that it enables researchers to partition huge graphs on cheap machines. Lastly, researchers also work on algorithms for other platforms like GPUs to be able to fully use the capabilities of existing hardware. However, GPUs have limited memory size and current algorithms running on a GPU compute partitions that cut significantly more edges than high-quality internal memory schemes.

3 PRELIMINARIES

3.1 Notation

Hypergraphs & Graphs. A *weighted undirected hypergraph* $H = (V, E, c, \omega)$ is defined as a set of n vertices V and a set of m hyperedges/nets E with vertex weights $c : V \rightarrow \mathbb{R}_{>0}$ and net weights $\omega : E \rightarrow \mathbb{R}_{>0}$, where each net e is a subset of the vertex set V (i.e., $e \subseteq V$). The vertices of a net are called *pins*. We extend c and ω to sets in the natural way, i.e., $c(U) := \sum_{v \in U} c(v)$ and $\omega(F) := \sum_{e \in F} \omega(e)$. A vertex v is *incident* to a net e if $v \in e$. $I(v)$ denotes the set of all incident nets of v . The set $\Gamma(v) := \{u \mid \exists e \in E : \{v, u\} \subseteq e\}$ denotes the neighbors of v . The *degree* of a vertex v is $d(v) := |I(v)|$. We assume hyperedges to be sets rather than multisets, i.e., a vertex can only be contained in a hyperedge *once*. Nets of size one are called *single-vertex* nets. Given a subset $V' \subset V$, the *subhypergraph* $H_{V'}$ is defined as $H_{V'} := (V', \{e \cap V' \mid e \in E : e \cap V' \neq \emptyset\})$.

A *weighted undirected graph* $G = (V, E, c, \omega)$ is defined as a set of n vertices V and a set of m and edges E with vertex weights $c : V \rightarrow \mathbb{R}_{>0}$ and edge weights $\omega : E \rightarrow \mathbb{R}_{>0}$. In contrast to hypergraphs, the size of the edges is restricted to two. Let $G = (V, E, c, \omega)$ be a weighted (directed) graph. We use *hyperedges/nets* when referring to hypergraphs and *edges* when referring to graphs. However, we use the same notation to refer to vertex weights c , edge weights ω , vertex degrees $d(v)$, and the set of neighbors Γ . In an undirected graph, an edge $(u, v) \in E$ implies an edge $(v, u) \in E$ and $\omega(u, v) = \omega(v, u)$.

Partitions and Clusterings. A k -way partition of a (hyper)graph H is a partition of its vertex set into k blocks $\Pi = \{V_1, \dots, V_k\}$ such that $\bigcup_{i=1}^k V_i = V$, $V_i \neq \emptyset$ for $1 \leq i \leq k$, and $V_i \cap V_j = \emptyset$ for $i \neq j$. We call a k -way partition Π ε -balanced if each block $V_i \in \Pi$ satisfies the *balance constraint*: $c(V_i) \leq L_{\max} := (1 + \varepsilon) \left\lceil \frac{c(V)}{k} \right\rceil$ for some parameter ε .¹ We call a block V_i *overloaded* if $c(V_i) > L_{\max}$. For each net e , $\Lambda(e) := \{V_i \mid V_i \cap e \neq \emptyset\}$ denotes the *connectivity set* of e . The *connectivity* $\lambda(e)$ of a net e is the cardinality of its connectivity set, i.e., $\lambda(e) := |\Lambda(e)|$. A net is called a *cut net* if $\lambda(e) > 1$; otherwise (i.e., if $|\lambda(e)| = 1$) it is called an *internal net*. A vertex u that is incident to at least one cut net is called a *border vertex*. The number of pins of a net e in block V_i is defined as $\Phi(e, V_i) := |\{V_i \cap e\}|$. A block V_i is *adjacent* to a vertex $v \notin V_i$ if $\exists e \in I(v) : V_i \in \Lambda(e)$. We use $B(v)$ to denote the set of all blocks adjacent to v . Given a k -way partition Π of H , the *quotient graph* $Q := (\Pi, \{(V_i, V_j) \mid \exists e \in E : \{V_i, V_j\} \subseteq \Lambda(e)\})$ contains an edge between each pair of adjacent blocks. A *clustering* $C = \{C_1, \dots, C_l\}$ of a hypergraph is a partition of its vertex set. In contrast to a k -way partition, the number of clusters is not given in advance, and there is no balance constraint on the actual sizes of the clusters C_i .

A k -way (hyper)edge partition of a (hyper)graph H is a partition of its (hyper)edge set into k blocks $\Pi = \{E_1, \dots, E_k\}$ such that $\bigcup_{i=1}^k E_i = E$, $E_i \neq \emptyset$ for $1 \leq i \leq k$, and $E_i \cap E_j = \emptyset$ for $i \neq j$. We call a k -way (hyper)edge partition Π ε -balanced if each block $E_i \in \Pi$ satisfies the *balance constraint*: $\omega(E_i) \leq (1 + \varepsilon) \left\lceil \frac{\omega(E)}{k} \right\rceil$ for some parameter ε . A vertex is called a *cut vertex* if it contains two or more edges in different blocks. The (weighted) *vertex-cut* of a k -way (hyper)edge partition is the total weight of cut vertices.

3.2 The k -way (Hyper)Graph Partitioning Problem

Problem Definition. The k -way (hyper)graph partitioning problem is to find an ε -balanced k -way partition Π of a (hyper)graph $H = (V, E, c, \omega)$ that *minimizes* an objective function over the cut nets for some value of ε . The two most commonly used cost functions in case we are dealing with hypergraphs are the *cut-net* metric $f_c(\Pi) := \sum_{e \in E'} \omega(e)$ and the *connectivity* metric $f_\lambda(\Pi) := \sum_{e \in E'} (\lambda(e) - 1) \omega(e)$, where E' is the *cut-set* (i.e., the set of all cut nets) [52, 57]. While the cut-net metric sums the weights of all nets that connect more than one block of the partition Π , the connectivity metric additionally takes into account the actual number λ of blocks connected by the cut nets. When partitioning graphs, the objective is often to minimize $\sum_{i < j} \omega(E_{ij})$ (weight of all cut edges), where $E_{ij} := \{\{u, v\} \in E \mid u \in V_i, v \in V_j\}$. Note that for graphs the objective functions $f_c(\Pi)$ and $f_\lambda(\Pi)$ revert to edge-cut (i.e., the sum of the weights of those edges that have endpoints in different blocks). Apart from these common cost functions, other specialized objective functions exist [34]. Figure 1 shows an example partition of a graph. Throughout the paper, we use the term high-quality if the respective objective function is small compared to other tools, and low-quality if the opposite is the case.

4 APPLICATIONS

Research on graph partitioning algorithms has always been motivated by its numerous applications. Some traditional applications are parallel processing (e.g., in scientific computing), VLSI design, route planning and image segmentation; see also the previous survey [34]. Yet, in recent years several new applications have emerged.

Distributed Databases and Query Optimization. A prominent application is distributed database sharding [16, 47, 110, 126, 226]. Vertices represent data records, hyperedges represent queries

¹The $\lceil \cdot \rceil$ in this definition ensures that there is always a feasible solution for inputs with unit vertex weights. For general weighted inputs, there is no commonly accepted way how to deal with feasibility; see also [86, 103].

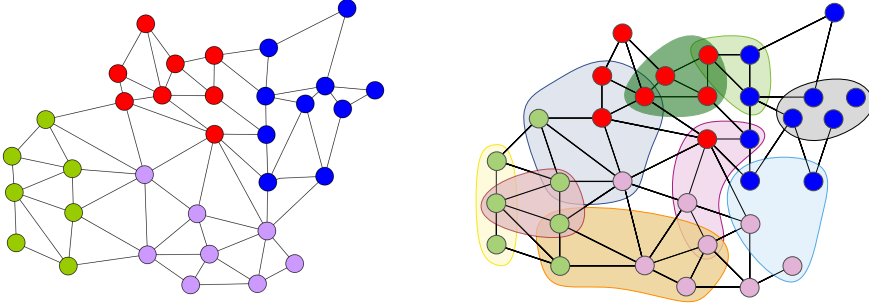


Fig. 1. Left: An example graph that is partitioned into four blocks indicated by the colors. The partition has an edge cut of 17, the block weights $|V_i|$ are green (7), red (8), blue (10), purple (8). Right: An example hypergraph that is partitioned into four blocks indicated by the colors. Hyperedges are indicated by black lines or colored blobs around the vertices (if more than two nodes are contained in a hyperedge). Blocks have the same weight as in the graph case. The connectivity objective is 22 and the number of hyperedges cut is 20.

that access records, and blocks correspond to shards (machines). Minimizing connectivity thus corresponds to minimizing the average number of shards involved in a query, which optimizes the latency and query processing time.

A similar application is to boost IO throughput in the Google search engine backend by improving cache utilization. Archer et al. [15] initialize a voting table that assigns search requests, using graph partitioning on a bipartite graph where vertices are search terms and queries, and an edge exists for each term contained in a query. Subsequent simulation-and-refinement further boosts the prediction accuracy (item in cache) of the voting table.

Li et al. [133] use graph partitioning to construct a set similarity search index. The approach first filters candidate sets using the index, then checks the remaining candidates brute-force. Vertices represent sets, edges connect sets that are k -nearest neighbors (or within a similarity threshold, depending on the query type), and cut size is correlated with pruning efficiency.

Data Distribution and Scheduling. Djidjev et al. [55] use graph partitioning to accelerate and parallelize the computation of density matrices for molecular dynamics simulations. Lattice Boltzmann Fluid Flow simulations [66, 76, 125] fall into the classic load balancing and communication minimization application category of graph partitioning. Yu et al. [231] consider sparse matrix vector multiplication [38], however focused on shared-memory machines with many NUMA nodes instead of distributed systems.

Parallel graph processing systems such as Pregel [142] or Giraph [95] have become wide-spread tools for network analysis tasks. These systems employ a think-like-a-vertex or think-like-an-edge programming paradigm, requiring a balanced partition of vertices or edges across machines. Streaming approaches [105, 140] based on label propagation result in better performance than traditional range-based or hash-based partitioning.

Quantum Circuit Simulation. Several papers employ nested dissection on hypergraphs to find good contraction trees, in order to speed up the simulation of quantum circuits on classical non-quantum machines [89, 107, 157]. This can be used to experimentally verify the correctness of a quantum circuit, and push back premature claims of having achieved quantum supremacy.

SAT Solving. Boolean satisfiability (SAT) formulas can be represented as hypergraphs. One representation is the dual representation with clauses as vertices and variables as hyperedges,

consisting of clauses that contain the variable (either negated or not). Mann and Papp [138] use balanced bipartitioning to identify variables that branching solvers should focus on assigning first, such that the formula is split into two once variables in the cut are assigned. The two sub-formulas can be solved independently, which is expected to be faster. Most SAT solvers employ a heuristic (VSIDS) that assigns priorities (frequency in observed conflict clauses) to variables, selecting the highest priority to branch on next. The authors consider two variants, the latter of which is more successful: force split the formula first and use VSIDS for tie-breaking, or initialize the priorities with the partition and let VSIDS overrule the decision.

Miscellaneous. Lamm et al. [128] formulate recombine-operators in an evolutionary framework for independent sets based on separator decompositions and graph partitions. Given two independent sets, their vertices in the two blocks of a separator decomposition are exchanged, yielding two offspring independent sets that are refined using local search.

Kumar et al. [127] use a clustering formulation to compute trajectories of moving objects in videos. Detected objects in a frame correspond to vertices and each block corresponds to an object. The authors enforce special constraints such that each partition contains only one object per frame and such that objects do not jump between frames.

Yao et al. [227] consider the placement of control units in software-defined networks to minimize the average latency of messages from switches (nodes) to controllers. First the number of necessary controllers is estimated based on message volume capacities, then the switch graph is partitioned and one control unit is placed in each block, using a separate routine to perform placement inside the blocks.

Quantum chemical simulations such as *density functional theory* exhibit quadratic time complexity, which is why calculations on largely independent sub-systems are used to approximate and accelerate the process. Von Looz et al. [215] propose to use graph partitioning to automate the sub-system construction process, where small edge cuts correspond to small introduced calculation errors.

5 SEQUENTIAL TECHNIQUES

5.1 Classic (Hyper)Graph Partitioning Techniques

This section discusses partitioning techniques repeatedly used in this survey. We briefly outline the multilevel paradigm and the most common local search algorithms. These algorithms move vertices according to gain value. The gain value $g_u(V_j)$ reflects the change in the objective function when we move a vertex u from its current to a target block V_j (e.g., reduction in the edge cut).

The Multilevel Paradigm. The most successful approach to solve the (hyper)graph partitioning problem is the *multilevel* paradigm. It consists of three phases. In the *coarsening* phase, a hierarchy of successively smaller and structurally similar (hyper)graphs are created by contracting matchings or clusters of vertices. Once the (hyper)graph is small enough, an *initial partitioning* algorithm obtains a partition of the coarsest (hyper)graph. In the *uncoarsening* phase, the partition is projected to the next larger (hyper)graph in the hierarchy, and, at each level, *local search* algorithms improve the objective function (e.g., edge cut). Figure 3 illustrates the multilevel paradigm.

A contraction of several vertices into a supervertex aggregates their weight in the supervertex. To further reduce the size of the coarser (hyper)graph, one can also remove all edges that become identical except for one, at which their weight is aggregated (self-loops are discarded). This way, one obtains a partition with the same balance and cut properties when projecting a partition to the next larger hypergraph in the hierarchy.

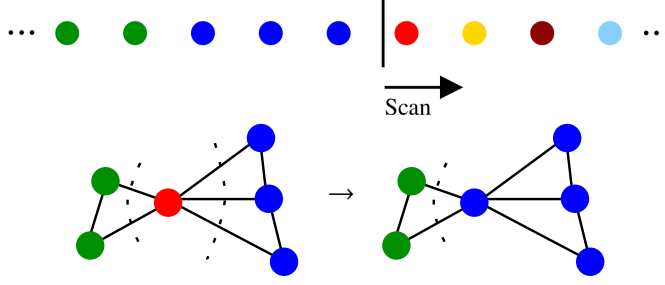


Fig. 2. Label propagation algorithm in graphs. Top: nodes are scanned in a specific order. Colors of nodes indicate their block. The algorithm initially assigns each node its own label and then visits the vertices in some order, assigning the current vertex the label that is most frequent in its neighborhood.

The Label Propagation Algorithm. The label propagation algorithm, illustrated in Fig. 2, was originally proposed to detect community structures in large-scale networks [168, 235] but was also used as a refinement technique in the partitioning context [117]. The label propagation algorithm works in rounds, and each vertex u is associated with a label $L[u]$. Initially, each vertex is assigned its own label (i.e., $L[u] = u$). In each round, the vertices are visited in some order, and whenever a vertex u is visited, it adapts its label to the label appearing most frequently in its neighborhood (ties are broken randomly). The algorithm proceeds until it reaches a predefined number of rounds or none of the vertices changes its label in a round. The algorithm can be used as a clustering algorithm in the coarsening phase [144] or a local search algorithm when the current k -way partition is used for the initial label assignment.

The Fiduccia-Mattheyses Algorithm. Fiduccia and Mattheyses [64] presented the first linear-time heuristic for the balanced bipartitioning problem (referred to as FM algorithm). The algorithm works in passes, and a vertex can change its block at most once in a pass. The FM algorithm uses two priority queues (one for each block). Initially, it inserts each boundary vertex into the priority queue, along with the gain of moving the vertex to the opposite block. Each step repeatedly performs the move with the highest gain that does not violate the balance constraint and, subsequently, updates the gain of all non-moved neighbors. A pass ends when all vertices are moved, or the balance constraint prevents further moves. The FM algorithm also performs negative gain moves and is therefore able to escape from local optima. In the end, it reverts to best seen solution during the pass.

5.2 Recent Advances in Multilevel Partitioning

This section discusses recent developments in the multilevel partitioning context. We start with two new multilevel partitioning schemes preferable in situations where either the number of blocks is large or high solution quality is required. We then take a closer look at the different phases of multilevel scheme and highlight recent algorithmic improvements.

Deep Multilevel Partitioning. A k -way partition of a (hyper)graph can be obtained either by *recursive bipartitioning* (RB) or *direct k -way partitioning*. The former computes a bipartition of the input (hyper)graph and then recurses on both blocks until the (hyper)graph is divided into the desired number of blocks. The latter partitions the (hyper)graph directly into k blocks and applies k -way local search algorithms to improve the solution.

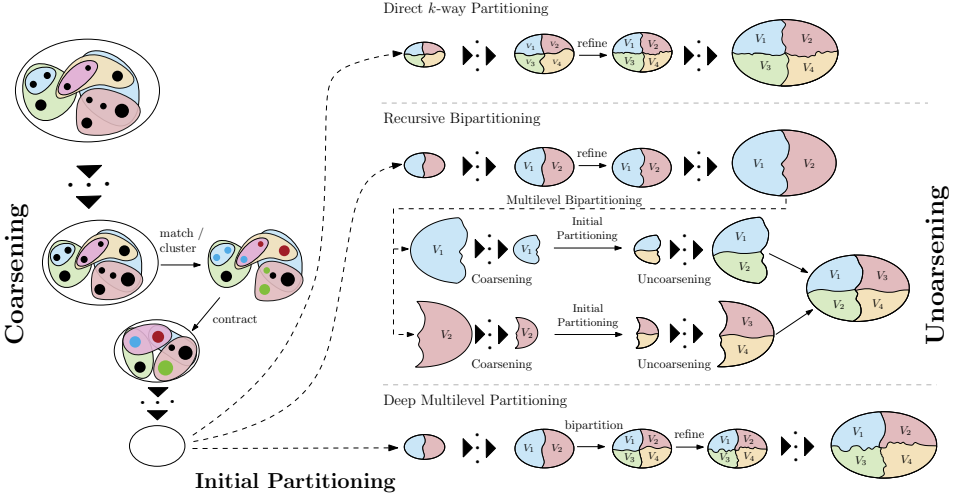


Fig. 3. The multilevel paradigm and its different instantiations to obtain a k -way partition.

Recently, these approaches have been generalized to *deep multilevel partitioning* [86]. The approach continues coarsening until only $2X$ vertices are left (where X is an input parameter) and computes an initial bipartition of the coarsest (hyper)graph. In the uncoarsening phase, it bipartitions a block when its size becomes larger than $2X$ vertices as long as there are less than k blocks.

The deep multilevel approach combines the strengths of the RB and direct k -way scheme: It recursively bipartitions blocks of size $O(1)$ in the uncoarsening phase and thereby enables using k -way local search algorithms, while the RB scheme recursively bipartitions blocks of size $O(k)$ and direct k -way reverts to RB for initial partitioning [4, 20, 117, 188]. Figure 3 illustrates the different instantiations of the multilevel scheme.

n-Level (Hyper)Graph Partitioning. The depth of the multilevel hierarchy offers a trade-off between running time and solution quality of a multilevel algorithm [7]. More levels provide “more opportunities to refine the current solution” [7] at different granularities but require fast local search algorithms to achieve reasonable running times. The most extreme version of the multilevel paradigm is to contract only a single vertex at each level, which induces a hierarchy with almost n levels. This technique was first studied by Osipov et al. [155] for graph partitioning and later improved further by Schlag et al. [4, 182, 183] for hypergraph partitioning. The approach is made feasible by using a highly-localized variant of the classical FM algorithm [64] that initializes the priority queue only with the uncontracted nodes and expands the search to neighbors of moved nodes [155, 175]. Furthermore, the implementation uses a gain cache to avoid expensive recomputations of move gains [4] (reducing the running time of the FM algorithm by 45%) and an adaptive stopping rule that terminates a search early if it becomes unlikely to find further improvements [155] (reducing the running time by an order of magnitude). A parallel version of the n -level scheme exists [84, 85] that uncontracts a fixed number of vertices in parallel on each level (instead of a single vertex)..

Coarsening. The coarsening phase aims to compute successively coarser approximations of the input (hyper)graph such that its structural properties are maintained and act in some sense as a filter that removes as much unnecessary information from the search space as possible [219]. In the past years, research focused on clustering techniques for complex networks and enhancing the coarsening process with information about the community structure of the (hyper)graph.

Meyerhenke et al. [145] use the size-constraint label propagation algorithm [168] to compute a vertex clustering, which is then contracted to form the multilevel hierarchy. Clustering algorithms can reduce the size of complex networks (power-law degree distribution) more efficiently than previously used matching-based approaches. The latter has the problem that vertices incident to high-degree vertices often remain unmatched. Therefore, Davis et al. [48] propose several techniques to reduce the number of unmatched vertices. Two non-adjacent unmatched vertices are matched if they share a common neighbor (also known as 2-hop matching [112]) or if they are adjacent to two vertices that are already matched. Additionally, one can extend two matched vertices with an unmatched neighbor (3-way match).

The following two techniques recompute the (hyper)edge weights of the input (hyper)graph and use them as an input for a multilevel algorithm. The new weights encode more information about the importance of a (hyper)edge and should prevent a coarsening algorithm from collapsing (hyper)edges into a single vertex that are in the cut set of a good partition. Glantz et al. [74] define the weight of an edge $e \in E$ as the minimum conductance value of all bipartitions induced by a spanning tree in which e is a cut edge. The conductance of a bipartition (V_1, V_2) is $cond(V_1, V_2) := \frac{\omega(E'(V_1, V_2))}{\min\{vol(V_1), vol(V_2)\}}$ where $vol(V_i)$ is the weight of all edges incident to vertices in V_i . Chen and Safo [192] use the maximum algebraic distance [44, 171] between two pins of a hyperedge as its weight.

Lotfifar and Johnson [136] compute a hyperedge clustering and assign a vertex to the cluster containing most of its incident nets. Their matching-based coarsening algorithm uses the restriction that matched vertices must be part of the same cluster. Heuer and Schlag [101] also use vertex clustering to restrict contractions to densely-coupled regions of a hypergraph. Their algorithm transforms the hypergraph into its bipartite graph representation and then uses the Louvain algorithm [29] maximizing the modularity objective function to exploit its community structure.

Shaydulin and Safo [193] use ideas from algebraic multigrid as well as stable matching in which the preference lists determine a good combination of nodes for aggregation-based coarsening. The authors integrate their approaches into the Zoltan tool. Their experimental results with Zoltan, hMetis and PaToH demonstrate that given the same refinement, the proposed schemes are at least as effective as traditional matching-based schemes, while outperforming them on many instances.

Initial Partitioning. Coarsening usually proceeds until $\Omega(k)$ vertices remain. Partitioners based on the direct k -way scheme often use multilevel recursive bipartitioning to obtain an initial k -way partition of the coarsest (hyper)graph [4, 20, 117, 188]. Heuer [102] showed that this leads to better initial partitions than flat partitioning techniques. To obtain an initial bipartition, many partitioners run a *portfolio* of different flat bipartitioning algorithms multiple times (e.g., greedy graph growing, random or spectral partitioning) followed by label propagation or FM local search and continue uncoarsening with the best bipartition out of these runs [4, 39, 112]. Preen and Smith [167] showed that the decision when to stop coarsening is often instance-dependent and proposed an adaptive stopping criterion (instead of using the same constant number of vertices for all instances).

Refinement. Sanders and Schulz [176] propose a flow-based refinement technique for bipartitions that is scheduled on block pairs for k -way partitions. The idea is to choose a subset R of vertices around the cut of a bipartition $\Pi = \{V_1, V_2\}$. A flow network is constructed by contracting $V_1 \setminus R$ to the source and $V_2 \setminus R$ to the sink. The resulting maximum flow induces a possibly improved cut which may violate the balance constraint. Therefore, $|R|$ is chosen adaptively and a strongly-connected-component calculation is used to characterize the set of *all* minimum cuts that can then be searched for a balanced bipartition. Heuer et al. [104] generalize this approach to hypergraphs. Gottesbüren et al. [81, 82] accelerate flow-based hypergraph refinement by running the flow algorithm directly

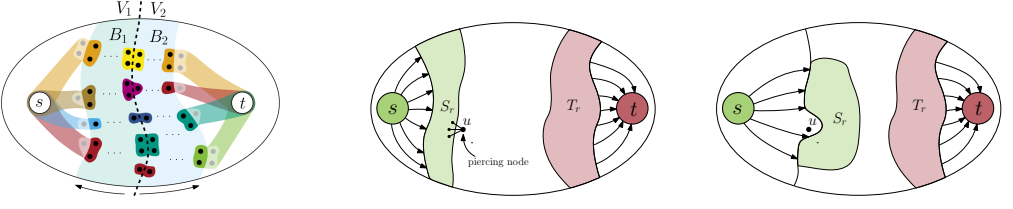


Fig. 4. Left: a flow network that is constructed around a bipartition of a hypergraph. Running a max-flow min-cut algorithm on this network is used to find a reduced cut in the original network. Right: illustration of one step in FlowCutter [92, 225], where the smaller side (source-side) plus a piercing node are contracted to the corresponding terminal.

on the hypergraph. By incorporating FlowCutter [92, 225], which solves incremental flow problems to trade off cut size for better balance, they explore the solution space more effectively, which leads to partitions with smaller cuts/connectivity. Examples are shown in Figure 4. FlowCutter [92, 225] is an iterative algorithm that starts with the minimum cut (and corresponding bipartition) between a given source and sink. If the bipartition is imbalanced, the smaller block is grown by contracting all of its vertices plus an additional vertex (piercing node) to the corresponding terminal. This vertex is chosen incident to the cut, and if possible without increasing the cut-size in the following iteration. The resulting flow problems are nested in the sense that the terminal sets only grow, which makes the flow assignment from the previous iteration feasible but its maximality is violated.

Henzinger et al. [98] use integer linear programming (ILP) to refine k -way partitions directly. Analogously to flow-based refinement, a small region of vertices allowed to move is selected. The remaining vertices are contracted to one super-vertex per block. The ILP formulation is then run on this model graph. An example is shown in Figure 5. Techniques to speed up the ILP are giving a heuristic solution to the solver and symmetry breaking by fixing sufficiently heavy super-vertices to their corresponding blocks. Ugander and Backstrom [208] use linear programming to select from a set of possible moves a subset that yields the largest reduction in cut while satisfying the balance constraint.

In experiments, flow based (hyper)graph refinement turns out to be a key ingredient of high quality partitioning, i.e., it has a much better cost-benefit-ratio than other approaches to improve quality like using ILPs, evolutionary techniques, restarts, V-cycles, etc.

Partitioning With Unevenly Distributed Vertex Weights. Whereas real-world applications often use vertex and edge weights to accurately model the underlying problem, the (hyper)graph partitioning research community commonly works with unweighted instances [103, 182]. Multilevel algorithms incorporate techniques to prevent the formation of heavy vertices (restricting the maximum allowed vertex weight or incorporate the weight of a vertex into the coarsening rating function as a penalty term), but considerably struggle if such vertices are already present in the input [103], as is the case for many real-world instances [9].

There are two approaches to compute balanced k -way partitions under a tight balance constraint in a multilevel partitioner: (i) ensure that initial partitioning finds a balanced partition [103] and that refinement applies node moves to the partition only when they satisfy the balance constraint, or (ii) allow intermediate balance violations [36, 37, 60, 86] and use rebalancing techniques to ensure that the final k -way partition is balanced [86, 137, 178, 218]. Given that finding a balanced k -way partition is a NP-hard problem [70] (reducible to the most common version of the job scheduling problem), both approaches do not guarantee balance.

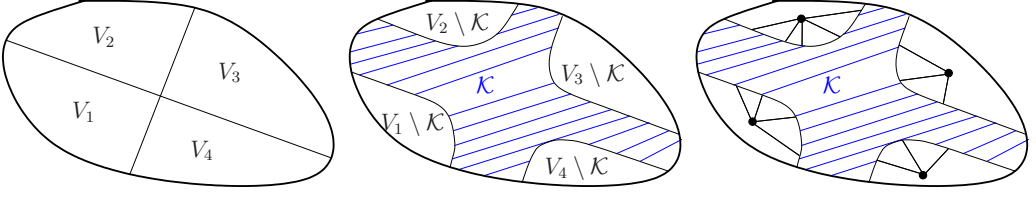


Fig. 5. ILP-based refinement. From left to right: a graph that is partitioned into four blocks; the set $\mathcal{K}$ close to the boundary that will stay in the ILP model; the model in which the sets $V_i \setminus \mathcal{K}$ have been contracted. The latter is used as input to an ILP graph partitioning solver and the improved result is adopted.

Recently, Heuer et al. [103] proposed a technique that enables partitioners based on recursive bipartitioning to reliably compute balanced partitions in practice. The idea is to preassign a small portion of heaviest vertices to one of the two blocks (treated as fixed vertices) and optimize the objective function on the remaining vertices.

5.3 Evolutionary Computation

A genetic or evolutionary algorithm (GA) starts with a population of individuals (in our case, partitions of the (hyper)graph) and evolves the population over several generational cycles or rounds. If a genetic algorithm is combined with local search, then it is called a memetic algorithm (MA) [121]. In each round, the GA uses a selection rule to select good individuals based on the fitness of the individuals of the population and combines them to obtain improved offspring [77]. The combination step is typically done using a recombination operation. When an offspring is generated, an eviction rule is used to select a member of the population to be replaced by the new offspring. For an evolutionary algorithm, it is of major importance to preserve diversity in the population [21]; i.e., the individuals should not become too similar in order to avoid premature convergence of the algorithm. This is usually achieved by using mutation operations and by using eviction rules that take similarity of individuals into account. All of the algorithms in the literature follow a rather generic overall scheme. Hence, we focus on the description of the recombination operations.

Benlic and Hao [24] cluster large sets of vertices together that have been assigned to same block in each individual to perform a recombination operation and combine their operator with tabu search. The recombination operation is motivated by the observation that given a number of high-quality solutions, there is always a high number of vertices that are clustered together throughout these solutions.

Sanders and Schulz introduced a distributed evolutionary algorithm, KaFFPaE (KaFFPaEvolutionary) [177]. The recombination operation uses a modified version of the multilevel graph partitioning solver within KaHIP [176] that will not contract edges that are cut in one of the input partitions. Thus, the better of the two input individuals can be used as initial partitioning and local search can efficiently exchange good parts of solutions on multiple levels of the multilevel hierarchy. Moreover, the recombination operation guarantees that the offspring is at least as good as the better of the two input individuals.

Ruiz and Segura [172] present a memetic algorithm using a weighted matching-based recombination and diversity preservation that is similar to the algorithm by Benlic and Hao [24]. More precisely, given two individuals, the recombination operation computes a matching in a bipartite graph where the left hand side represents the blocks of the first individual and the right hand side represent the blocks of the second individual. Edges between the vertices are weighted by the size of the intersection of the corresponding blocks. In this bipartite graph, a matching is computed and

the corresponding intersections are used as the core of the sets in the new partition (offspring). Moreover, the algorithms use local search with negative cycle detection to further reduce the edge-cut of the computed partitions [179]. When an offspring is inserted into the population, the individual having the highest similarity is evicted. Here, similarity between two individuals is based on the weight of the matching in the bipartite graph defined above.

Henzinger et al. [98] provide a recombination operator that is based on integer linear programming and integrate this into KaFFPaE [177]. The new recombination operation builds and solves an integer linear program from multiple individuals of the population. Roughly speaking, the authors take l individuals (partitions) and build an overlap graph by contracting pairs of nodes that are in the same block in every partition. The original partitioning problem is then solved on the (much smaller) overlap graph using integer linear programming that is initialized with the block affiliations according to the partition that has the lowest cut value. When multiple partitions have the same cut value, one is chosen at random.

Andre et al. [12] generalize KaFFPaE to the multilevel memetic hypergraph partitioner KaHyParE. They also introduce a multilevel multi-point recombination operator that is applied to the best individuals. This operator penalizes contraction of vertices that appear in nets that are cut in many parents. Preen and Smith [167] refine KaHyParE with an adaptive scheme to stop coarsening.

Besides memetic algorithms for hypergraph and graph partitioning, there are also algorithms for other problem variations. For example, Moreira et al. [149] and Popp et al. [164] gave memetic algorithms for acyclic (hyper)graph partitioning in which the input is an acyclic (hyper)graph and the partition has to fulfill an acyclicity constraint. Moreover, Schulz and Sanders [180] gave a distributed evolutionary framework to compute k -way vertex separators in graphs.

5.4 Streaming

We consider a (hyper)graph partitioning algorithm as a streaming algorithm if it satisfies the following conditions: (i) it receives as input the vertices or (hyper)edges of a graph in some sequential order, (ii) it cannot keep the whole (hyper)graph in memory (usually it can only spend linear memory on the number of vertices and blocks), and (iii) it performs permanent partitioning decisions on the fly based on partial information of the (hyper)graph. More specifically, streaming partitioning algorithms usually follow the iterative load-compute-store sequence of operations shown in Figure 6. This sequence of operations can be used to partition either a stream of vertices or a stream of (hyper)edges. Moreover, it operates in batches that can contain either a single vertex/(hyper)edge or multiple of them at a time.

The most common streaming model is the *one-pass* model, where vertices are loaded one at a time alongside with their adjacency lists, then some logic is applied to permanently assign them to blocks. The algorithm logic can be as simple as a Hashing function or as complex as scoring all blocks based on some objective and then assigning the vertex to the block with highest score. There are other streaming models such as the sliding window model [159], where a fixed-size *window* containing the next vertices is kept in memory to help with the one-pass decisions, and the buffered streaming model [62, 109], where *buffers* or *batches* of vertices are consecutively loaded, partitioned at once, and then permanently assigned to blocks.

Besides the traditional streaming graph partitioning approach in which vertices are assigned to blocks [18, 62, 109, 154, 159, 206, 233], many works on streaming graph partitioning adopt an edge-based approach [31, 105, 141, 162, 173]. To disambiguate, we refer to this alternative approach as *streaming edge partitioning* and we refer to the traditional approach either as streaming (hyper)graph partitioning or *streaming vertex partitioning*. In streaming edge partitioning, edges are loaded one by one and directly assigned to blocks on the fly in order to minimize the vertex-cut. From an application perspective, streaming edge partitioning can be used to distribute the workload

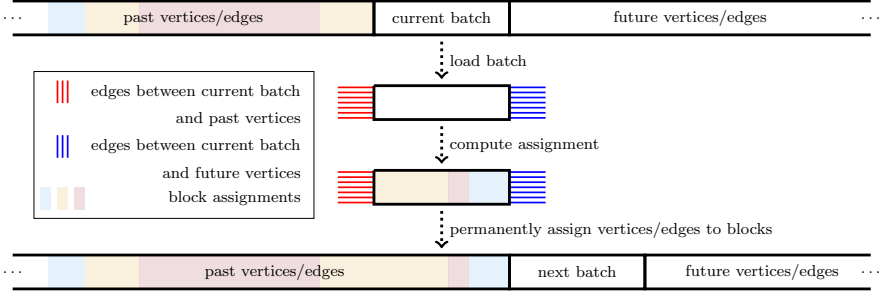


Fig. 6. General iterative structure of streaming algorithms for graph partitioning.

of an application represented by a graph where computations are edge-centric. In this context, the vertex-cut of a partition indicates the number of vertices that will be *replicated* or *mirrored* in multiple blocks, which is usually associated with communication overhead. Another common metric is the *replication factor*, which is defined as the number of replicated vertices divided by the total of vertices in the graph.

5.4.1 One-Pass Streaming Vertex Partitioning. Stanton and Kliot [200] propose heuristics to tackle the graph partitioning problem in the streaming model. Among their most prominent heuristic is the one-pass method *linear deterministic greedy* (LDG) which produces solutions with the best overall edge-cut. In this algorithm, vertex assignments prioritize blocks containing more neighbors and use a penalty multiplier to control imbalance. Particularly, a vertex v is assigned to the block V_i that maximizes $|V_i \cap \Gamma(v)|\Phi(i)$ with $\Phi(i)$ being a multiplicative degrading factor defined as $(1 - \frac{|V_i|}{L_{\max}})$. The intuition is that the degrading factor avoids to overload blocks that are already very heavy. In case of ties on the objective function, LDG moves the vertex to the block with fewer vertices. Overall, LDG partitions a graph in $O(m + nk)$ time. Moreover, the authors also propose a simple one-pass methods based on *hashing*, which has running time $O(n)$ and produces a poor edge-cut.

Later, Stanton [199] studies the streaming vertex partitioning problem from a more theoretical perspective. The author proves that no algorithm can obtain an $o(n)$ -approximation with a random or adversarial stream ordering. Next, two variants of a randomized greedy algorithm are analyzed by using a novel coupling to finite Polya Urn [46] processes, which intuitively explains the performance of the compared algorithms.

Tsourakakis et al. [206] propose Fennel, a simple one-pass partitioning algorithm which adapts the widely-known clustering objective function *modularity* [32]. Roughly speaking, Fennel assigns a vertex v to a block V_i in order to maximize an expression of type $|V_i \cap \Gamma(v)| - f(|V_i|)$, where $f(|V_i|)$ is an additive degrading factor. More specifically, the authors defined the Fennel objective function by using $f(|V_i|) = \alpha\gamma|V_i|^{\gamma-1}$, in which γ is a free parameter and $\alpha = m \frac{k^{\gamma-1}}{n^{\gamma}}$. After a parameter tuning made by the authors, Fennel uses $\gamma = \frac{3}{2}$, which implies $\alpha = \sqrt{k} \frac{m}{n^{3/2}}$. Although the objective function penalizes imbalanced partitions, the authors define the possibility of a hard constraint to enforce balancing. In their experiments, Fennel cuts fewer edges than LDG [200] and, for some instances, it cuts roughly the same amount of edges as an offline partitioning algorithm.

Zhang et al. [233] propose AKIN, a streaming vertex partitioning algorithm for distributed graph storage systems. AKIN is able to partition graphs where the number of nodes n is not known in advance by allowing the migration of vertices between blocks over time. The assignment decisions are mainly based on the *similarity* between vertices, which is evaluated with the *Jaccard similarity*

coefficient [94]. Given the (partial) neighborhoods of two vertices, this coefficient is defined as the ratio of their intersection over their union. Initially, AKIN assumes a *base block* for each vertex, which is given by a hash function. This base block is taken as a preliminary vertex assignment as well as a constant-time index for reaching information in the distributed graph storage. More specifically, for each vertex, AKIN stores in its base block a fixed-length list containing its loaded neighbors with largest degree. This list is used for computing the similarity between vertices. As soon as an edge is loaded, AKIN assigns and migrates it and both of its endpoints to the block which maximizes a similarity-based heuristic. In the experimental evaluation, the version of AKIN keeping up to 100 neighbors per vertex cuts fewer edges compared to Fennel while maintaining equivalent imbalance and spending 10% more running time.

5.4.2 Restreaming Vertex Partitioning. Nishimura and Ugander [154] introduce a restreaming approach to partition vertices. Their approach is motivated by scenarios where the same graph is streamed multiple times. In their model, a one-pass partitioning algorithm can pass multiple times through the entire input while the edge-cut is iteratively reduced. The authors propose ReLDG and ReFennel, which are respective restreaming adaptations of linear deterministic greedy (LDG) [200] and Fennel [206]. On the one hand, ReLDG modifies the objective of LDG to account only for vertex assignments performed during the current pass when computing block weights. On the other hand, ReFennel uses the same objective as Fennel during restreaming, but its additive balancing degrading factor is increased after each pass in order to enforce balance. Additionally, the authors prove that ReFennel converges after a finite number of restreams even without increasing the degrading factor. Their experiments confirm that their restreaming methods can iteratively reduce edge-cut.

Awadelkarim and Ugander [18] investigate how the order in which vertices are streamed influences one-pass vertex partitioning. The authors introduce the notion of *prioritized streaming*, where (re)streamed vertices are statically or dynamically reordered based on some predefined priority. Their approach, which is a prioritized version of ReLDG, uses multiplicative weights of restreaming algorithms and adapts the ordering of the streaming process inspired by balanced label propagation. In their experiments, the authors consider a wide range of stream orderings. The minimum overall edge-cut is obtained using a dynamic vertex ordering based on their own metric *ambivalence*. This approach is closely followed by a static ordering based on vertex degree.

5.4.3 Buffered Streaming Vertex Partitioning. Patwary et al. [159] propose WStream, a simple streaming vertex partitioning algorithm that keeps a sliding window in memory. The authors allow a few hundred vertices in the sliding window in order to obtain more information about a vertex before it is permanently assigned to a block based on a greedy function. As soon as a vertex is allocated to a block, one more vertex is loaded from the input stream into the sliding window, which keeps the window size constant. In their experiments, WStream cuts fewer edges than LDG and more edges than offline multilevel partitioning for most tested graphs.

Jafari et al. [109] perform graph partitioning using a buffered streaming computational model. The authors propose a shared-memory algorithm which repeatedly loads a batch of vertices from the stream input, partitions it using a multilevel scheme, and then permanently assigns the vertices to blocks. Their multilevel scheme is based on a simplified structure where the one-pass algorithm LDG is used for coarsening, computing an initial partition, and refining it. They parallelize LDG in a vertex-centric way by simply splitting vertices among processors, which yields a parallelization of the three steps of their multilevel scheme. In their experiments, their algorithms cuts fewer edges than LDG while scaling better than offline partitioning algorithms.

Faraj and Schulz [62] propose HeiStream, an algorithm which also partitions vertices in a buffered streaming model. Their algorithm loads a batch of vertices, builds a graph model, and then partitions this model with a multilevel algorithm. In their graph model, the vertices from previous

batches assigned to each block are represented as a single big vertex fixed to the respective block. Analogously, an edge between a vertex v from the current batch b and a vertex $\bar{v}$ from a previous batch $\bar{b}$ is represented by an edge $(\bar{b}, b)$. In addition, when a vertex from a current batch has a neighbor from a future batch (i.e., not yet streamed), their model compactly represents this neighbor in a contracted form. Their multilevel algorithm has a traditional structure and components, except that the initial partitioning is a one-pass execution of Fennel and the label propagation refinement also optimizes the objective function used by Fennel to assign vertices to blocks. In particular, the Fennel objective is extended to weighted graphs. In experiments, HeiStream cuts fewer edges than LDG, Fennel, and the buffered streaming algorithm proposed by Jafari et al. [109], while being faster than Fennel for large numbers of blocks.

5.4.4 Streaming Vertex Partitioning for Hypergraphs. Alistarh et al. [6] propose Min-Max, a one-pass streaming algorithm to assign the vertices of a hypergraph to blocks. For each block, this algorithm keeps track of nets which contain pins in it. This implies a memory consumption of $O(mk)$, which is more than the typical memory consumption of a streaming algorithm for graph partitioning. When a vertex is loaded, Min-Max allocates it to the block containing the largest intersection with its nets while respecting a hard constraint for load balance. The authors theoretically prove that their algorithm is able to recover a hidden *co-clustering* with high probability, where a co-clustering is defined as a simultaneous clustering of vertices and hyperedges. In the experimental evaluation, Min-Max outperforms five intuitive streaming approaches with respect to load imbalance, while producing solutions up to five times more imbalanced than internal-memory algorithms such as hMetis.

Taşyaran et al. [202] propose improvements for the algorithm Min-Max [6]. First, the authors present a modified version of Min-Max where, for each net, the blocks containing its pins are stored, instead of storing nets per block as done in Min-Max. In their experiments, this modified version is three orders of magnitude faster than Min-Max while keeping the same cut-net but at the cost of up to two times more memory. The authors also introduce three alternative adaptations using less memory than Min-Max: a constant upper-bound to restrict memory usage, the use of Bloom filters to answer membership queries, and the use of a hashing function to substitute the connectivity information between blocks and nets. In their experiments, their three algorithms reduce the running time in comparison to Min-Max, especially the upper-bound and the hashing approaches, which are up to four orders of magnitude faster. The memory consumption is also reduced in all three algorithms, especially the bloom filter and the hashing approaches, which respectively need one and two orders of magnitude less memory. On the other hand, the three algorithms generate solutions with worse cut-net than Min-Max, especially the hashing approach, which increases the cut-net metric by up to an order of magnitude. Moreover, the authors propose a buffered approach where a modified version of Min-Max goes multiple times through a buffer of vertices before permanently assigning them to blocks. Their experimental results show that the buffered approaches improve partitioning quality in between 5% and 20% over Min-Max at the cost of a significant overhead in comparison with the fast modified versions of Min-Max.

5.4.5 Streaming Edge Partitioning. Bourse et al. [31] study the streaming edge partitioning problem from a theoretical and practical perspective. The authors start by providing two alternative formulations to vertex partitioning: a *basic* formulation whose objective is to minimize the traditional edge-cut metric and an *aggregation* formulation whose objective is to minimize a modified edge-cut metric in which, for each vertex, only one incident cut edge is counted per block. The authors provide two analogous formulations for edge partitioning assuming that each cut vertex is assigned to a *master* block and replicated to the blocks containing its remaining edges. In their *basic* formulation, the objective is to minimize the number of edges incident to replicated vertices,

i.e., edges not assigned to the master block of one of its endpoints. In their *aggregation* formulation, the objective is to minimize the number of replicated vertices. Assuming a uniform random assignment of vertices and edges, the authors show that the expected cost for aggregation-based edge partitioning is always smaller than or equal to the cost for aggregation-based vertex partitioning. Moreover, it is proven that there is a one-to-one correspondence between vertex partitioning and edge partitioning in their basic formulation. As a corollary, there exists an approximate algorithm for edge partitioning which matches the best known approximation ratio for vertex partitioning. Finally, the authors propose greedy heuristics for the streaming version of vertex partitioning and edge partitioning. Their experiments demonstrate the practical advantages of edge partitioning over vertex partitioning and validate their approximation algorithms.

Petroni et al. [162] propose the “High-Degree are Replicated First” (HDRF) algorithm for streaming edge partitioning. The HDRF algorithm, which aims at minimizing vertex-cut, operates by explicitly exploiting highly skewed power-law degree distributions for the edge assignment decision. The intuition behind the algorithm is that high-degree vertices should preferably be cut, i.e., be replicated in several blocks, since there are very few of them in power-law graphs. The authors derive an average-case upper bound for its replication factor. In their experiments, HDRF outperforms all the competitor algorithms with respect to replication factor.

Sajjad et al. [173] propose HoVerCut, a platform for streaming edge partitioning algorithms which can scale in multi-threaded and distributed systems by decoupling the state from the partitioning algorithm. Each thread running HoVerCut receives a unique fraction of the streamed edges and partitions it using any streaming edge partitioning algorithm, such as HDRF [162]. The threads keep *local states* containing specific information required by the streaming algorithm, such as partial degree of processed vertices, partial block weights, and vertex assignments. HoVerCut uses a buffering technique where these local states are shared on demand via a *global state* which is contained in a storage accessible by all threads. All threads repeat the following sequence of steps asynchronously until the whole graph is partitioned: collect a buffer of edges, pull relevant information from global state, partition their buffers, and push local state to global state. In their experiments with real-world and synthetic graphs, HoVerCut consistently speeds up the partitioning process of algorithms such as HDRF while roughly maintaining vertex-cut of the original partitioning algorithms.

Mayer et al. [141] propose ADWISE, a window-based streaming algorithm for edge partitioning. The algorithm keeps a dynamic window of edges and always picks the best edge in the window and assigns it to a block. More specifically, the window starts containing only one edge; then its size dynamically increases or decreases in order to increase partition quality while roughly sticking to a given latency upper-bound. Here, latency is defined as the time needed by the partitioning algorithm to assign an edge in the current window to a block. Thus ADWISE sticks to it by keeping the average latency to assign an edge under control throughout the execution.

Hoang et al. [105] propose CuSP, a distributed and parallel streaming algorithm to partition edges based on user-defined policies. The authors build CuSP as a programmable framework which is able to express the most common streaming edge partitioning strategies used in the literature. The motivation for this approach is the wide variety of available partitioning policies and the lack of an integrated partitioner able to deal with them. In their experiments, CuSP can partition huge graphs with billions of vertices and hundreds of billions of edges in a few minutes, while allowing different partitioning policies.

5.4.6 Experimental Studies. Zainab et al. [1] provide a theoretical and experimental survey on the main one-pass streaming algorithms for vertex partitioning and edge partitioning. After proposing a



Fig. 7. Illustration of the repartitioning process. A graph which has an initial 4-way partition goes through two subsequent changes: Δ^1 where portions of the graph are removed, Δ^2 where new elements (vertices and edges) are inserted in the graph. After each of these changes, a repartitioning routine adapts the previous 4-way partition to the new graph state while optimizing for one or more metrics such as edge-cut, imbalance, and migration of elements between blocks.

classification for these algorithms based on their requirements, formulations, and complexity, the authors experimentally evaluate them with respect to offline metrics such as imbalance, edge-cut, and vertex-cut. Their results indicate that Fennel [206] achieves the lowest edge-cuts while HDRF [31] obtains the lowest replication factors. For edge-cut partitioning algorithms, the authors also present experimental comparisons based on application metrics such as communication cost and total execution time. In their experiments on application performance, HDRF [31] and Greedy [78] yield higher partitioning cost that is compensated by lower computation time for iterative applications and better end-to-end latencies for single-pass stream processing applications. Hashing, on the other hand, has the lowest partitioning time, which increases the overall performance of applications in which data locality does not impact communication cost considerably.

Pacaci and Özsu [156] present an experimental survey of streaming algorithms for vertex and edge partitioning. They investigate performance, resource usage and scalability of the algorithms. The authors study both vertex and edge-partitioning algorithms based on offline graph metrics and online graph queries. Pacaci and Özsu conclude that primarily minimizing edge-cut or vertex-cut fails to represent the workload performance in practice, as some workload characteristics and graph structures are not addressed by these metrics. For example, all tested algorithms suffer from poor workload distribution in practice even when the algorithms theoretically should provide balanced partitions. Furthermore, their experiments show that edge-cut and vertex-cut are reliable indicators of network communication, although this does not always improve the application performance in practice. On the other hand, experiments also show that hashing is an effective strategy for scale-out graph databases since it provides a good trade-off between throughput and latency for online graph queries.

5.5 Repartitioning

Graph repartitioning is an extension of graph partitioning which copes with dynamic graphs, i.e., graphs whose set of vertices and edges are modified over time. Assuming that a graph is partitioned, dynamic changes on its components can make its blocks imbalanced and its edge-cut or vertex-cut larger. Figure 7 illustrates the evolution of a graph over time followed by repartitioning efforts. In general, a rough upper bound for repartitioning quality can be obtained by partitioning the whole graph from scratch every time it is modified. Since this is expensive, many repartitioners use faster approaches.

Vaquero et al. [211] propose a distributed repartitioning algorithm for large-scale graph processing systems. In their algorithm, hashing is used as initial partitioning for all vertices. Based on local information, a label propagation scheme iteratively migrates vertices to blocks where most of their neighbors are located until a convergence is achieved. This procedure natively deals with modifications in the graph. Their experiments show that their approach is able to keep a better

edge-cut and application performance upon graph changes in comparison to the hashing algorithm. The authors did not compare their algorithm against other state-of-the-art repartitioning algorithms.

Xu et al. [224] propose LogGP, a repartitioning algorithm with centralized coordination for graph processing systems. Their algorithm analyzes and reuses historical information to improve the partitioning. Particularly, LogGP builds a hypergraph by combining the graph with hyperedges which represent previous historical partitions. This hypergraph is then partitioned by a novel streaming pin partitioning algorithm. During the execution, the system uses statistical inferences from running logs to optimize the partitioning process. In their experiments, LogGP outperforms state-of-the-art streaming and repartitioning algorithms with respect to edge-cut and application runtime.

Nicoara et al. [153] propose a lightweight repartitioning algorithm and integrate it in their own distributed social network database system Hermes. Within Hermes, the initial partitioning is performed with hashing or Metis, while their repartitioning algorithm is triggered when there is a change in the vertex set of the graph. Repartitioning uses iterative local improvement of balance and edge cut. The associated data is moved only when the improvement process is finished. In their experiments with real-world workloads, their algorithm produces roughly the same edge-cut while migrating up to an order of magnitude less data in comparison with Metis (repartitioning from scratch). Moreover, their algorithm improves around 1.7 times over hashing with respect to aggregate throughput.

Huang and Abadi [108] propose Leopard, an algorithm that solves graph repartitioning while also replicating vertices. Leopard can potentially take advantage of any one-pass vertex partitioning algorithm. This is possible since these algorithms are vertex-centered, so they can be used locally on a dynamic graph at any moment. Moreover, Leopard is the first algorithm to integrate vertex replication with the repartitioning. This replication provides fault tolerance and improves locality, which even increases the partitioning quality, since it is mostly based on local information. Their experiments show that Leopard without vertex replication maintains a partition quality comparable to running Metis from scratch at any moment. On the other hand, Leopard with vertex replication further reduces edge-cut by up to an order of magnitude.

Kiefer et al. [120] propose a repartitioning algorithm optimized for systems whose performance does not scale linearly with the input size. The authors start by formulating a penalized version of the graph partitioning problem by defining the weight of a block as the sum of weights of its vertices plus a penalty function which grows monotonically with the cardinality of the block. In practice, this formulation penalizes resource consumption, which models the nonlinear behavior of the performance of some real-world systems. The authors modify a static multilevel algorithm for graph partitioning to solve the penalized version of the problem, then this multilevel algorithm is used within an adapted version of a hybrid repartitioning strategy. Their hybrid repartitioning algorithm works by triggering a local refinement whenever the balance constraint is violated and computing a whole new partition in the background after a given number of refinements. More specifically, this new partition is computed from scratch; afterward its blocks are mapped onto the blocks of the previous partition to minimize migration costs (see Section 5.6.5 for more details on process mapping). Then it replaces the current partition only if the new edge-cut compensates the migration overhead. The authors implement their algorithms on the Metis [112] framework and show that their static algorithm takes 28% more time than Metis on average while keeping a linear complexity on the amounts of vertices, edges and blocks. Finally, their repartitioning algorithm is able to keep imbalance, edge-cut, and migration time low throughout graph updates.

Fan et al. [61] study the problem of repartitioning vertices (edges) in order to simultaneously minimize edge-cut (vertex-cut) and the modifications to the initial partition. The authors prove robust intractability even for restricted cases of the problem. Moreover, they provide nontrivial

proofs that this problem is unbounded with respect to edge-cut and vertex-cut. Based on the presented theorems, the authors derive a strategy to convert successful partitioning algorithms into repartitioning algorithms while ensuring small migration cost and keeping the same partition quality bounds of the original algorithm. As proof of concept, the authors apply their strategy on algorithms such as Fennel [206] (vertex partitioning) and HDRF [31] (edge partitioning). Finally, the authors validate their algorithm experimentally against the repartitioning algorithm Hermes [153] and algorithms to partition from scratch such as ParMetis [114]. In their experiments, their algorithms are up to an order of magnitude faster than their original counterparts while retaining a comparable or even better partition quality.

5.6 Objective Functions and Problem Variations

The simple balanced (hyper)graph partitioning problems studied in most of this survey does not capture all aspects relevant to applications. Therefore, additional issues such as multiple constraints, directed edges, matrix partitioning, process mapping, and various ways of modelling communication costs have been considered.

5.6.1 Objective Functions. Kaya et al. [118] present an experimental study on the influence of different partitioning models and metrics on the performance of parallel sparse matrix-vector multiplication (SpMxV). To do so, real-life and artificial sparse matrices are partitioned using several matrix partitioning models. The authors then measure the running time of matrix-vector multiplications using three different SpMxV implementations and determine the influence of different partition metrics on the running time by performing a regression analysis. The experimental evaluation shows that *the right* partition model and metric depends on the number of processing elements used, the specific SpMxV implementation and the size and structure of the matrix. More specifically, minimizing the total number of messages sent is more important than minimizing the per-processing element send volume, in particular if the number of processing elements is ≥ 64 . The checkerboard partition model [40] reduces most of the partition metrics and generally achieves the lowest running time.

Originally proposed by Schloegel et al. [186], the multi-objective approach attempts to minimize multiple objective functions simultaneously. To this end, Deveci et al. [52] recently introduced the multi-objective hypergraph partitioner UMPa. While previous work [27, 207] on this problem variation minimized multiple objectives in multiple phases, where the first phase optimizes the primary objective which may not be worsened by later phases, UMPa uses a single-phased approach to optimize a primary metric in combination with a secondary and tertiary metric. This is done by greedily moving boundary vertices such that the primary metric is optimized, while using the secondary (and tertiary) metrics for tie-breaking if multiple eligible target blocks maximize the primary (or secondary) metric. The choice of the best target block of a boundary vertex depends on its gain values, i.e., the change of the metric if the vertex is moved to another block. To this end, the authors describe how gain values can be computed efficiently for the communication volume and number of messages metrics as well as both metrics on a per-block basis.

5.6.2 Multi-constraint Graph Partitioning. Balanced graph partitions are usually constrained by an upper limit on the weight of each block. In the multi-constraint graph partitioning model introduced by Karypis and Kumar [115], each vertex of the graph is associated with a vector of weights, and a balance constraint is imposed on each weight. Multi-constraint partitioning was extended for hypergraphs [43], enabling 2D coarse-grain partitioning of sparse matrices/graphs [40]. Recently, Recalde et al. [170] introduced an exact algorithm for the multi-constraint graph partitioning problem. Two integer programming formulations are provided, and the authors prove several families of inequalities associated with the respective polyhedras. Using a branch-and-bound based

approach, Recalde et al. can solve real-world instances with 30 vertices in approximately half a minute of CPU time. Moreover, several of the proven families of inequalities significantly reduce the number of branch-and-bound vertices and the optimality gap.

5.6.3 Directed Acyclic (Hyper)graph Partitioning. If the input instance is a directed acyclic graph (DAG), one is often interested in finding a partition with an acyclic quotient graph. Moreira et al. [150] show that perfectly balanced graph partitioning is NP-complete even under the additional acyclicity constraint, and that there are no constant-factor approximation algorithms for $k \geq 3$. Herrmann et al. [99, 100] and Moreira et al. [150] both propose multilevel heuristics for DAG partitioning. Popp et al. [164] adapt these techniques to directed acyclic hypergraph partitioning.

Moreira et al. [150] coarsen the graph using size-constraint label propagation [145]. Since this algorithm is not adapted to DAGs, contracting the computed clusters creates coarse graphs with cycles. Thus, the partitioner computes an initial acyclic partition *before* coarsening. Restricting clusters to single blocks then ensures that the partition of the input graph can also be used as a partition of the coarsest graph. Herrmann et al. [99] present an acyclic clustering algorithm which ensures that coarse graphs are acyclic. The top-level of a DAG vertex v is defined as the longest distance from any source of the DAG to v . Based on the top-level of adjacent vertices, conditions for inter- and intra-cluster edges are formulated and it is shown that a clustering respecting those conditions yields an acyclic coarse graph. Moreira et al. compute an initial partition by cutting the topologically ordered vertices of the graph into k chunks of equal size. Herrmann et al. adapt greedy graph growing to initial bipartitioning [100] and initial k -way partitioning [99]. Moreover, bipartitioning the graph as if it was undirected (using, for instance, Metis [112]) and fixing the acyclicity constraint afterwards [99] seems promising. For refinement, both Herrmann et al. and Moreira et al. adapt the FM algorithm to DAG partitioning by restricting the set of possible vertex moves to those that do not violate the acyclicity constraint. Herrmann et al. [99] note that this condition can be checked efficiently if the partition has only two blocks.

5.6.4 Symmetric Rectilinear Matrix Partitioning. Yaşar et al. [228, 229] consider the symmetric rectilinear sparse matrix partitioning problem. Here, the columns and rows of a sparse matrix are partitioned using the same partition vector (in contrast to the more general rectilinear matrix partitioning problem [90, 139], in which different vectors are used). This yields a tiling of the sparse matrix with square tiles on the diagonal. The goal is to find a partition that minimizes the weight (i.e., the sum of all nonzero matrix entries assigned to a tile) of the heaviest tile or, given an upper limit on the weight of a tile, to find a partition subject to the constraint with a minimum number of cuts. The authors show that both problem variations are NP-complete [228], propose heuristics to optimize either objective, and provide efficient implementations thereof. Running time is reduced by sparsifying the matrix. Using these techniques, the algorithm can partition a graph representing the Twitter social network with approximately 1.5 billion edges in less than 3 seconds on a modern 24 core system.

5.6.5 Process Mapping. Distributing a set of parallel and communicating processes among the processing elements (PEs) of a high-performance system is one of the main applications of graph partitioning. Assume these processes and their communication are respectively represented by the vertices and edges of an *application graph*, which can be partitioned in order to split processes among PEs. When different pairs of PEs communicate with each other at different speeds, the edge-cut does not reflect the total communication cost. Process mapping is a generalization of graph partitioning which aims at partitioning a set of processes into blocks of roughly equal size and mapping the blocks onto PEs in order to minimize the *total communication cost* or related objective functions.

The exact total communication cost of a given mapping of processes onto PEs depends on a combination of *bandwidth* and *latency*, which depends on a multitude of factors in practice. Computationally, the total communication cost is usually modeled based on an (implicit) *topology matrix* which contains the distances between PEs. Given a mapping of an application graph onto PEs, the total communication cost is defined as the sum of weights of the cut edges multiplied by the distance between the PEs containing their respective endpoints. The *hierarchical topology* is a special case where PEs are contained in a multi-layered hierarchy of modules and sub-modules. Latency-based topologies are another special kind of topology in which PEs are contained in a *topology graph* such that the distance between a pair of PEs is defined as the number of edges in the shortest path between them. The total communication cost in latency-based topologies is also known as the *Coco(.)* or *hop-byte* objective function. An alternative objective function to the total communication cost is the bandwidth-based metric *maximum congestion*, which is defined as the maximum number of message exchanges through any link of the topology graph. Another possible objective is the *maximum dilation*, which is defined as the maximum communication cost directly associated with a pair of PEs for a given mapping.

The algorithms for process mapping can be categorized in two groups. On the one hand, the single-phase algorithms combine process mapping with graph partitioning [63, 161, 216], such that the objective of the partitioning – commonly edge-cut – is typically replaced by a function that measures communication cost. On the other hand, the two-phase algorithms decouple partitioning and mapping [33, 73, 96, 151, 189]. In a two-phase algorithm, a default graph partitioning algorithm is used to partition a communication graph into k blocks while typically minimizing edge-cut. Afterwards, the quotient graph of the partitioned communication graph is mapped onto PEs in order to minimize the total communication cost or other objective function. The reader is referred to [26, 34] for older works on process mapping.

Deveci et al. [51] propose a greedy construction algorithm and two refinement algorithms to map processes to PEs based on a topology graph. The construction algorithm iteratively picks the process with highest connectivity to the already mapped processes. Then, this process is assigned to the module which minimizes hop-bytes, which is found by performing a breadth-first search on the topology graph. Both their refinement methods are based on process swapping, but one of them aims at minimizing hop-bytes while the other one aims at minimizing congestion. The authors experimentally compare their algorithms on a torus-based system against the default mapper of the system as well as Scotch [160] and LibTopoMap [217]. Their algorithms induce performance increases of 43% and 23% for a communication-only application and a sparse matrix vector multiplication, respectively, compared to the default mapper of the system, and have the best overall results among the competitors.

Deveci et al. [50] also propose a scheme that exploits geometric partitioning to map processors to PEs. The application data and PEs are partitioned separately using the MultiJagged geometric partitioner [53]. Blocks from the data partition are then mapped to corresponding blocks in the PE partition, effectively assigning interdependent data to “nearby” PEs in the network. The method is appropriate for applications that have geometric coordinates as well as graph data.

Vogelstein et al. [212] solve an application-agnostic problem where there are two graphs with the same number of vertices which are bijectively mapped onto one another in order to minimize the number of induced edge disagreements. The authors call this problem *graph matching problem* and formulate it as a quadratic assignment problem (QAP). They solve it with a non-linear approximation algorithm based on the gradient vector. Their algorithm has complexity $O(k^3)$ and performs better than the previous state-of-the-art [195, 209, 232] regarding running time and objective function for over 80% of the QAPLIB benchmark library [35].

Glantz et al. [73] propose two greedy algorithms to bijectively map blocks onto PEs assuming a communication graph. Their most successful algorithm is an adaptation of the greedy mapping algorithm proposed by Müller-Merbach [151]. The algorithm by Müller-Merbach [151] initially computes the total communication volume for each process and also the sum of distances from each PE to all the others. Afterwards, it iteratively assigns the process with largest communication volume to the PE with the smallest total distance. Glantz et al. [73] modify this algorithm by scaling the distance with the amount of communication to be done. This modification improves the overall mapping quality with respect to the quality measures maximum congestion and maximum dilation.

Glantz et al. [75] propose a local improvement algorithm for one-to-one process mapping in which the hardware topology is a partial cube, i.e., an isometric subgraph of a hypercube. The authors exploit the regularity of these topologies to label PEs as well as processes with bit-strings along convex cuts. These bit-strings permit fast computation of distances between PEs and the implementation of effective hierarchical refinement methods to improve the mapping induced by the labels. Their experimental results show that their algorithm reduces the total communication cost of the mappings produced by state-of-the-art mapping algorithms [73, 160] in a range from 6% to 34% while the mapping time stays within the same order of magnitude.

Schulz and Träff [189] solve the process mapping for a hierarchical topology using a two-phase approach. Their algorithm uses the state-of-the-art partitioner KaHIP [174] for the partitioning phase and then a multi-section for the one-to-one mapping phase. The referred multi-section algorithm recursively partitions the quotient graph of the previously obtained partition throughout the layers of the communication topology in a top-down direction. After this algorithm obtains a one-to-one mapping, a light-weight modification of the swap-based refinement method proposed by Brandfass et al. [33] is executed to further minimize the communication cost. Their experiments show that their approach produces mappings with lower total communication cost in comparison with alternative approaches which combine the same KaHIP-based first phase with state-of-the-art algorithms for one-to-one mapping [73, 151, 217]. Kirchbach et al. [122] further improve the two-pass approach by Schulz and Träff [189]. In particular, the authors specialize the partitioning phase to apply KaHIP multiple times, namely throughout the hierarchical topology. Kirchbach et al. experimentally show that their best algorithm is faster while also decreasing the total communication cost in comparison to the algorithm from Schulz and Träff [189].

Faraj et al. [63] propose a single-phase multilevel algorithm to solve the process mapping for a hierarchical topology. Their multi-section scheme uses a top-down recursive multi-section algorithm to the initial mapping. This initial mapping is then improved during the uncoarsening using modified versions of label propagation, k -way FM, multi-try FM, and quotient graph refinement which are specifically designed to minimize the total communication cost. Additionally, their algorithm implicitly represents the hierarchical topology in a condensed binary labeling of PEs, which uses linear space in the number of PEs and provides the distance between PEs in constant time. Experimentally, their algorithm improves the total communication cost over the algorithms proposed by Schulz and Träff [189] and Kirchbach et al. [122] while also having a lower mapping time.

Predari et al. [166] propose a single-phase distributed algorithm to solve the process mapping for a hierarchical topology. The authors model the system hierarchy as an implicit labeled tree and label the vertices of the communication graph in order to implicitly induce the mapping. The algorithm optimizes the mapping by using an adapted version of parallel label propagation where the labeling scheme is used for quick gain computations. In their experiments, the proposed algorithm has good scalability for up to thousands of PEs while achieving a total communication cost smaller than the distributed algorithms ParMetis [114] and ParHIP [146] and comparable cost to the state-of-the-art sequential mapping algorithm proposed by Faraj et al. [63].

Kirchbach et al. [213] address the process mapping problem assuming processes that communicate in a sparse stencil pattern and PEs organized in Cartesian grids. First, the authors prove that the Cartesian mapping is already NP-hard for a two-dimensional Cartesian grid and a one-dimensional stencil. Then three fast construction algorithms which exploit the regularity of the problem are proposed. Their algorithm with best overall results is the *stencil strips* algorithm, which partitions the grid into strips of lengths close to the scaled length of an optimal bounding rectangle of the coordinates of the target stencil. In their experimental evaluation using the `MPI_Neighbor_alltoall` routine of MPI, their algorithms are up to two orders of magnitude faster than general purpose graph mapping tools such as VieM [190] while resulting in a similar communication performance. Moreover, their algorithms are up to three times faster than other Cartesian grid mapping algorithms while resulting in a much better communication performance.

6 PARALLEL ALGORITHMS

Most of the initial work on graph partitioning involved sequential algorithms. These algorithms have been extended to work in distributed memory environments, in particular for balancing processor workloads in parallel applications. In this use case, a distributed memory application already has a distribution of the graph; for memory scalability, the entire graph is not stored in every processor. Thus, distributed-memory partitioning algorithms do not typically have a global view of the entire graph; they often make partitioning decisions based on partial views of local graph data. As a result, they can have lower solution quality than their sequential counterparts. Still, distributed memory algorithms are crucial for graphs that are too large to fit in a single memory space, and for applications wishing to partitioning their data dynamically to adjust for changing computational workloads.

In recent years, progress has been made on shared memory algorithms as shared memory architectures offer greater flexibility than distributed memory architectures. For example, random memory accesses or atomic updates can be done orders of magnitude faster compared to distributed memory machines. Because shared-memory algorithms have a global view of the graph, they can achieve the same solution quality as their sequential predecessors. They are not feasible, however, for extremely large graphs that do not fit in a single memory space.

6.1 Shared Memory

Most of the algorithms discussed in this section are multilevel algorithms. Therefore we structure the discussion into the separate contributions in each phase. The multilevel algorithms we consider are Mt-Metis by Lasalle and Karypis [130], Mt-KaHiP by Akhremtsev et al. [5], Mt-KaHyPar by Gottesbüren et al. [83] (hypergraphs), KaMinPar by Gottesbüren et al. [86] (deep multilevel), and BiPart by Maleki et al. [137]. The following work is focused on transferring the sequential approaches to shared memory as faithfully as possible, while sometimes incorporating lessons learned from distributed memory.

6.1.1 Coarsening. Most coarsening algorithms are based on greedy matching or greedy clustering. These algorithms visit vertices in some order (e.g., random), calculate the ratings (e.g., heavy-edge) for joining neighboring clusters (or match with neighbor), and then join the highest rated cluster. Visiting vertices in parallel yields faithful parallelizations of the sequential approaches. The challenge is to prevent inconsistent clustering decisions between threads.

Çatalyürek et al. [42] propose parallel schemes for agglomerative clustering and greedy matching that use locking, in addition to a lock-free version of greedy matching that resolves conflicts in a second pass. The lock-based algorithms first try to lock the visited vertex, calculate ratings, and then iterate through the candidates. When a better candidate is found, its lock is tested. If successful, the

old candidate is replaced and unlocked. For the lock-free resolution scheme, the matches are stored in a global array M without protecting write access. After one pass, vertices u with $M[M[u]] \neq u$ are matched with themselves $M[u] \leftarrow u$. These are vertices whose match was visited concurrently and matched to a different vertex. In their evaluation, the lock-based algorithms fare equally well as the sequential versions in terms of cardinality and heavy-edge metric, whereas the resolution-based one falls off by 10% in the heavy-edge metric.

The resolution-based matching scheme is also used by LaSalle and Karypis [130] in Mt-Metis. On inputs with skewed degree distributions (such as complex networks), maximal matchings are small and thus coarsening converges too slowly. If too few vertices are matched after a pass, pairs of non-adjacent vertices that have identical neighbors and low degree are matched. If still too few vertices are matched, any vertex pair that shares a common neighbor can be matched. This technique is dubbed two-hop matching [132].

Akhremtsev et al. [5] instead use parallel size-constrained label propagation clustering [144] to coarsen skewed inputs more effectively. No locks are employed, so cyclic cluster joins may occur. The cluster size constraint ensures that initial partitioning can find a feasible partition. Cluster sizes are updated with atomic instructions. The size constraint is checked before the update, but this offers no atomic consistency. Hence, the instructions' results are checked to guarantee the size constraint (and revert if exceeded). For coarsening it is not necessary to strictly adhere to the size constraint (whereas it is for refinement to guarantee balance). Label propagation is also used in KaMinPar [86], but the atomic check is omitted for coarsening (not for refinement).

For Mt-KaHyPar, Gottesbüren et al. [83] use parallel agglomerative clustering but defer the locking after the rating and target cluster selection, locking only the visited vertex and target cluster. Vertices trying to cyclically join each other are detected and resolved on-the-fly, recursively merging the associated clusters.

6.1.2 Initial Partitioning. For initial partitions, LaSalle and Karypis [130] use recursive bipartitioning. Each thread sequentially computes a bipartition, from which the best one is selected. For recursion, the threads are statically split into two groups, working on the two separate subgraphs. This is later improved [132] to threads cooperating on one bipartition if the graph is sufficiently large instead of independent sequential trials. Akhremtsev et al. [5] compute independent k -way partitions with sequential KaHiP [176]. The downsides of these two approaches are potentially severe load imbalance (recursive bipartitioning) or being a sequential bottleneck. To overcome this, Gottesbüren et al. [83] use a work-stealing task scheduler instead of splitting threads for recursive bipartitioning with parallel (un)coarsening. For flat bipartitions, a portfolio of sequential algorithms is run independently in parallel [102, 183].

The deep multilevel approach [86] offers additional parallelism through repetitions (forks) at different coarsening levels. At these stages the graphs may be too small to make efficient use of parallel (un)coarsening, such that splitting threads off for repetitions comes at little to no running time penalty. Using randomized components (coarsening, initial bipartitioning, refinement) thus offers multiple diversified solutions from which the best is chosen.

Slota et al. [196] use parallel k -source BFS to compute flat k -way partitions. Vertices join the blocks of their parents. Their approach does not use coarsening, which justifies using a flat parallel algorithm at this stage.

Maleki et al. [137] parallelize greedy hypergraph growing [38, 112] for bipartitioning. All vertices are assigned to block V_0 . Then the gains of moving the vertices in block V_0 to V_1 are computed. The $\sqrt{|V|}$ highest rated vertices are moved, before the gains of vertices remaining in block V_0 are recomputed. This is repeated until the bipartition is balanced.

6.1.3 Refinement. The two most crucial questions that must be addressed in the refinement phase are how to maintain a balanced partition and how to make sure that concurrent moves do not result in worse cuts. Some refinement algorithms are more difficult to parallelize than others. Unsurprisingly, the more sophisticated techniques, in particular those able to escape local minima, are more difficult to parallelize, as intermediate negative gain moves are required. In particular FM and KL are known to be P-complete [181], which makes the existence of poly-log depth algorithms unlikely.

Label Propagation. The most straightforward approach to parallelize is label propagation: simply visit vertices in parallel. While gains can be incorrect due to concurrent moves in their neighborhood, this does not affect cut optimization too much in practice. Balance can be ensured by using atomic instructions to update block weights [5, 83], or by first collecting moves and approving in a second step [87, 110, 137]. During refinement, it is important to enforce the size constraint by checking the result of the atomic instruction, in order to guarantee a balanced partition.

Greedy. LaSalle and Karypis [130] parallelize Metis' greedy refinement by statically assigning vertices to threads and running the sequential algorithm on the local vertices. The sequential algorithm performs FM on boundary vertices, but stops once no positive gain moves remain. This eases parallelization since no moves must be reverted, and thus there is no serial move order to observe. However, it does eliminate the ability to escape local minima. Moves are communicated to other threads of neighboring vertices via message buffers. The threads frequently check their buffers to update local gains. Further, the refinement is split into an upstream and a downstream pass, where vertices are only allowed to move into blocks with higher (resp. smaller) identifier than their current block. This avoids accidental cut-degrading concurrent moves [114] but restricts the search space since some blocks quickly become close to overloaded. Balance checks are done optimistically with locally updated block weights. Since this may result in balance violations, synchronization points after passes are used to revert some moves to restore balance.

Parallel Localized FM. Localized FM [176] is a variant of FM that starts with a few boundary vertices and expands its search space to neighbors of moved vertices. It is good at escaping local minima due to allowing negative gain moves, but not wasting too much time on unpromising areas through short search sprints. This approach can be parallelized by performing multiple independent localized searches, as opposed to standard FM which is difficult to parallelize efficiently [181]. Akhremtsev et al. [5] organize the boundary vertices in a queue that is randomly shuffled. Threads repeatedly poll seed vertices from the queue and perform localized FM around their seeds. Moves are not communicated to other threads. Instead, each thread maintains a local partition – an array for block weights and a hash table for partition IDs. The rationale for keeping moves private is that reverting (negative gain) moves at the end of a localized search confuses other searches. Searches may overlap in their local vertices, but each vertex is moved only once (atomic test-and-set). Once the global queue is empty, the local move sequences of the threads are concatenated into a single sequence. To ensure a balanced partition and no cut degradation, the gains of this sequence are recomputed sequentially, and the prefix that yields the smallest edge cut subject to the balance constraint is applied.

In Mt-KaHyPar, Gottesbüren et al. [83] apply local move sequences to the global partition as soon as a local optimum is found. This provides more accurate information to the other threads. Additionally, the resulting move order more adequately reflects the partition state on which the move decisions of the localized searches are based. Finally, it reduces the memory for hash tables, while keeping the negative gain moves at the end of a localized search private. This is important for hypergraphs where there is more information (e.g., $\Phi(e, i)$) stored in local partition data structures

than in graph partitioning. Moves applied to the global partition are rejected if they violate the balance constraint and block weights are maintained with atomic instructions.

Parallel Flow-based Refinement. The flow-based refinement from Section 5.2 offers two parallelism sources that are investigated in [88]. The core routine works on bipartitions, such that it can be applied to different block pairs of a k -way partition in parallel. A basic version only runs non-overlapping block pairs in parallel, but the authors show that overlapping searches are feasible with certain restrictions, and necessary for better parallelism. The second parallelism source is the flow algorithm, where the well-known push-relabel algorithm is nicely amenable to parallelization. The approach is integrated in the state-of-the-art parallel multilevel framework Mt-KaHyPar. Experiments show that the partition quality of the new algorithm is on par with the highest quality sequential code (KaHyPar), while being an order of magnitude faster when using 10 threads.

Hill Scanning. LaSalle and Karypis extend their greedy refinement to escape local minima to some extent [131] by employing a simplified variant of localized search. Whenever only negative gain moves remain in the thread-local priority queue, a small group of vertices around u (up to 16) is incrementally constructed, with the hope that moving the entire group reduces the cut. The group is constructed in the same way as localized FM expands its search; however, the expansion stops as soon as an improvement is possible. The selected vertices are not restricted to the thread-local ones, as opposed to the greedy algorithm. One restriction is that all vertices must be moved to the same block.

Techniques for Accurate Gains. Gottesbüren et al. [83] double-check gains for localized FM and label propagation using a technique named *attributed gains*. Recall that $\Phi(e, i)$ denotes the number of pins that hyperedge e has in block V_i . When vertex u is moved from block V_s to V_t , $\Phi(e, s)$ is (atomically) decremented and $\Phi(e, t)$ is incremented for each $e \in I(u)$. Reducing $\Phi(e, s)$ to zero attributes an $\omega(e)$ connectivity reduction, whereas increasing $\Phi(e, t)$ to one attributes an $\omega(e)$ increase to moving u . If the overall attributed gain of a move is negative, it is reverted.

The global moves in localized FM are collected in a sequence in the order in which they were applied. As in Mt-KaHiP, Gottesbüren et al. perform a gain recalculation on this sequence in Mt-KaHyPar, for which a parallelization is proposed [83]. Analogously to attributed gains the last pin of a hyperedge e moved out of a block is attributed an $\omega(e)$ reduction (if no pin moved in before) and the first pin moved into the block is attributed an $\omega(e)$ increase (if none were contained before). This can be calculated in two passes over the pins of e . Each hyperedge incident to a moved vertex is handled independently in parallel, with gain attributions distributed using atomic fetch-and-add.

Finally, the authors [83] show that updating a global gain table [4, 64, 129] can be performed in parallel with atomic instructions. Analogously to attributed gains, gain table updates are triggered by specific observed values when the $\Phi(e, i)$ values are updated. For neighbors of moved vertices their priority queue key is updated with the associated table entries. Further, when a vertex is extracted from the priority queue, its key is checked against the table entries, and reinserted with the new key, if worse. This way, searches gradually update their priority queues to global partition changes without resorting to message passing.

Rebalancing. If vertices are moved concurrently, it does not suffice to check whether each single move would preserve balance, but rather some synchronization mechanism is necessary. Therefore, some refinement algorithms cannot guarantee balanced partitions, so there is a need for explicit rebalancing algorithms. Furthermore, even if some refinement algorithms can guarantee balance, intermediate balance violations can lead to smaller cuts after a rebalancing step.

Slota et al. [196] use label propagation with gains multiplied by $L_{\max}/|c(V_i)|$ to favor moving into lighter blocks [140]. Maleki et al. [137] use the same approach as their graph growing parallelization for rebalancing two-way partitions, starting with a non-empty lighter block that is gradually filled.

Lasalle and Karypis [130] integrate rebalancing into their parallel greedy refinement. The thread-local move buffers are traversed in reverse order and moves into overloaded blocks are reverted. Each thread is responsible for restoring excess weight proportional to the amount it moved into that block.

For large k , Gottesbüren et al. [86] use one priority queue per overloaded block with the ratio of highest gain (to a non-overloaded block) and vertex weight as key. Each queue is filled with just enough vertices to remove the excess weight, but neighbors of moved vertices in the same former block are inserted, for the case that designated target blocks become close to overloaded. Parallelism is achieved by emptying different overloaded blocks in parallel.

Gottesbüren et al. [83] allow controlled balance violations in the gain recalculation step of localized FM in Mt-KaHyPar. Label propagation on the next level is often able to rebalance, and explicit rebalancing similar to label propagation is employed on the finest level.

Parallel n -level (Un)Coarsening. Gottesbüren et al. [85] propose an n -level version [4, 155] of Mt-KaHyPar [83]. With sequential n -level, only one vertex is (un)contracted at a time, which is inherently sequential but offers high solution quality through highly localized refinement. In the parallel version, vertex pairs are contracted asynchronously with vertices to be contracted organized in a hierarchical forest data structure. The forest yields precedence conditions (bottom up) for the asynchronous contractions. With this, vertices in different subtrees can be contracted independently in parallel, as soon as their children are finished. Fine-grained locking is employed to edit the hypergraph data structure, and dynamically maintain consistency of the forest.

Uncoarsening introduces parallelism by uncontracting batches of $b > 1$ vertices in parallel. The batches are constructed by traversing the forest in top-down order, assembling contractions that can be reverted independently in a batch. Uncontracting a batch resolves the last dependencies required to uncontract the next batch. The vertices of the current batch serve as seeds for highly localized parallel refinement (label propagation and FM).

Determinism. Researchers have advocated the benefits of deterministic parallel algorithms for several decades [28, 201], including ease of debugging, reasoning about performance, and reproducibility. A downside is less flexibility in terms of algorithm design choices and potential overheads for synchronizing optimization decisions. For example, the deterministic version of Mt-KaHyPar is a factor of 1.16 slower than an equivalent non-deterministic configuration, and exhibits a factor of 1.029 higher connectivity (both aggregated using geometric mean). Interestingly this degradation stems from coarsening [87], not refinement.

Maleki et al. [137] propose BiPart [137], a deterministic multilevel recursive bipartitioning algorithm. For coarsening, vertices are matched to their smallest hyperedge with ID hashes as tie breakers. All vertices matched to the same hyperedge are contracted without restricting coarse vertex weights. Initial partitioning uses the parallel greedy graph growing described above. For refinement, the gain for moving each vertex to the opposite block is computed in parallel. Let l_0, l_1 denote the number of vertices with positive gain in block V_0, V_1 . Then the $\min(l_0, l_1)$ vertices with highest gains are swapped, an approach that was already proposed for Social Hash [110]. If the hypergraph has unit vertex weights this maintains a balanced partition. However this algorithm is used within the multilevel framework, where non-uniform vertex weights occur from coarsening. Thus, the graph growing rebalancing described above is employed. Note that gains become stale after a neighbor is moved, and thus the overall cut reduction is not the sum of the gains.

Gottesbüren and Hamann [87] present a deterministic version of Mt-KaHyPar [83]. The label propagation style algorithms for preprocessing, coarsening and refinement of Mt-KaHyPar are re-implemented in a deterministic fashion using synchronous local moving [93]. Move decisions do not depend on other moves in the current round. Rounds are further split into sub-rounds to incorporate more up-to-date information. After each sub-round some of the moves are approved and some are denied, for example due to the balance constraint or a cluster size constraint for coarsening. For refinement, the same method as BiPart [137] and SocialHash [110] is used: swapping highest gain prefixes between block pairs. To incorporate non-unit vertex weights, the cumulative weights of all prefixes of the move sequences are computed. The best prefix combination is then selected similar to a parallel merge.

6.2 Distributed-Memory

Distributed (hyper)graph partitioning is one way to handle large inputs that do not fit into the main memory of a single machine. In the distributed memory model, several processors (PEs) are interconnected via a communication network, and each has its private memory inaccessible to others. Computational tasks on each PE usually operate independently only on local data representing a small subset of the input. Intermediate computational results must be exchanged via dedicated network communication primitives.

Distributed (hyper)graph processing algorithms require that the vertices and edges of the input are partitioned among the processors. Since many applications use balanced (hyper)graph partitioning to obtain a good initial assignment, much simpler techniques are used in distributed partitioners. There exist range-based [146] and hash-based partitioning techniques [197, 198]. The former splits the vertex IDs into equidistant ranges, which are then assigned to the PEs. Since both techniques do not consider the (hyper)graph structure, it could lead to load imbalances or high communication overheads. However, one could also migrate vertices as more information about the structure of the (hyper)graph is available, e.g., when recursing on a subgraph obtained via recursive bipartitioning [25, 53]. If geometric information are available, one can also use space-filling curves [19, 214].

Each PE then stores the vertices assigned to it and the edges incident to them. The edges stored on a PE can be incident to local vertices or vertices on other PEs (also called *ghost* or *halo* vertices). We say that a PE is adjacent to another PE if they share a common edge. Processors must be able to identify adjacent PEs to propagate updates, e.g. if we move a vertex to a different block, we have to communicate that change to other PEs in the network such that local search algorithms can work on accurate partition information. However, each communication operation introduces overheads that can limit the scalability of the system. Thus, the main challenge in distributed (hyper)graph partitioning is keeping the global partition information on each PE in some sense up to date while simultaneously minimizing the required communication.

The remainder of this section describes the algorithmic core ideas of recent publications in that field and abstracts from the physical placement of the vertices and the actual representation of the distributed (hyper)graph data structure. However, we assume that each vertex knows on which PE its neighbors are stored.

Local Search. The label propagation heuristic is the most widely-used local search algorithm in distributed systems [54, 110, 111, 140, 146, 197, 204, 218]. Other approaches schedule sequential 2-way FM [64] on adjacent block pairs in parallel [45, 106, 123]. However, this limits the available parallelism to at most the number of blocks k .

Parallel label propagation implementations mostly follow the *bulk synchronous parallel* model. In a computation phase, each PE computes for its local vertices their desired target block. In

the communication phase, updates are made visible to other PEs via personalized all-to-all communication [146, 197]. Meyerhenke et al. [146] use an asynchronous communication model. If the computation phase of a PE ends, it sends and receives updates to and from other PEs and immediately continues with the next round.

In the parallel setting, the move gain of two adjacent vertices may suggest an improvement when moved individually, but moving both simultaneously may worsen the solution quality. Therefore, some partitioners use a vertex coloring [111] or a two-phase protocol where in the first phase, vertices can only move from a block V_i to V_j if $i < j$ and vice versa in the second phase [54, 130, 204]. Many systems do not use any techniques to protect against move conflicts. This can be seen as an optimistic strategy assuming that conflicts rarely happen in practice.

The *Social Hash Partitioner* [110] (Facebook’s internal hypergraph partitioner) also uses the label propagation heuristic to optimize $\text{fanout}(\Pi) := \frac{1}{|E|} \sum_{e \in E} \lambda(e)$ where $\Pi = \{V_1, \dots, V_k\}$ is a k -way partition. The authors note that the label propagation algorithm can easily get stuck in local optima for fanout optimization and suggest a probabilistic version of the fanout metric, called p -fanout(Π) := $\frac{1}{|E|} \sum_{e \in E} \sum_{V_i \in \Pi} 1 - (1 - p)^{\Phi(e, V_i)}$ for some probability $p \in (0, 1)$. The probabilistic fanout function samples the pins of hyperedges with probability p and represents the expected fanout for a family of similar hypergraphs. Thus, it should be more robust and reduce the impact of local minima.

Other recently published distributed local search techniques are based on vertex swapping techniques that preserve the balance of the partition. Rahimian et al. [169] present JA-BE-JA that uses such an approach. The algorithm iterates over the local vertices of each PE and for each vertex, it considers all adjacent vertices as swap candidates. If no partner was found, it selects a random vertex from a sample as a candidate. If the selected vertex is assigned to a different PE, the instantiating PE sends a request with all the required information such that the receiving PE can verify whether or not the swap operation would improve the edge cut. On success, both vertices change their blocks. Additionally, simulated annealing is used to avoid local minima.

Aydin et al. [19] implement a distributed partitioner that computes a linear ordering of the vertices, which is then split into k equally-sized ranges to obtain an initial k -way partition. The idea is similar to space-filling curves [22, 163], but does not require geometric information. The initial ordering is computed by assigning labels to a tree constructed via agglomerative hierarchical clustering. Afterward, it sorts the labels of the leaves to obtain an initial ordering. To further improve the ordering, it solves the *minimum linear arrangement* problem that tries to optimize $\sum_{(u,v) \in E} |\pi(u) - \pi(v)| \omega(u, v)$ where $\pi(u)$ denotes the position of $u \in V$ in the current ordering. To do so, it uses a two-stage MapReduce algorithm that is repeated until convergence: First, each vertex computes its desired new position as the weighted median of its neighbor’s positions. Second, the final positions are assigned to the vertices by resolving duplicates with simple ID-based ordering. The second local search algorithm performs vertex swaps. It splits the vertices of each block into r disjoint sets and pairs adjacent blocks of the partition and randomly their sets. The paired sets are then mapped to the processors that perform the following algorithm: It sorts the vertices in both sets according to their cut reduction if moved to the opposite block and swaps the vertices with the highest combined cut reduction.

Balance Constraint. The label propagation algorithm only knows the exact block weights at the beginning of each computation phase. In the computation phase, block weights are only maintained locally. In the communication phase, the combination of all moves may result in a partition that violates the balance constraint. Thus, partitioners based on this scheme have to employ techniques to ensure balance.

The distributed multilevel graph partitioner ParHIP [146] divides a label propagation round into subrounds and restores the exact block weights with an All-Reduce operation after each subround. Note that this does not guarantee balance but gives a good approximation of the block weights when the number of moved vertices in a subround is small.

Slota et al. [197] implemented a distributed graph partitioner that alternates between a balance and refinement phase, both utilizing the label propagation algorithm. In the refinement phase, each PE maintains approximate block weights $a(V_i) := c(V_i) + \gamma \Delta(V_i)$ where $c(V_i)$ is the weight of block V_i at the beginning of the computation phase, $\Delta(V_i)$ is the weight of vertices that locally moved out respectively into block V_i , and γ is a tuning parameter that depends on the number of PEs. Each PE then ensures locally that $a(V_i) \leq L_{\max}$ for all $i \in \{1, \dots, k\}$. In the balancing phase, the gain of moving a vertex to block V_i is multiplied with $\frac{L_{\max}}{a(V_i)}$. As a consequence, moves to underloaded blocks become more attractive. In a subsequent publication [198], the approach is generalized to the multi-constraint partitioning problem, where each vertex is associated with multiple weights.

Recently, probabilistic methods were proposed that preserve the balance in expectation [110, 140]. The Social Hash Partitioner [110] aggregates the number of vertices $S_{i,j}$ that want to move from block V_i to V_j after each computation phase at a dedicated master process. Then, a vertex part of block V_i is moved to its desired target block V_j with probability $\frac{\min(S_{i,j}, S_{j,i})}{S_{i,j}}$. This ensures that the expected number of vertices that move from block V_i to V_j and vice versa is the same and, thus, preserves the balance of the partition in expectation. However, each PE moves its highest ranked vertices with probability one and all remaining probabilistically. Martella et al. [140] moves a vertex u to its desired target block V_j with probability $\frac{L_{\max} - c(V_j)}{M_j}$ where M_j are the number of vertices that want to move to block V_j . The advantage of the probabilistic method is that only the number of vertices preferring a different block need to be communicated instead of all moves.

Multilevel Algorithms. Although it is widely known that multilevel algorithms produce better partitions than flat partitioning schemes, the systems used in industry, e.g., at Google [19] or Facebook [110, 140], are primarily non-multilevel algorithms. The main reason for this is that the scalability of multilevel algorithms is often limited to a few hundred processors [106, 123]. Furthermore, most parallel multilevel systems implement matching-based coarsening algorithms [45, 54, 106, 123, 204, 220] that are not capable to efficiently reducing the size of today's complex networks (power-law node degree distribution). The most prominent distributed multilevel algorithms are Jostle [220], ParMetis [114], PT-Scotch [45], KaPPa [106], ParHIP [146] and ScalaPart [123] for graph, and Parkway [204] and Zoltan [54] for hypergraph partitioning.

Meyerhenke et al. [146] build the parallel multilevel partitioner ParHIP that uses a parallel version of the size-constraint label propagation algorithm [144]. The algorithm is used to compute a clustering in the coarsening phase and as a local search algorithm in the refinement phase. To obtain an initial partition of the coarsest graph, it uses the distributed evolutionary graph partitioner KaFFPaE [177]. On complex networks, ParHIP computes edge cuts 38% smaller than those of ParMetis [114] on average, while it is also more than a factor of two faster.

Wang et al. [221] use a similar approach that also utilizes the label propagation algorithm to compute a clustering in the coarsening phase. The algorithm is implemented on top of Microsoft's Trinity graph engine [191]. The partitioner additionally uses external memory techniques to partition large graphs on a small number of machines. However, it does not perform multilevel refinement (the initial partition is projected to the input graph).

Geometric Partitioners. Many graphs are derived from geometric applications and are enriched with coordinate information (e.g., each vertex is associated with a d -dimensional point). A mesh with coordinate information and a partition based on this coordinates is shown in Figure 8. Geometric

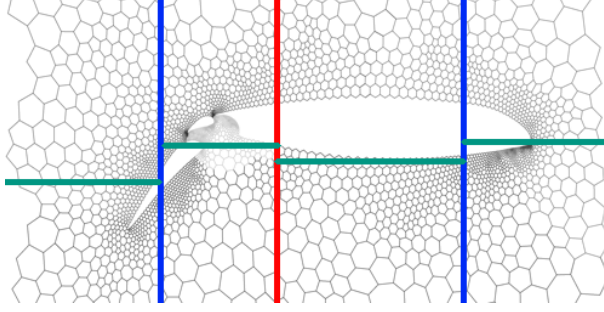


Fig. 8. A visualization of an airfoil. The graph has coordinates for each vertex. Geometric partitioning algorithms use this type of information for partitioning. In the example above, recursive coordinated bisection (always split the graph along the shorter axis) has been applied to derive a partition.

partitioning techniques use this information to partition the corresponding point set into k equally-sized clusters while minimizing an objective function defined on the clusters. The objective function should be chosen such that it implicitly optimizes the desired graph partitioning metric (e.g., sum of the lengths of all bounding boxes approximates the total communication volume [53]). Since geometric methods ignore the underlying structure of the (hyper)graph, the quality of the partitions is often inferior compared to traditional multilevel algorithms. However, these algorithms are often simpler leading to faster and more scalable algorithms. Prominent techniques use space-filling curves [22, 163] that maps a set of d -dimensional points to an one-dimensional line. A fundamental property of this curve is that points that are close in the original space are also close on the line. Other approaches recursively divide the space via cutting planes such as Octree-based partitioning [147], recursive coordinate bisection [25, 194], and recursive inertial bisection [203, 223]. The MultiJagged algorithm of Deveci et al. [53] uses multisectioning rather than bisection to reduce the depth of the recursion and speed computation relative to recursive coordinate bisection; its hybrid implementation uses MPI and Kokkos [205] to support both distributed memory message passing between PEs and multithreading or GPU computation within PEs.

Recently, Von Looz et al. [214] presented a scalable balanced k -means algorithm to partition geometric graphs. The k -means problem asks for a partition of a point set P into k roughly equally-sized clusters such that the squared distances of each point to the mean of its cluster is minimized (in the following also referred to as the center of a cluster). Clusters obtained with this problem definition tend to have better shapes than computed with previous methods and also produce better partitions when measured with graph metrics [143]. They present a parallel implementation of Lloyd's greedy algorithm [135] that repeats the following steps until convergence. First, each point $p \in P$ is assigned to the cluster that minimizes the distance of p to its center. Afterwards, the center of each cluster is updated by calculating the arithmetic mean of all points assigned to it. To achieve balanced cluster sizes, an influence factor γ_c is introduced individually for each cluster c and $\text{eff_dist}(p, \text{center}(c)) := \text{dist}(p, \text{center}(c)) / \gamma_c$ is used as the distance of point p to the center of a cluster c . If a cluster c becomes overloaded, the influence factor γ_c is decreased and otherwise it is increased. Thus, underloaded clusters become more attractive. The implementation replicates the cluster centers and influence factors globally and after each computation phase, it updates the values via a parallel sum operation. To obtain an initial solution, it sorts the points according to the index on a space-filling curve and splits the order into k equally-sized clusters. Furthermore, it establishes a lower bound for the distances of each point to the second-closest cluster which allows to skip expensive distance computations for most of the points. Additionally, each processor sorts

the cluster centers according to their distances to a bounding box around the process-local points. Evaluating the target clusters in increasing distance order allows to abort early when the minimum distance of the remaining clusters is above the distance of already found candidates.

Scalable Edge Partitioning. Schlag et al. [184] present a distributed algorithm to solve the edge partitioning problem. The edge partitioning problem asks for a partition $\Pi = \{E_1, \dots, E_k\}$ of the edge set into k blocks each containing roughly the same number of edges, while minimizing the vertex cut $\sum_{v \in V} \rho(v) - 1$ where $\rho(v) = |\{E_i \mid E_i \in \Pi : I(v) \cap E_i \neq \emptyset\}|$. They evaluated two methods to solve the problem. The first transforms the graph into its dual hypergraph representation (edges of the graph become vertices of the hypergraph and each vertex of the graph induces a hyperedge spanning its incident edges). Using a hypergraph partitioner that optimizes the connectivity metric to partition the vertex set directly optimizes the vertex cut of the underlying edge partitioning problem. The second method uses a distributed construction algorithm of the so-called *split-and-connect* graph (SPAC). For each vertex u , it inserts $d(u)$ auxiliary vertices into the SPAC graph and connects them to a cycle using auxiliary edges each with weight one. Each auxiliary vertex is a representative for exactly one incident edge of u . For each edge $(u, v) \in E$, it adds an infinite weight edge between the two representatives of the corresponding edge. Thus, a partition of the vertex set of the SPAC graph cannot cut an edge connecting two representatives. Therefore, such a partition can be transformed into an edge partition by assigning each edge to the block of its representatives. In the evaluation, they compare both representations while using different graph and hypergraph partitioners. The results showed that parallel graph partitioners outperform distributed hypergraph partitioners. However, the sequential hypergraph partitioner KaHyPar [182] produces significantly better vertex cuts than all other approaches (more than 20% better than the best graph-based approach), but is an order of magnitude slower than the evaluated distributed algorithms.

6.3 GPU

Due to their high computational power, modern GPUs have become an important tool for accelerating data-parallel applications. However, due to the highly irregular structure of graphs, it remains challenging to design graph algorithms that efficiently utilize the SIMD architecture of modern GPUs.

Multilevel Graph Partitioning. Goodarzi et al. [79, 80] present two algorithms for GPU based multilevel graph partitioning. Their earlier approach [79] uses heavy-edge matching for coarsening and transfers the coarsest graph onto CPU for initial partitioning (using Mt-Metis [113]). During refinement, vertices are distributed among threads and each thread finds the blocks maximizing the gain values of its assigned vertices. To prevent conflicting moves that worsen the edge cut in combination, refinement alternates between rounds in which only moves to blocks with increasing (resp. decreasing) block IDs are considered. For each block, potential moves to the block are collected in a global buffer, which is then sorted, and the highest rated moves are executed.

Their later approach [80] brings several improvements. First, the authors use Warp Segmentation [119] to improve the efficiency of the heavy-matching computation during coarsening. Initial partitioning is then performed on the GPU using a greedy growing technique. During refinement, vertices are once more divided among threads, and each thread finds the blocks maximizing the gain of its assigned boundary vertices and collects the potential moves in a global buffer. Then, the algorithm finds the highest rated ℓ moves in the global buffer, for some small input constant ℓ . Since moves might conflict with each other, their algorithm distributes all 2^ℓ move combinations across thread groups and finds the best combination, which is then applied to the graph partition. This process is repeated until the global buffer is empty. On average, their GPU based approach

is approximately 1.9 times faster than Mt-Metis while computing slightly worse edge cuts across their benchmark set of 16 graphs.

Fagginger Auer and Bisseling [17] present a fine-grained shared-memory parallel algorithm for graph coarsening and apply this algorithm in the context of graph clustering to obtain a fast greedy heuristic for maximising modularity in weighted undirected graph. The algorithm is suitable for both multi-core CPUs and GPUs. Later, Gilbert et al. [72] present performance-portable graph coarsening algorithms. In particular, the authors study a GPU parallelization of the heavy edge coarsening method. The authors evaluate their coarsening method using a multilevel spectral graph partitioning algorithm as primary use case.

Spectral Graph Partitioning. The availability of efficient eigensolvers on GPUs has lead to a recent re-emergence of spectral techniques for graph partitioning on GPU systems [2, 3, 152]. These techniques were first developed by Donath and Hoffman [58, 59] and Fiedler [65] to compute graph bisections in the 1970s. Subsequently, these techniques have been improved [23, 30, 97, 165, 194] and extended to partition a graph into more than two blocks using multiple eigenvectors [10, 97].

Naumov and Moon [152] present an implementation of spectral graph partitioning for single GPU systems as part of the nvGRAPH library, whereas Acer et al. [2, 3] propose the multi-GPU implementation Sphynx. Both partitioners precondition the matrix and use the LOBPCG [124] eigenvalue solver. The eigenvectors are then used to embed the graph into a multidimensional coordinate space, which is then used to derive a partition of the graph. In nvGRAPH, this is done using a k -means clustering algorithm on the embedded graph, whereas Sphynx uses the geometric graph partitioner Multi-Jagged [53] which supports multi-GPU systems. Since the approach by Acer et al. outperforms nvGRAPH in terms of partition balance, cut size and running time (when run on a single GPU system), we focus on their experimental evaluation. To this end, when Sphynx is compared against ParMETIS [116], ParMETIS generally obtains significantly better cuts than Sphynx — approx. 20% (resp. 70%) lower cuts on regular (resp. irregular) graph instances). On irregular graphs, the authors report a significant speedup of approximately 19 using Sphynx on 24 GPUs compared to ParMETIS with 168 MPI processes across four compute nodes, although ParMETIS is approximately 3 times faster than Sphynx on regular graphs even when using a single CPU core for each GPU used by Sphynx. Additionally, Acer et al. report the influence of several matrix preconditioners on different classes of graphs.

6.4 Approaches for Other Types of Hardware

Recently, Ushijima-Mwesigwa [210] et al. explore graph partitioning using quantum annealing on the D-Wave 2X machine. The main idea is to formulate the graph partitioning problem as a quadratic unconstrained binary optimization problem via a spectral approach. These problems can be mapped to minimizing Ising objective functions which the D-wave system can minimize. The method could run directly on the D-Wave system for small graphs and used a hybrid approach for large graphs.

Lio [134] et al. experiment with solving the graph partitioning problem on the Fujitsu Digital Annealer which is a special-purpose hardware designed for solving combinatorial optimization problems. In particular, the authors also model the problem as a quadratic unconstrained binary optimization problem via a spectral approach. The Fujitsu Digital Annealer then utilizes application-specific integrated circuit hardware for solving fully connected quadratic unconstrained binary optimization problems. The authors identify a dense network for which their approach significantly outperforms KaffpaE. Then a range of similar instances is created using the MUSKETEEER [91] framework. On most of those instances, their approach outperforms KaffpaE and Gurobi.

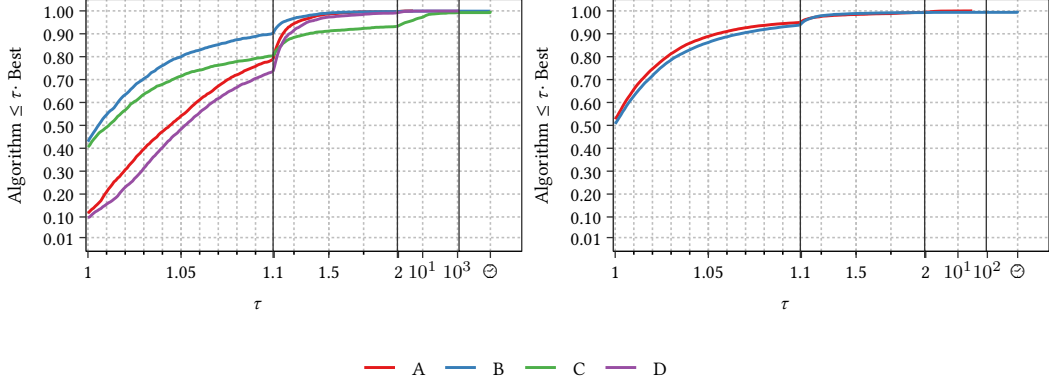


Fig. 9. Performance profiles comparing four different algorithms (left) and the result of an effectiveness test between two algorithms (right). The $\otimes$ tick marks instances for which the corresponding algorithm has computed an infeasible solution or has ran into a time limit.

7 EXPERIMENTAL METHODOLOGY

As more applications of (hyper)graph partitioning have emerged in recent years, it becomes increasingly important to evaluate algorithms on many (hyper)graph instances to demonstrate their effectiveness for practical applications. The presentation of results using tables comparing the running time and solution quality of different algorithms can quickly become difficult to interpret, while evaluations based on aggregated numbers can lead to misleading conclusions. This section discusses several alternative methods helpful for presenting and interpreting experimental results.

Performance Profile. Performance profiles can be used to compare the solution quality of different algorithms [56]. Let $\mathcal{A}$ be the set of all algorithms, $\mathcal{I}$ the set of instances, and $q_A(I)$ the value of a quality metric (e.g., edge-cut) of algorithm $A \in \mathcal{A}$ on instance $I \in \mathcal{I}$. For each algorithm A , performance profiles show the fraction of instances (y -axis) for which $q_A(I) \leq \tau \cdot \text{Best}(I)$, where τ is on the x -axis and $\text{Best}(I) := \min_{A' \in \mathcal{A}} q_{A'}(I)$ is the best solution produced by an algorithm $A \in \mathcal{A}$ on an instance $I \in \mathcal{I}$. For $\tau = 1$, the y -value indicates the percentage of instances for which an algorithm $A \in \mathcal{A}$ performs best. Achieving higher fractions at smaller τ values is considered better.

Figure 9 (left) compares a quality metric of four different algorithms using a performance profile. Algorithm A and C compute on roughly 40% of the instances the best solutions while algorithm B and D only on 10% (see $\tau = 1$). The solutions produced by algorithm A are worse than the best solutions by $\approx 4\%$ in the median (intersection of $y = 0.5$ with the red line is at $\tau \approx 1.04$). If we compare algorithm C and D based on their geometric mean of the quality metric, we would observe that algorithm C is only 0.2% better than D. Hence, we would probably conclude that there is no significant difference between both. If we look at the performance profile, we see that algorithm C is on most of the instances closer to the best solution than D. However, on $\approx 10\%$ of the instances the solutions are worse than the best once by more than a factor of two, which has large influence on its geometric mean (see green line at $\tau = 2$).

Effectiveness Tests. The performance profile in Figure 9 (left) suggests that algorithm B produces solutions that are better than those of A. However, if we look at their running times, we observe that algorithm A is more than an order of magnitude faster than B on average. Thus, algorithm B may have an unfair advantage due to its longer running time. Therefore, Akhremtsev et al. [5] introduces *effectiveness tests* to compare solution quality when two algorithms are given a similar running time, by performing additional repetitions with the faster algorithm. Consider two algorithms A

and B, and an instance I . Effectiveness tests first sample one run of both algorithms. Let t_A^1 and t_B^1 be their running times and assume that $t_A^1 \geq t_B^1$. Then, additional runs of B are sampled until the accumulated time exceeds t_A^1 . Let $t_B^2, \dots, t_B^l$ denote their running times. The last run is accepted with probability $(t_A^1 - \sum_{i=1}^{l-1} t_B^i) / t_B^l$ so that the expected time for the runs of B equals t_A^1 . The quality metric taken is the minimum out of all runs.

Figure 9 (right) shows the result of the effectiveness test for algorithm A and B using a performance profile. The plot shows that there is no significant difference between the two algorithms.

Significance Tests. It is not immediately obvious from the performance profile in Figure 9 (left) whether algorithm B performs better than C or vice versa since both lines are close to each other. Here, statistical tests can give further information that helps to decide whether the difference between two algorithms is statistically significant.

A widely used technique is to formulate a *null hypothesis* assuming that there is no difference between the observed distributions. A significance test then calculates the probability (p -value) that the observed distribution occurs under the assumption that the null hypothesis is true [230]. The null hypothesis is rejected if the probability is below a predefined significance level α (e.g., $\alpha = 5\%$ as suggested by Fisher [68]). To compute the p -value, the Wilcoxon signed rank test [222] is often used to compare two algorithms while the Friedman test [69] makes pairwise comparisons between multiple algorithms.

Significance tests based on a null hypothesis decide if the difference between two measurements is statistically significant but do not reveal any information whether or not the difference is relevant in practice. For example, a significance test comparing the running times of two algorithms A and B may conclude that A is faster than B. However, A might be only 1% faster on average and, thus, the relevance of the improvement is questionable. Therefore, Angriman et al. [14] recommend using *parameter estimation* instead of hypothesis testing. To apply parameter estimation to our example, we could compute the running time ratios $t_A(I)/t_B(I)$ for each instance I and determine the *confidence interval* that contains 95% of the measurements. The confidence interval gives additional information on the *effect size* of the improvement.

8 FUTURE CHALLENGES

While there has been a considerable progress in the field in the last decade, there is a wide range of challenges that remain open. It is an interesting question to what extent the multitude of results sketched above have reached a state of maturity where future improvements become less and less likely. Algorithms like multi-threaded graph partitioning for balanced graph partitioning using a small number of threads and standard inputs may be difficult to improve. Long time open problems like large gaps between theory and practice may remain open for a long time. On the other hand, other important issues have considerable potential. We try to identify some of them below.

Parallelism and Other Hardware Issues. Scalability of high quality parallel (hyper)graph partitioning remains an active area of research. In particular, achieving good scalability and quality on large distributed memory machines is still a challenge, but even on shared-memory machines, scalability to a large number of threads seems difficult. Even more difficult is aligning the inherent complexity and irregularity of state-of-the-art algorithms with the restrictions of GPUs or SIMD-instructions. Another conundrum is that, for good memory access locality during partitioning, (hyper)graphs need to already be partitioned reasonably well.

Hierarchies of supercomputers have to be taken into account during partitioning. This can be done by using multi-recursive approaches taking the system hierarchy into account or by adapting the deep multilevel partitioning approach sketched above to the distributed memory case. When arriving at a compute-node level, additional techniques are necessary to employ the full

capabilities of a parallel supercomputer. For example, many of those machines have GPU's on a node level. Recently, researchers started to develop partitioning algorithms that run on GPUs and while of independent interest, partitioning algorithms developed for this type of hardware can help in that regard. Hence, future parallel algorithms have to compute partitions on and for heterogeneous machines. On the other hand, algorithms should be energy-efficient and performance per watt has to be considered. Lastly, future hardware platforms have to be taken into consideration when developing such algorithms. One way to achieve this will be to use performance portable programming ecosystems like the Kokkos library [205].

Problem Variations. Current partitioning algorithms work well on a wide range of instances. Most of these instances are either unweighted and translate into well behaved weighted problems if a multilevel algorithm is used or instances have a fairly even distribution of node weights. For instances with vastly different values of node weights, partitioning problems discussed in this paper get close to bin packing problems. Currently solvers are not able to handle this case well.

Algorithms for the well established objective functions like edge cut (in the graph case) or connectivity/cut (in the hypergraph case) are able to compute very high-quality partitions. Many open problems remain when optimizing for other or multiple objectives and when the problem has other/multiple constraints. For example, in parallel computing, bottleneck objectives such as minimizing the maximum edge cut of a block should perform better, but few partitioners optimize for such objectives. The same is true for high-quality solvers for problems with multiple constraints in the case of repartitioning. Streaming algorithms currently compute much worse quality partitions than internal memory partitioning algorithms. This is partially due to the fact that such algorithms don't have a global view of the optimization problem. Improving quality of streaming algorithms is an open problem. Lastly, we believe that directed variants of the problems will become more important in order to model different applications.

Multilevel Partitioning. The multilevel technique has been incredibly successful in the field of decomposition. Yet, multilevel algorithms in the area still consist in practice of a very limited number of multilevel techniques. Development and understanding components of more sophisticated coarsening schemes, edge ratings, and metrics of nodes' similarity that can be propagated throughout the hierarchies are among the future challenges for (hyper)graph partitioning as well as any attempt of their rigorous analysis. For example, coarsening at the moment is typically stopped based on simple formulas. However, in practice it may be much more fruitful to derive stopping rules for coarsening that take the given instance into account. This also translates to different parameters of sub-algorithms that should be chosen based on the type of input that is provided to the algorithm.

Modeling and Experiments. Traditionally instances for partitioning algorithms came from scientific simulations and thus were fairly well structured. For these networks, edge cut and communication volume of the application have been highly correlated. However, for instances having much more complex structures like social networks that recently became important in practice, the correlation is not that high. It remains an open question how different models of (hyper)graph partitioning translate into concrete performance of applications.

Acknowledgements. We would like to thank Ilya Safro for feedback and additional references on an early version of the manuscript. We acknowledge support by DFG grants SCHU 2567/1-2, WA654/19-2, SA933/10-2, SA933/11-1, and NSF grant CCF-1919021.

REFERENCES

- [1] Z. Abbas, V. Kalavri, P. Carbone, and V. Vlassov. Streaming graph partitioning: An experimental study. *PVLDB*, 11(11):1590–1603, 2018. doi:10.14778/3236187.3236208.
- [2] S. Acer, E. G. Boman, C. A. Glusa, and S. Rajamanickam. Sphynx: A parallel multi-gpu graph partitioner for distributed-memory systems. *Parallel Comput.*, 106:102769, 2021. URL: <https://doi.org/10.1016/j.parco.2021.102769>,

doi:10.1016/j.parco.2021.102769.

- [3] S. Acer, E. G. Boman, and S. Rajamanickam. SPHYNX: spectral partitioning for hybrid and axelerator-enabled systems. In *IEEE International Parallel and Distributed Processing Symposium Workshops, IPDPSW*, pages 440–449, 2020. URL: <https://doi.org/10.1109/IPDPSW50202.2020.00082>, doi: 10.1109/IPDPSW50202.2020.00082.
- [4] Y. Akhremtsev, T. Heuer, P. Sanders, and S. Schlag. Engineering a direct k -way hypergraph partitioning algorithm. In *19th Workshop on Algorithm Engineering and Experiments (ALENEX)*, pages 28–42. SIAM, 01 2017. doi:10.1137/1.9781611974768.3.
- [5] Y. Akhremtsev, P. Sanders, and C. Schulz. High-quality shared-memory graph partitioning. *IEEE Transactions on Parallel Distributed Systems*, 31(11):2710–2722, 2020. doi:10.1109/TPDS.2020.3001645.
- [6] D. Alistarh, J. Iglesias, and M. Vojnovic. Streaming min-max hypergraph partitioning. In *Advances in Neural Information Processing Systems*, pages 1900–1908, 2015. URL: <https://dl.acm.org/doi/10.5555/2969442.2969452>.
- [7] C. J. Alpert, J.-H. Huang, and A. B. Kahng. Multilevel Circuit Partitioning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 17(8):655–667, 1998. doi:10.1109/43.712098.
- [8] C. J. Alpert and A. B. Kahng. Recent Directions in Netlist Partitioning: A Survey. *Integration, the VLSI Journal*, 19(1-2):1–81, 1995.
- [9] C. J. Alpert. The ISPD98 circuit benchmark suite. In M. Sarrafzadeh, editor, *Proceedings of the 1998 International Symposium on Physical Design, ISPD 1998, Monterey, CA, USA, April 6-8, 1998*, pages 80–85. ACM, 1998. URL: <https://doi.org/10.1145/274535.274546>, doi: 10.1145/274535.274546.
- [10] C. J. Alpert and S. Yao. Spectral partitioning: The more eigenvectors, the better. In *32st Conference on Design Automation*, pages 195–200. ACM Press, 1995. URL: <https://doi.org/10.1145/217474.217529>, doi: 10.1145/217474.217529.
- [11] Z. An, Q. Feng, I. Kanj, and G. Xia. The complexity of tree partitioning. *Algorithmica*, 82(9):2606–2643, 2020.
- [12] R. Andre, S. Schlag, and C. Schulz. Memetic multilevel hypergraph partitioning. In *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO 2018, Kyoto, Japan, July 15-19, 2018*, pages 347–354. ACM, 2018. URL: <https://doi.org/10.1145/3205455.3205475>, doi: 10.1145/3205455.3205475.
- [13] K. Andreev and H. Räcke. Balanced Graph Partitioning. *Theory of Computing Systems*, 39(6):929–939, 2006. doi: 10.1007/s00224-006-1350-7.
- [14] E. Angriman, A. van der Grinten, M. von Looz, H. Meyerhenke, M. Nöllenburg, M. Predari, and C. Tzovas. Guidelines for Experimental Algorithmics: A Case Study in Network Analysis. *Algorithms*, 12(7):127, 2019. URL: <https://doi.org/10.3390/a12070127>, doi: 10.3390/a12070127.
- [15] A. Archer, K. Aydin, M. Bateni, V. S. Mirrokni, A. Schild, R. Yang, and R. Zhuang. Cache-aware load balancing of data center applications. *Proceedings of the VLDB Endowment*, 12(6):709–723, 2019. doi:10.14778/3311880.3311887.
- [16] A. Atrey, G. Van Seghbroeck, H. Mora, B. Volckaert, and F. De Turck. Unifydr: A generic framework for unifying data and replica placement. *IEEE Access*, 8:216894–216910, 2020. doi:10.1109/ACCESS.2020.3041670.
- [17] B. O. F. Auer and R. H. Bisseling. Graph coarsening and clustering on the gpu. In *Graph Partitioning and Graph Clustering*, 2012.
- [18] A. Awadelkarim and J. Ugander. Prioritized restreaming algorithms for balanced graph partitioning. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1877–1887, 2020. doi:10.1145/3394486.3403239.
- [19] K. Aydin, M. Bateni, and V. S. Mirrokni. Distributed balanced partitioning via linear embedding. *Algorithms*, 12(8):162, 2019. doi:10.3390/a12080162.
- [20] C. Aykanat, B. B. Cambazoglu, and B. Uçar. Multi-Level Direct K-Way Hypergraph Partitioning with Multiple Constraints and Fixed Vertices. *J. Parallel Distributed Comput.*, 68(5):609–625, 2008. URL: <https://doi.org/10.1016/j.jpdc.2007.09.006>, doi: 10.1016/j.jpdc.2007.09.006.
- [21] T. Bäck. *Evolutionary algorithms in theory and practice - evolution strategies, evolutionary programming, genetic algorithms*. Oxford University Press, 1996.
- [22] M. Bader. *Space-Filling Curves - An Introduction with Applications in Scientific Computing*, volume 9 of *Texts in Computational Science and Engineering*. Springer, 2013. URL: <https://doi.org/10.1007/978-3-642-31046-1>, doi: 10.1007/978-3-642-31046-1.
- [23] S. T. Barnard and H. D. Simon. A Fast Multilevel Implementation of Recursive Spectral Bisection for Partitioning Unstructured Problems. In *Proc. of the 6th SIAM Conference on Parallel Processing for Scientific Computing*, pages 711–718, 1993.
- [24] U. Benlic and J. Hao. An effective multilevel memetic algorithm for balanced graph partitioning. In *22nd IEEE International Conference on Tools with Artificial Intelligence, ICTAI 2010*, pages 121–128. IEEE Computer Society, 2010. URL: <https://doi.org/10.1109/ICTAI.2010.25>, doi: 10.1109/ICTAI.2010.25.
- [25] M. J. Berger and S. H. Bokhari. A Partitioning Strategy for Nonuniform Problems on Multiprocessors. *IEEE Transactions on Computers*, 100(5):570–580, 1987.
- [26] C. Bichot and P. Siarry, editors. *Graph Partitioning*. Wiley, 2011. doi:10.1002/9781118601181.

- [27] R. H. Bisseling and W. Meesen. Communication balancing in parallel sparse matrix-vector multiplication. *ETNA. Electronic Transactions on Numerical Analysis [electronic only]*, 21:47–65, 2005. URL: <http://eudml.org/doc/128024>.
- [28] G. E. Blelloch, J. T. Fineman, P. B. Gibbons, and J. Shun. Internally Deterministic Parallel Algorithms Can Be Fast. In *PPoPP 2012*, 2012. doi:10.1145/2145816.2145840.
- [29] V. D. Blondel, J. Guillaume, R. Lambiotte, and E. Lefebvre. Fast Unfolding of Communities in Large Networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10), 2008. doi:10.1088/1742-5468/2008/10/p10008.
- [30] R. B. Boppana. Eigenvalues and Graph Bisection: An Average-Case Analysis (Extended Abstract). In *Proc. of the 28th Symposium on Foundations of Computer Science*, pages 280–285, 1987.
- [31] F. Bourse, M. Lelarge, and M. Vojnovic. Balanced Graph Edge Partition. In *Proc. of 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '14*, pages 1456–1465. ACM, 2014. doi:10.1145/2623330.2623660.
- [32] U. Brandes, D. Delling, M. Gaertler, R. Gorke, M. Hoefer, Z. Nikoloski, and D. Wagner. On modularity clustering. *IEEE transactions on knowledge and data engineering*, 20(2):172–188, 2007. doi:10.1109/TKDE.2007.190689.
- [33] B. Brandfuss, T. Alrutz, and T. Gerhold. Rank reordering for MPI communication optimization. *Computers & Fluids*, 80:372–380, 2013. doi:10.1016/j.compfluid.2012.01.019.
- [34] A. Buluç, H. Meyerhenke, I. Safro, P. Sanders, and C. Schulz. *Recent Advances in Graph Partitioning*, pages 117–158. Springer International Publishing, Cham, 2016. doi:10.1007/978-3-319-49487-6_4.
- [35] R. E. Burkard, S. E. Karisch, and F. Rendl. Qaplib—a quadratic assignment problem library. *Journal of Global optimization*, 10(4):391–403, 1997. URL: <https://doi.org/10.1023/A:1008293323270>, doi:10.1023/A:1008293323270.
- [36] A. E. Caldwell, A. B. Kahng, and I. L. Markov. Improved algorithms for hypergraph bipartitioning. In *Proceedings of ASP-DAC 2000, Asia and South Pacific Design Automation Conference*, pages 661–666. ACM, 2000. URL: <https://doi.org/10.1145/368434.368864>, doi:10.1145/368434.368864.
- [37] A. E. Caldwell, A. B. Kahng, and I. L. Markov. Optimal Partitioners and End-Case Placers for Standard-Cell Layout. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, 19(11):1304–1313, 2000. URL: <https://doi.org/10.1109/43.892854>, doi:10.1109/43.892854.
- [38] Ü. V. Çatalyürek and C. Aykanat. Hypergraph-Partitioning-Based Decomposition for Parallel Sparse-Matrix Vector Multiplication. *IEEE Transactions on Parallel and Distributed Systems*, 10(7):673–693, Jul 1999. doi:10.1109/71.780863.
- [39] Ü. V. Çatalyürek and C. Aykanat. PaToH: Partitioning Tool for Hypergraphs. <https://www.cc.gatech.edu/~umit/PaToH/manual.pdf>, 2011.
- [40] Ü. V. Çatalyürek, C. Aykanat, and B. Uçar. On two-dimensional sparse matrix partitioning: Models, methods, and a recipe. *SIAM J. Sci. Comput.*, 32(2):656–683, 2010. URL: <https://doi.org/10.1137/080737770>, doi:10.1137/080737770.
- [41] Ü. V. Çatalyürek, E. G. Boman, K. D. Devine, D. Bozdağ, R. T. Heaphy, and L. A. Riesen. A repartitioning hypergraph model for dynamic load balancing. *Journal of Parallel and Distributed Computing*, 69(8):711–724, 2009.
- [42] Ü. V. Çatalyürek, M. Deveci, K. Kaya, and B. Uçar. Multithreaded clustering for multi-level hypergraph partitioning. In *26th IEEE International Parallel and Distributed Processing Symposium, IPDPS 2012*, pages 848–859. IEEE Computer Society, 2012. doi:10.1109/IPDPS.2012.81.
- [43] Ü. V. Çatalyürek and C. Aykanat. A hypergraph-partitioning approach for coarse-grain decomposition. In *ACM/IEEE SC2001*, Denver, CO, Nov 2001. doi:10.1109/SC.2001.10035.
- [44] J. Chen and I. Safro. Algebraic Distance on Graphs. *SIAM Journal on Scientific Computing*, 33(6):3468–3490, 2011. doi:10.1137/090775087.
- [45] C. Chevalier and F. Pellegrini. PT-Scotch: A Tool for Efficient Parallel Graph Ordering. *Parallel Computing*, 34(6):318–331, 2008.
- [46] F. Chung, S. Handjani, and D. Jungreis. Generalizations of polya’s urn problem. *Annals of combinatorics*, 7(2):141–153, 2003. URL: <https://link.springer.com/content/pdf/10.1007/s00026-003-0178-y.pdf>.
- [47] C. Curino, E. Jones, Y. Zhang, and S. Madden. Schism: A Workload-driven Approach to Database Replication and Partitioning. *Proceedings of the VLDB Endowment*, 3(1-2):48–57, 2010. URL: <http://dx.doi.org/10.14778/1920841.1920853>, doi:10.14778/1920841.1920853.
- [48] T. A. Davis, W. W. Hager, S. P. Kolodziej, and S. N. Yeralan. Algorithm 1003: Mongoose, a graph coarsening and partitioning library. *ACM Transactions on Mathematical Software (TOMS)*, 46(1):1–18, 2020. doi:10.1145/3337792.
- [49] D. Delling, A. V. Goldberg, T. Pajor, and R. F. Werneck. Customizable Route Planning. In *Proc. of the 10th International Symposium on Experimental Algorithms*, volume 6630 of *LCNS*, pages 376–387. Springer, 2011.
- [50] M. Deveci, K. D. Devine, K. Pedretti, M. A. Taylor, S. Rajamanickam, and Ü. V. Çatalyürek. Geometric mapping of tasks to processors on parallel computers with mesh or torus networks. *IEEE Transactions on Parallel and Distributed Systems*, 30(9):2018–2032, 2019. doi:10.1109/TPDS.2019.2900043.
- [51] M. Deveci, K. Kaya, B. Uçar, and Ü. V. Çatalyürek. Fast and high quality topology-aware task mapping. In *2015 IEEE International Parallel and Distributed Processing Symposium, IPDPS 2015, Hyderabad, India, May 25-29, 2015*, pages

- 197–206. IEEE Computer Society, 2015. URL: <https://doi.org/10.1109/IPDPS.2015.93>, doi: 10.1109/IPDPS.2015.93.
- [52] M. Deveci, K. Kaya, B. Uçar, and Ü. V. Çatalyürek. Hypergraph partitioning for multiple communication cost metrics: Model and methods. *J. Parallel Distributed Comput.*, 77:69–83, 2015. URL: <https://doi.org/10.1016/j.jpdc.2014.12.002>, doi: 10.1016/j.jpdc.2014.12.002.
- [53] M. Deveci, S. Rajamanickam, K. D. Devine, and Ü. V. Çatalyürek. Multi-jagged: A scalable parallel spatial partitioning algorithm. *IEEE Trans. Parallel Distributed Syst.*, 27(3):803–817, 2016. URL: <https://doi.org/10.1109/TPDS.2015.2412545>, doi: 10.1109/TPDS.2015.2412545.
- [54] K. D. Devine, E. G. Boman, R. T. Heaphy, R. H. Bisseling, and Ü. V. Çatalyürek. Parallel Hypergraph Partitioning for Scientific Computing. In *20th International Conference on Parallel and Distributed Processing (IPDPS)*, pages 124–124. IEEE, 2006.
- [55] H. N. Djidjev, G. Hahn, S. M. Mniszewski, C. F. A. Negre, and A. M. N. Niklasson. Using graph partitioning for scalable distributed quantum molecular dynamics. *Algorithms*, 12(9):187, 2019. URL: <https://doi.org/10.3390/a12090187>, doi: 10.3390/a12090187.
- [56] E. D. Dolan and J. J. Moré. Benchmarking optimization software with performance profiles. *Math. Program.*, 91(2):201–213, 2002. URL: <https://doi.org/10.1007/s101070100263>, doi: 10.1007/s101070100263.
- [57] W. E. Donath. Logic partitioning. *Physical Design Automation of VLSI Systems*, pages 65–86, 1988.
- [58] W. E. Donath and A. J. Hoffman. Algorithms for Partitioning of Graphs and Computer Logic Based on Eigenvectors of Connection Matrices. *IBM Technical Disclosure Bulletin*, 15(3):938–944, 1972.
- [59] W. E. Donath and A. J. Hoffman. Lower Bounds for the Partitioning of Graphs. *IBM Journal of Research and Development*, 17(5):420–425, 1973.
- [60] S. Dutt and H. Thény. Partitioning Around Roadblocks: Tackling Constraints with Intermediate Relaxations. In *Proceedings of the 1997 IEEE/ACM International Conference on Computer-Aided Design, ICCAD 1997, San Jose, CA, USA, November 9-13, 1997*, pages 350–355. IEEE Computer Society / ACM, 1997. URL: <https://doi.org/10.1109/ICCAD.1997.643546>, doi: 10.1109/ICCAD.1997.643546.
- [61] W. Fan, M. Liu, C. Tian, R. Xu, and J. Zhou. Incrementalization of graph partitioning algorithms. *Proceedings of the VLDB Endowment*, 13(10):1261–1274, 2020. doi: 10.14778/3389133.3389142.
- [62] M. F. Faraj and C. Schulz. Buffered streaming graph partitioning. *CoRR*, abs/2102.09384, 2021. URL: <https://arxiv.org/abs/2102.09384>, arXiv: 2102.09384.
- [63] M. F. Faraj, A. van der Grinten, H. Meyerhenke, J. L. Träff, and C. Schulz. High-quality hierarchical process mapping. In *18th International Symposium on Experimental Algorithms, SEA 2020*, pages 4:1–4:15, LIPIcs 160, 2020. Schloss Dagstuhl - Leibniz-Zentrum für Informatik. doi: 10.4230/LIPIcs.SEA.2020.4.
- [64] C. M. Fiduccia and R. M. Mattheyses. A Linear-Time Heuristic for Improving Network Partitions. In *Proc. of the 19th Conference on Design Automation*, pages 175–181, 1982.
- [65] M. Fiedler. A Property of Eigenvectors of Nonnegative Symmetric Matrices and its Application to Graph Theory. *Czechoslovak Mathematical Journal*, 25(4):619–633, 1975.
- [66] J. Fietz, M. Krause, C. Schulz, P. Sanders, and V. Heuveline. Optimized Hybrid Parallel Lattice Boltzmann Fluid Flow Simulations on Complex Geometries. In *Proc. of Euro-Par 2012 Parallel Processing*, volume 7484 of LNCS, pages 818–829. Springer, 2012.
- [67] J. Fietz, M. J. Krause, C. Schulz, P. Sanders, and V. Heuveline. Optimized hybrid parallel lattice boltzmann fluid flow simulations on complex geometries. In *18th International Conference on Parallel Processing (Euro-Par)*, volume 7484 of *Lecture Notes in Computer Science*, pages 818–829. Springer, 2012. URL: https://doi.org/10.1007/978-3-642-32820-6_81, doi: 10.1007/978-3-642-32820-6_81.
- [68] R. A. Fisher. Statistical Methods for Research Workers. In *Breakthroughs in Statistics*, pages 66–70. Springer, 1992.
- [69] M. Friedman. A Comparison of Alternative Tests of Significance for the Problem of m Rankings. *The Annals of Mathematical Statistics*, 11(1):86–92, 1940.
- [70] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*, volume 174. W.H. Freeman, San Francisco, 1979.
- [71] M. R. Garey, D. S. Johnson, and L. Stockmeyer. Some Simplified NP-Complete Problems. In *Proc. of the 6th ACM Symposium on Theory of Computing, STOC '74*, pages 47–63. ACM, 1974.
- [72] M. S. Gilbert, S. Acer, E. G. Boman, K. Madduri, and S. Rajamanickam. Performance-portable graph coarsening for efficient multilevel graph analysis. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 213–222, 2021. doi: 10.1109/IPDPS49936.2021.00030.
- [73] R. Glantz, H. Meyerhenke, and A. Noe. Algorithms for mapping parallel processes onto grid and torus architectures. In *2015 23rd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*, pages 236–243. IEEE, 2015. doi: 10.1109/PDP.2015.21.
- [74] R. Glantz, H. Meyerhenke, and C. Schulz. Tree-based coarsening and partitioning of complex networks. *ACM J. Exp. Algorithmics*, 21(1):1.6:1–1.6:20, 2016. URL: <https://doi.org/10.1145/2851496>, doi: 10.1145/2851496.

- [75] R. Glantz, M. Predari, and H. Meyerhenke. Topology-induced enhancement of mappings. In *Proceedings of the 47th International Conference on Parallel Processing*, pages 1–10, 2018. doi:[10.1145/3225058.3225117](https://doi.org/10.1145/3225058.3225117).
- [76] C. Godenschwager, F. Schornbaum, M. Bauer, H. Köstler, and U. Rüde. A framework for hybrid parallel flow simulations with a trillion cells in complex geometries. In *International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 35:1–35:12. ACM, 2013. doi:[10.1145/2503210.2503273](https://doi.org/10.1145/2503210.2503273).
- [77] D. E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.
- [78] J. E. Gonzalez, Y. Low, H. Gu, D. Bickson, and C. Guestrin. Powergraph: Distributed graph-parallel computation on natural graphs. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*, pages 17–30. USENIX Association, 2012. doi:[10.5555/2387880.2387883](https://doi.org/10.5555/2387880.2387883).
- [79] B. Goodarzi, M. Burtscher, and D. Goswami. Parallel graph partitioning on a CPU-GPU architecture. In *2016 IEEE International Parallel and Distributed Processing Symposium Workshops, IPDPS Workshops*, pages 58–66, 2016. URL: <https://doi.org/10.1109/IPDPSW.2016.16>, doi:[10.1109/IPDPSW.2016.16](https://doi.org/10.1109/IPDPSW.2016.16).
- [80] B. Goodarzi, F. Khorasani, V. Sarkar, and D. Goswami. High performance multilevel graph partitioning on GPU. In *17th International Conference on High Performance Computing & Simulation, HPCS*, pages 769–778, 2019. URL: <https://doi.org/10.1109/HPCS48598.2019.9188120>, doi:[10.1109/HPCS48598.2019.9188120](https://doi.org/10.1109/HPCS48598.2019.9188120).
- [81] L. Gottesbüren, M. Hamann, S. Schlag, and D. Wagner. Advanced flow-based multilevel hypergraph partitioning. In *18th International Symposium on Experimental Algorithms, SEA*, pages 11:1–11:15, 2020. doi:[10.4230/LIPIcs.SEA.2020.11](https://doi.org/10.4230/LIPIcs.SEA.2020.11).
- [82] L. Gottesbüren, M. Hamann, and D. Wagner. Evaluation of a flow-based hypergraph bipartitioning algorithm. In *27th Annual European Symposium on Algorithms, ESA*, pages 52:1–52:17, 2019. doi:[10.4230/LIPIcs.ESA.2019.52](https://doi.org/10.4230/LIPIcs.ESA.2019.52).
- [83] L. Gottesbüren, T. Heuer, P. Sanders, and S. Schlag. Scalable Shared-Memory Hypergraph Partitioning. In *Proceedings of the Symposium on Algorithm Engineering and Experiments ALENEX*, pages 16–30, 2021. doi:[10.1137/1.9781611976472.2](https://doi.org/10.1137/1.9781611976472.2).
- [84] L. Gottesbüren, T. Heuer, P. Sanders, and S. Schlag. Shared-memory n -level hypergraph partitioning. In *Workshop on Algorithm Engineering and Experiments (ALENEX)*. SIAM, 2022. to appear. URL: <https://arxiv.org/abs/2104.08107>.
- [85] L. Gottesbüren, T. Heuer, P. Sanders, and S. Schlag. Shared-memory n -level hypergraph partitioning. In *24th Workshop on Algorithm Engineering and Experiments (ALENEX 2022)*. SIAM, 01 2022. doi:[10.1137/1.9781611977042.11](https://doi.org/10.1137/1.9781611977042.11).
- [86] L. Gottesbüren, T. Heuer, P. Sanders, C. Schulz, and D. Seemaier. Deep multilevel graph partitioning. In *29th Annual European Symposium on Algorithms, ESA*, volume 204 of *LIPIcs*, pages 48:1–48:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. URL: <https://doi.org/10.4230/LIPIcs.ESA.2021.48>, doi:[10.4230/LIPIcs.ESA.2021.48](https://doi.org/10.4230/LIPIcs.ESA.2021.48).
- [87] L. Gottesbüren and M. Hamann. Deterministic parallel hypergraph partitioning. *CoRR*, abs/2112.12704, 2021. URL: <https://arxiv.org/abs/2112.12704>, arXiv:[2112.12704](https://arxiv.org/abs/2112.12704).
- [88] L. Gottesbüren, T. Heuer, and P. Sanders. Parallel flow-based hypergraph partitioning. In *Proc. of the 20th International Symposium on Experimental Algorithms*, volume 233. *LIPIcs*, 2022. URL: <https://arxiv.org/abs/2201.01556>, arXiv:[2201.01556](https://arxiv.org/abs/2201.01556).
- [89] J. Gray and S. Kourtis. Hyper-optimized tensor network contraction. *Quantum*, 5:410, 2021. doi:[10.22331/q-2021-03-15-410](https://doi.org/10.22331/q-2021-03-15-410).
- [90] M. Grigni and F. Manne. On the complexity of the generalized block distribution. In A. Ferreira, J. D. P. Rolim, Y. Saad, and T. Yang, editors, *Parallel Algorithms for Irregularly Structured Problems, Third International Workshop, IRREGULAR '96, Santa Barbara, California, USA, August 19-21, 1996, Proceedings*, volume 1117 of *Lecture Notes in Computer Science*, pages 319–326. Springer, 1996. URL: <https://doi.org/10.1007/BFb0030123>, doi:[10.1007/BFb0030123](https://doi.org/10.1007/BFb0030123).
- [91] A. Gutfraind, I. Safro, and L. A. Meyers. Multiscale network generation. In *18th International Conference on Information Fusion, FUSION 2015, Washington, DC, USA, July 6-9, 2015*, pages 158–165. IEEE, 2015. URL: <https://ieeexplore.ieee.org/document/7266557/>.
- [92] M. Hamann and B. Strasser. Graph bisection with pareto optimization. *ACM Journal of Experimental Algorithmics*, 23, 2018. doi:[10.1145/3173045](https://doi.org/10.1145/3173045).
- [93] M. Hamann, B. Strasser, D. Wagner, and T. Zeitz. Distributed graph clustering using modularity and map equation. In M. Aldinucci, L. Padovani, and M. Torquati, editors, *Euro-Par 2018: Parallel Processing - 24th International Conference on Parallel and Distributed Computing*, volume 11014 of *Lecture Notes in Computer Science*, pages 688–702. Springer, 2018. URL: https://doi.org/10.1007/978-3-319-96983-1_49, doi:[10.1007/978-3-319-96983-1_49](https://doi.org/10.1007/978-3-319-96983-1_49).
- [94] L. Hamers et al. Similarity measures in scientometric research: The jaccard index versus salton's cosine formula. *Information Processing and Management*, 25(3):315–18, 1989.
- [95] M. Han and K. Daudjee. Giraph unchained: Barrierless asynchronous parallel execution in pregel-like graph processing systems. *Proc. VLDB Endow.*, 8(9):950–961, 2015. doi:[10.14778/2777598.2777604](https://doi.org/10.14778/2777598.2777604).
- [96] C. H. Heider. A computationally simplified pair-exchange algorithm for the quadratic assignment problem. Technical report, DTIC Document, 1972.

- [97] B. Hendrickson and R. Leland. An Improved Spectral Graph Partitioning Algorithm for Mapping Parallel Computations. *SIAM Journal on Scientific Computing*, 16(2):452–469, 1995.
- [98] A. Henzinger, A. Noe, and C. Schulz. ILP-based local search for graph partitioning. *ACM J. Exp. Algorithmics*, 25:1–26, 2020. URL: <https://doi.org/10.1145/3398634>, doi:10.1145/3398634.
- [99] J. Herrmann, M. Y. Özkaya, B. Uçar, K. Kaya, and Ü. V. Çatalyürek. Multilevel algorithms for acyclic partitioning of directed acyclic graphs. *SIAM J. Scientific Computing*, 41(4):A2117–A2145, 2019. doi:10.1137/18M1176865.
- [100] J. Herrmann, J. Kho, B. Uçar, K. Kaya, and Ü. V. Çatalyürek. Acyclic partitioning of large directed acyclic graphs. In *Proceedings of the 17th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing, CCGRID*, pages 371–380. IEEE Computer Society / ACM, 2017. URL: <https://doi.org/10.1109/CCGRID.2017.101>, doi:10.1109/CCGRID.2017.101.
- [101] T. Heuer and S. Schlag. Improving Coarsening Schemes for Hypergraph Partitioning by Exploiting Community Structure. In *16th International Symposium on Experimental Algorithms, (SEA)*, page 21:1–21:19, 2017. doi:10.4230/LIPIcs.SEA.2017.21.
- [102] T. Heuer. Engineering Initial Partitioning Algorithms for direct k -way Hypergraph Partitioning. Bachelor thesis, Karlsruhe Institute of Technology, 08 2015.
- [103] T. Heuer, N. Maas, and S. Schlag. Multilevel Hypergraph Partitioning with Vertex Weights Revisited. In *19th International Symposium on Experimental Algorithms (SEA)*, volume 190, pages 8:1–8:20, 2021. doi:10.4230/LIPIcs.SEA.2021.8.
- [104] T. Heuer, P. Sanders, and S. Schlag. Network flow-based refinement for multilevel hypergraph partitioning. *ACM Journal of Experimental Algorithmics*, 24(1):2.3:1–2.3:36, 2019. doi:10.1145/3329872.
- [105] L. Hoang, R. Dathathri, G. Gill, and K. Pingali. Cusp: A customizable streaming edge partitioner for distributed graph analytics. In *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 439–450. IEEE, 2019. doi:10.1109/IPDPS.2019.00054.
- [106] M. Holtgrewe, P. Sanders, and C. Schulz. Engineering a Scalable High Quality Graph Partitioner. *Proc. of the 24th IEEE International Parallel and Distributed Processing Symposium*, pages 1–12, 2010.
- [107] C. Huang, F. Zhang, M. Newman, J. Cai, X. Gao, Z. Tian, J. Wu, H. Xu, H. Yu, B. Yuan, et al. Classical simulation of quantum supremacy circuits. *arXiv preprint arXiv:2005.06787*, 2020.
- [108] J. Huang and D. J. Abadi. Leopard: Lightweight edge-oriented partitioning and replication for dynamic graphs. *Proceedings of the VLDB Endowment*, 9(7), 2016. doi:10.14778/2904483.2904486.
- [109] N. Jafari, O. Selvitopi, and C. Aykanat. Fast shared-memory streaming multilevel graph partitioning. *Journal of Parallel and Distributed Computing*, 147:140–151, 2021. doi:10.1016/j.jpdc.2020.09.004.
- [110] I. Kabiljo, B. Karrer, M. Pundir, S. Pupyrev, A. Shalita, Y. Akhremtsev, and A. Presta. Social hash partitioner: A scalable distributed hypergraph partitioner. *Proceedings of the VLDB Endowment*, 10(11):1418–1429, 2017. doi:10.14778/3137628.3137650.
- [111] G. Karypis and V. Kumar. Parallel Multilevel k -way Partitioning Scheme for Irregular Graphs. In *Proc. of the ACM/IEEE Conference on Supercomputing'96*, 1996.
- [112] G. Karypis and V. Kumar. A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs. *SIAM Journal on Scientific Computing*, 20(1):359–392, 1998.
- [113] G. Karypis and V. Kumar. Multilevel graph partitioning schemes. In *Proceedings of the 1995 International Conference on Parallel Processing, Urbana-Champaign, Illinois, USA, August 14-18, 1995. Volume III: Algorithms & Applications*, pages 113–122. CRC Press, 1995.
- [114] G. Karypis and V. Kumar. Parallel multilevel k -way partitioning scheme for irregular graphs. In *Proceedings of the 1996 ACM/IEEE Conference on Supercomputing*, Supercomputing '96. IEEE Computer Society, 1996. doi:10.1145/369028.369103.
- [115] G. Karypis and V. Kumar. Multilevel algorithms for multi-constraint graph partitioning. In *ACM/IEEE Conference on Supercomputing*, page 28. IEEE Computer Society, 1998. URL: <https://doi.org/10.1109/SC.1998.10018>, doi:10.1109/SC.1998.10018.
- [116] G. Karypis and V. Kumar. A parallel algorithm for multilevel graph partitioning and sparse matrix ordering. *J. Parallel Distributed Comput.*, 48(1):71–95, 1998. URL: <https://doi.org/10.1006/jpdc.1997.1403>, doi:10.1006/jpdc.1997.1403.
- [117] G. Karypis and V. Kumar. Multilevel k -way Hypergraph Partitioning. *VLSI Design*, 2000(3):285–300, 2000. URL: <https://doi.org/10.1155/2000/19436>, doi:10.1155/2000/19436.
- [118] K. Kaya, B. Uçar, and Ü. V. Çatalyürek. Analysis of partitioning models and metrics in parallel sparse matrix-vector multiplication. In *Parallel Processing and Applied Mathematics - 10th International Conference, PPAM, Revised Selected Papers, Part II*, volume 8385 of LNCS, pages 174–184. Springer, 2013. URL: https://doi.org/10.1007/978-3-642-55195-6_16, doi:10.1007/978-3-642-55195-6_16.
- [119] F. Khorasani, R. Gupta, and L. N. Bhuyan. Scalable simd-efficient graph processing on gpus. In *2015 International Conference on Parallel Architectures and Compilation, PACT*, pages 39–50. IEEE Computer Society, 2015. URL:

- <https://doi.org/10.1109/PACT.2015.15>, doi:10.1109/PACT.2015.15.
- [120] T. Kiefer, D. Habich, and W. Lehner. Penalized graph partitioning for static and dynamic load balancing. In *European Conference on Parallel Processing*, pages 146–158. Springer, 2016. doi:10.1007/978-3-319-43659-3_11.
 - [121] J. Kim, I. Hwang, Y.-H. Kim, and B.-R. Moon. Genetic Approaches for Graph Partitioning: A Survey. In *13th Genetic and Evolutionary Computation (GECCO)*, pages 473–480. ACM, 2011. URL: <http://doi.acm.org/10.1145/2001576.2001642>, doi:10.1145/2001576.2001642.
 - [122] K. V. Kirchbach, C. Schulz, and J. L. Träff. Better process mapping and sparse quadratic assignment. *Journal of Experimental Algorithmics (JEA)*, 25:1–19, 2020. doi:10.1145/3409667.
 - [123] S. Kirmani and P. Raghavan. Scalable Parallel Graph Partitioning. In *Intl. Conference for High Performance Computing, Networking, Storage and Analysis (SC'13)*, page 51. ACM, 2013.
 - [124] A. V. Knyazev. Toward the optimal preconditioned eigensolver: Locally optimal block preconditioned conjugate gradient method. *SIAM J. Sci. Comput.*, 23(2):517–541, 2001. URL: <https://doi.org/10.1137/S1064827500366124>, doi:10.1137/S1064827500366124.
 - [125] M. J. Krause, A. Kummerländer, S. J. Avis, H. Kusumaatmaja, D. Dapelo, F. Klemens, M. Gaedtko, N. Hafen, A. Mink, R. Trunk, J. E. Marquardt, M. Maier, M. Haussmann, and S. Simonis. Openlb - open source lattice boltzmann code. *Comput. Math. Appl.*, 81:258–288, 2021. doi:10.1016/j.camwa.2020.04.033.
 - [126] K. A. Kumar, A. Quamar, A. Deshpande, and S. Khuller. SWORD: workload-aware data placement and replica selection for cloud data management systems. *The VLDB Journal*, 23(6):845–870, 2014. doi:10.1007/s00778-014-0362-1.
 - [127] R. Kumar, G. Charpiat, and M. Thonnat. Multiple object tracking by efficient graph partitioning. In *12th Asian Conference on Computer Vision, Revised Selected Papers, Part IV*, volume 9006 of LNCS, pages 445–460. Springer, 2014. URL: https://doi.org/10.1007/978-3-319-16817-3_29, doi:10.1007/978-3-319-16817-3_29.
 - [128] S. Lamm, P. Sanders, and C. Schulz. Graph partitioning for independent sets. In *14th International Symposium on Experimental Algorithms SEA*, volume 9125 of LNCS, pages 68–81. Springer, 2015. URL: https://doi.org/10.1007/978-3-319-20086-6_6, doi:10.1007/978-3-319-20086-6_6.
 - [129] J. Larsson Träff. Direct graph k-partitioning with a Kernighan–Lin like heuristic. *Operations Research Letters*, 34(6):621–629, November 2006. doi:10.1016/j.orl.2005.10.003.
 - [130] D. LaSalle and G. Karypis. Multi-threaded graph partitioning. In *IEEE Transactions on Parallel and Distributed Systems*, pages 225–236. IEEE, 2013. doi:10.1109/IPDPS.2013.50.
 - [131] D. LaSalle and G. Karypis. A parallel hill-climbing refinement algorithm for graph partitioning. In *45th International Conference on Parallel Processing (ICPP)*, pages 236–241. IEEE, 2016. doi:10.1109/ICPP.2016.34.
 - [132] D. LaSalle, M. M. A. Patwary, N. Satish, N. Sundaram, P. Dubey, and G. Karypis. Improving graph partitioning for modern graphs and architectures. In *Proceedings of the 5th Workshop on Irregular Applications - Architectures and Algorithms, IA3*, pages 14:1–14:4, 2015. doi:10.1145/2833179.2833188.
 - [133] Y. Li, X. Yu, and N. Koudas. Les3: Learning-based exact set similarity search. *arXiv preprint arXiv:2107.10417*, 2021.
 - [134] X. Liu, H. Ushijima-Mwesigwa, I. Ghosh, and I. Safro. Partitioning dense graphs with hardware accelerators. In D. Groen, C. de Mulatier, M. Paszynski, V. V. Krzhizhanovskaya, J. J. Dongarra, and P. M. A. Sloot, editors, *Computational Science – ICCS 2022*, pages 476–483, Cham, 2022. Springer International Publishing.
 - [135] S. Lloyd. Least Squares Quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.
 - [136] F. Lotfifar and M. Johnson. A multi-level hypergraph partitioning algorithm using rough set clustering. In *EuroPar 2015: Parallel Processing - 21st International Conference on Parallel and Distributed Computing*, volume 9233 of LNCS, pages 159–170. Springer, 2015. URL: https://doi.org/10.1007/978-3-662-48096-0_13, doi:10.1007/978-3-662-48096-0_13.
 - [137] S. Maleki, U. Agarwal, M. Burtscher, and K. Pingali. Bipart: a parallel and deterministic hypergraph partitioner. In *PPoPP '21: 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 161–174, 2021. doi:10.1145/3437801.3441611.
 - [138] Z. Á. Mann and P. A. Papp. Guiding SAT solving by formula partitioning. *International Journal of Artificial Intelligence Tools*, 26(4):1750011:1–1750011:37, 2017. doi:10.1142/S0218213017500117.
 - [139] F. Manne and T. Sorevik. Partitioning an array onto a mesh of processors. In *Applied Parallel Computing, Industrial Computation and Optimization, Third International Workshop, PARA 96, Lyngby, Denmark, August 18-21, 1996, Proceedings*, volume 1184 of LNCS, pages 467–477. Springer, 1996. URL: https://doi.org/10.1007/3-540-62095-8_50, doi:10.1007/3-540-62095-8_50.
 - [140] C. Martella, D. Logothetis, A. Loukas, and G. Siganos. Spinner: Scalable graph partitioning in the cloud. In *33rd IEEE International Conference on Data Engineering, ICDE*, pages 1083–1094. IEEE Computer Society, 2017. URL: <https://doi.org/10.1109/ICDE.2017.153>, doi:10.1109/ICDE.2017.153.
 - [141] C. Mayer, R. Mayer, M. A. Tariq, H. Geppert, L. Laich, L. Rieger, and K. Rothermel. Adwise: Adaptive window-based streaming edge partitioning for high-speed graph processing. In *IEEE 38th International Conference on Distributed Computing Systems (ICDCS)*, pages 685–695. IEEE, 2018. doi:10.1109/ICDCS.2018.00072.

- [142] R. R. McCune, T. Weninger, and G. Madey. Thinking Like a Vertex: A Survey of Vertex-Centric Frameworks for Large-Scale Distributed Graph Processing. *ACM Computing Survey*, 48(2):25:1–25:39, 2015. doi:10.1145/2818185.
- [143] H. Meyerhenke, B. Monien, and S. Schamberger. Graph Partitioning and Disturbed Diffusion. *Parallel Comput.*, 35(10-11):544–569, 2009. URL: <https://doi.org/10.1016/j.parco.2009.09.006>, doi:10.1016/j.parco.2009.09.006.
- [144] H. Meyerhenke, P. Sanders, and C. Schulz. Partitioning Complex Networks via Size-Constrained Clustering. In *13th International Symposium on Experimental Algorithms*, volume 8504 of LNCS, pages 351–363. Springer, 2014. doi:10.1007/978-3-319-07959-2_30.
- [145] H. Meyerhenke, P. Sanders, and C. Schulz. Partitioning (hierarchically clustered) complex networks via size-constrained graph clustering. *J. Heuristics*, 22(5):759–782, 2016. URL: <https://doi.org/10.1007/s10732-016-9315-8>, doi:10.1007/s10732-016-9315-8.
- [146] H. Meyerhenke, P. Sanders, and C. Schulz. Parallel graph partitioning for complex networks. *IEEE Trans. Parallel Distrib. Syst.*, 28(9):2625–2638, 2017. URL: <https://doi.org/10.1109/TPDS.2017.2671868>, doi:10.1109/TPDS.2017.2671868.
- [147] T. Minyard and Y. Kallinderis. Octree Partitioning of Hybrid Grids for Parallel Adaptive Viscous Flow Simulations. *International journal for numerical methods in fluids*, 26(1):57–78, 1998.
- [148] B. Monien, R. Preis, and S. Schamberger. Approximation Algorithms for Multilevel Graph Partitioning. In *Handbook of Approximation Algorithms and Metaheuristics*, chapter 60, pages 60–1–60–15. Taylor & Francis, 2007. doi:10.1201/9781420010749.ch60.
- [149] O. Moreira, M. Popp, and C. Schulz. Evolutionary multi-level acyclic graph partitioning. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, pages 332–339, 2018. doi:10.1145/3205455.3205464.
- [150] O. Moreira, M. Popp, and C. Schulz. Evolutionary multi-level acyclic graph partitioning. *J. Heuristics*, 26(5):771–799, 2020. URL: <https://doi.org/10.1007/s10732-020-09448-8>, doi:10.1007/s10732-020-09448-8.
- [151] H. Müller-Merbach. *Optimale reihenfolgen*, volume 15 of *Ökonometrie und Unternehmensforschung*. Springer-Verlag, 1970.
- [152] M. Naumov and T. Moon. Parallel spectral graph partitioning. *NVIDIA tech. rep.*, 2016.
- [153] D. Nicoara, S. Kamali, K. Daudjee, and L. Chen. Hermes: Dynamic partitioning for distributed social network graph databases. In *EDBT*, pages 25–36, 2015. URL: <https://doi.org/10.5441/002/edbt.2015.04>, doi:10.5441/002/edbt.2015.04.
- [154] J. Nishimura and J. Ugander. Restreaming graph partitioning: simple versatile algorithms for advanced balancing. In *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 1106–1114, 2013. doi:10.1145/2487575.2487696.
- [155] V. Osipov and P. Sanders. *n*-Level Graph Partitioning. In *Proc. of the 18th European Conference on Algorithms: Part I*, volume 6346 of LNCS, pages 278–289. Springer, 2010. doi:10.1007/978-3-642-15775-2_24.
- [156] A. Pacaci and M. T. Özsu. Experimental analysis of streaming algorithms for graph partitioning. In *Proceedings of the 2019 International Conference on Management of Data*, pages 1375–1392, 2019. doi:10.1145/3299869.3300076.
- [157] F. Pan and P. Zhang. Simulating the sycamore quantum supremacy circuits. *arXiv preprint arXiv:2103.03074*, 2021.
- [158] D. A. Papa and I. L. Markov. Hypergraph Partitioning and Clustering. In *Handbook of Approximation Algorithms and Metaheuristics*. Chapman and Hall/CRC, 2007. URL: <http://dx.doi.org/10.1201/9781420010749.ch61>, doi:10.1201/9781420010749.ch61.
- [159] M. A. K. Patwary, S. Garg, and B. Kang. Window-based streaming graph partitioning algorithm. In *Proceedings of the Australasian Computer Science Week Multiconference*, pages 1–10, 2019. doi:10.1145/3290688.3290711.
- [160] F. Pellegrini. Scotch Home Page. <http://www.labri.fr/pelegri/scotch>.
- [161] F. Pellegrini and J. Roman. SCOTCH: A software package for static mapping by dual recursive bipartitioning of process and architecture graphs. In *High-Performance Computing and Networking*, volume 1067 of LNCS, pages 493–498. Springer, 1996. doi:10.1007/3-540-61142-8_588.
- [162] F. Petroni, L. Querzoni, K. Daudjee, S. Kamali, and G. Iacoboni. Hdrf: Stream-based partitioning for power-law graphs. In *Proceedings of the 24th ACM International on Conference on Information and Knowledge Management*, pages 243–252, 2015. doi:10.1145/2806416.2806424.
- [163] J. R. Pilkington and S. B. Baden. Dynamic Partitioning of Non-Uniform Structured Workloads with Spacefilling Curves. *IEEE Trans. Parallel Distributed Syst.*, 7(3):288–300, 1996. URL: <https://doi.org/10.1109/71.491582>, doi:10.1109/71.491582.
- [164] M. Popp, S. Schlag, C. Schulz, and D. Seemaier. Multilevel acyclic hypergraph partitioning. In *Proceedings of the Symposium on Algorithm Engineering and Experiments, ALENEX*, pages 1–15. SIAM, 2021. URL: <https://doi.org/10.1137/1.9781611976472.1>, doi:10.1137/1.9781611976472.1.
- [165] A. Pothén, H. D. Simon, and K. P. Liou. Partitioning Sparse Matrices with Eigenvectors of Graphs. *SIAM Journal on Matrix Analysis and Applications*, 11(3):430–452, 1990.
- [166] M. Predari, C. Tzovas, C. Schulz, and H. Meyerhenke. An mpi-based algorithm for mapping complex networks onto hierarchical architectures. *CoRR*, abs/2107.02539, 2021. URL: <https://arxiv.org/abs/2107.02539>, arXiv:2107.02539.

- [167] R. J. Preen and J. Smith. Evolutionary n-level hypergraph partitioning with adaptive coarsening. *IEEE Trans. Evol. Comput.*, 23(6):962–971, 2019. URL: <https://doi.org/10.1109/TEVC.2019.2896951>, doi:10.1109/TEVC.2019.2896951.
- [168] U. N. Raghavan, R. Albert, and S. Kumara. Near Linear Time Algorithm to Detect Community Structures in Large-Scale Networks. *Physical Review E*, 76(3), 2007. doi:10.1103/PhysRevE.76.036106.
- [169] F. Rahimian, A. H. Payberah, S. Girdzijauskas, M. Jelasity, and S. Haridi. JA-BE-JA: A distributed algorithm for balanced graph partitioning. In *7th IEEE International Conference on Self-Adaptive and Self-Organizing Systems*, pages 51–60. IEEE Computer Society, 2013. URL: <https://doi.org/10.1109/SASO.2013.13>, doi:10.1109/SASO.2013.13.
- [170] D. Recalde, R. Torres, and P. Vaca. An exact approach for the multi-constraint graph partitioning problem. *EURO J. Comput. Optim.*, 8(3):289–308, 2020. URL: <https://doi.org/10.1007/s13675-020-00126-9>, doi:10.1007/s13675-020-00126-9.
- [171] D. Ron, I. Safro, and A. Brandt. Relaxation-Based Coarsening and Multiscale Graph Organization. *Multiscale Modeling & Simulation*, 9(1):407–423, 2011. doi:10.1137/100791142.
- [172] E. R. Ruiz and C. Segura. Memetic algorithm with hungarian matching based crossover and diversity preservation. *Computación y Sistemas*, 22(2), 2018. URL: <http://www.cys.cic.ipn.mx/ojs/index.php/CyS/article/view/2951>.
- [173] H. P. Sajjad, A. H. Payberah, F. Rahimian, V. Vlassov, and S. Haridi. Boosting vertex-cut partitioning for streaming graphs. In *2016 IEEE International Congress on Big Data (BigData Congress)*, pages 1–8. IEEE, 2016. doi:10.1109/BigDataCongress.2016.10.
- [174] P. Sanders and C. Schulz. KaHIP – Karlsruhe High Quality Partitioning Homepage. <http://algo2.iti.kit.edu/kahip>.
- [175] P. Sanders and C. Schulz. Engineering Multilevel Graph Partitioning Algorithms (see ArXiv preprint arXiv:1012.0006v3). Technical report, Karlsruhe Institute of Technology, 2010.
- [176] P. Sanders and C. Schulz. Engineering Multilevel Graph Partitioning Algorithms. In *Proc. of the 19th European Symp. on Algorithms*, volume 6942 of LNCS, pages 469–480. Springer, 2011. doi:10.1007/978-3-642-23719-5_40.
- [177] P. Sanders and C. Schulz. Distributed Evolutionary Graph Partitioning. In *Proc. of the 12th Workshop on Algorithm Engineering and Experimentation (ALENEX’12)*, pages 16–29, 2012. doi:10.1137/1.9781611972924.2.
- [178] P. Sanders and C. Schulz. Think Locally, Act Globally: Perfectly Balanced Graph Partitioning (see ArXiv preprint arXiv:1210.0477). Technical report, Karlsruhe Institute of Technology, 2012.
- [179] P. Sanders and C. Schulz. Think Locally, Act Globally: Highly Balanced Graph Partitioning. In *Proc. of the 12th International Symposium on Experimental Algorithms (SEA’12)*, LNCS. Springer, 2013. doi:10.1007/978-3-642-38527-8_16.
- [180] P. Sanders, C. Schulz, D. Strash, and R. Williger. Distributed evolutionary k -way node separators. In *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO*, pages 345–352. ACM, 2017. URL: <https://doi.org/10.1145/3071178.3071204>, doi:10.1145/3071178.3071204.
- [181] J. E. Savage and M. G. Wloka. Parallelism in Graph-Partitioning. *Journal of Parallel and Distributed Computing*, 13(3):257–272, 1991. doi:10.1016/0743-7315(91)90074-J.
- [182] S. Schlag. *High-Quality Hypergraph Partitioning*. PhD thesis, Karlsruhe Institute of Technology, Germany, 2020. URL: <https://nbn-resolving.org/urn:nbn:de:101:1-2020030403581620165765>.
- [183] S. Schlag, V. Henne, T. Heuer, H. Meyerhenke, P. Sanders, and C. Schulz. k -way hypergraph partitioning via n -level recursive bisection. In *Proceedings of the Eighteenth Workshop on Algorithm Engineering and Experiments, ALENEX*, pages 53–67. SIAM, 2016. URL: <https://doi.org/10.1137/1.9781611974317.5>, doi:10.1137/1.9781611974317.5.
- [184] S. Schlag, C. Schulz, D. Seemaier, and D. Strash. Scalable edge partitioning. In *Proceedings of the Twenty-First Workshop on Algorithm Engineering and Experiments, ALENEX*, pages 211–225. SIAM, 2019. URL: <https://doi.org/10.1137/1.9781611975499.17>, doi:10.1137/1.9781611975499.17.
- [185] K. Schloegel, G. Karypis, and V. Kumar. Graph Partitioning for High-Performance Scientific Simulations. In *Sourcebook of parallel computing*, pages 491–541. Morgan Kaufmann Publishers, 2003.
- [186] K. Schloegel, G. Karypis, and V. Kumar. A new algorithm for multi-objective graph partitioning. In P. Amestoy, P. Berger, M. J. Daydé, I. S. Duff, V. Frayssé, L. Giraud, and D. Ruiz, editors, *Euro-Par ’99 Parallel Processing, 5th International Euro-Par Conference, Toulouse, France, August 31 - September 3, 1999, Proceedings*, volume 1685 of *Lecture Notes in Computer Science*, pages 322–331. Springer, 1999. URL: https://doi.org/10.1007/3-540-48311-X_42, doi:10.1007/3-540-48311-X_42.
- [187] K. Schloegel, G. Karypis, and V. Kumar. Graph partitioning for dynamic, adaptive and multi-phase scientific simulations. In *IEEE International Conference on Cluster Computing (CLUSTER 2001)*, pages 271–273. IEEE Computer Society, 2001. URL: <https://doi.org/10.1109/CLUSTER.2001.959987>, doi:10.1109/CLUSTER.2001.959987.
- [188] C. Schulz. *High Quality Graph Partitioning*. PhD thesis, Karlsruhe Institute of Technology, 2013. doi:10.5445/IR/1000035713.
- [189] C. Schulz and J. L. Träff. Better process mapping and sparse quadratic assignment. In *16th International Symposium on Experimental Algorithms*, volume 75 of *LIPIcs*, pages 4:1–4:15, 2017. doi:10.4230/LIPIcs.SEA.2017.4.

- [190] C. Schulz and J. L. Träff. VieM – Vienna Process Mapping and Sparse Quadratic Assignment. <http://viem.taa.univie.ac.at/>.
- [191] B. Shao, H. Wang, and Y. Li. The Trinity Graph Engine. Technical report, Microsoft Research Asia, 2012.
- [192] R. Shaydulin, J. Chen, and I. Safro. Relaxation-based coarsening for multilevel hypergraph partitioning. *Multiscale Model. Simul.*, 17(1):482–506, 2019. URL: <https://doi.org/10.1137/17M1152735>, doi: 10.1137/17M1152735.
- [193] R. Shaydulin and I. Safro. Aggregative coarsening for multilevel hypergraph partitioning. In G. D’Angelo, editor, *17th International Symposium on Experimental Algorithms, SEA 2018, June 27-29, 2018, L’Aquila, Italy*, volume 103 of *LIPIcs*, pages 2:1–2:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018. URL: <https://doi.org/10.4230/LIPIcs.SEA.2018.2>, doi: 10.4230/LIPIcs.SEA.2018.2.
- [194] H. D. Simon. Partitioning of Unstructured Problems for Parallel Processing. *Computing Systems in Engineering*, 2(2):135–148, 1991.
- [195] R. Singh, J. Xu, and B. Berger. Pairwise global alignment of protein interaction networks by matching neighborhood topology. In *Annual international conference on research in computational molecular biology*, pages 16–31. Springer, 2007. URL: https://doi.org/10.1007/978-3-540-71681-5_2, doi: 10.1007/978-3-540-71681-5_2.
- [196] G. M. Slota, K. Madduri, and S. Rajamanickam. Complex network partitioning using label propagation. *SIAM Journal of Scientific Computing*, 38(5), 2016. doi: 10.1137/15M1026183.
- [197] G. M. Slota, S. Rajamanickam, K. D. Devine, and K. Madduri. Partitioning trillion-edge graphs in minutes. In *2017 IEEE International Parallel and Distributed Processing Symposium, IPDPS*, pages 646–655, 2017. doi: 10.1109/IPDPS.2017.95.
- [198] G. M. Slota, C. Root, K. D. Devine, K. Madduri, and S. Rajamanickam. Scalable, multi-constraint, complex-objective graph partitioning. *IEEE Transactions on Parallel Distributed Systems*, 31(12):2789–2801, 2020. doi: 10.1109/TPDS.2020.3002150.
- [199] I. Stanton. Streaming balanced graph partitioning algorithms for random graphs. In *Proceedings of the twenty-fifth annual ACM-SIAM symposium on Discrete algorithms*, pages 1287–1301. SIAM, 2014. doi: 10.1137/1.9781611973402.95.
- [200] I. Stanton and G. Kliot. Streaming graph partitioning for large distributed graphs. In *Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 1222–1230, 2012. doi: <https://doi.org/10.1145/2339530.2339722>.
- [201] G. L. Steele. Making Asynchronous Parallelism Safe for the World. In *POPL 90*, pages 218–231. ACM Press, 1990. doi: 10.1145/96709.96731.
- [202] F. Taşyaran, B. Demireller, K. Kaya, and B. Uçar. Streaming hypergraph partitioning algorithms on limited memory environments. *arXiv preprint arXiv:2103.05394*, 2021.
- [203] V. E. Taylor and B. Nour-Omid. A Study of the Factorization Fill-In for a Parallel Implementation of the Finite Element Method. *International journal for numerical methods in engineering*, 37(22):3809–3823, 1994.
- [204] A. Trifunovic and W. J. Knottenbelt. Towards a Parallel Disk-Based Algorithm for Multilevel k-way Hypergraph Partitioning. In *18th International Parallel and Distributed Processing Symposium*. IEEE Computer Society, 2004. URL: <https://doi.org/10.1109/IPDPS.2004.1303286>, doi: 10.1109/IPDPS.2004.1303286.
- [205] C. R. Trott, D. Lebrun-Grandié, D. Arndt, J. Ciesko, V. Dang, N. Ellingwood, R. Gayatri, E. Harvey, D. S. Hollman, D. Ibanez, N. Liber, J. Madsen, J. Miles, D. Poliakoff, A. Powell, S. Rajamanickam, M. Simberg, D. Sunderland, B. Turcksin, and J. Wilke. Kokkos 3: Programming model extensions for the exascale era. *IEEE Transactions on Parallel and Distributed Systems*, 33(4):805–817, 2022. doi: 10.1109/TPDS.2021.3097283.
- [206] C. Tsourakakis, C. Gkantsidis, B. Radunovic, and M. Vojnovic. Fennel: Streaming graph partitioning for massive scale graphs. In *Proceedings of the 7th ACM international conference on Web search and data mining*, pages 333–342, 2014. doi: <https://doi.org/10.1145/2556195.2556213>.
- [207] B. Uçar and C. Aykanat. Encapsulating multiple communication-cost metrics in partitioning sparse rectangular matrices for parallel matrix-vector multiplies. *SIAM J. Sci. Comput.*, 25(6):1837–1859, 2004. URL: <https://doi.org/10.1137/S1064827502410463>, doi: 10.1137/S1064827502410463.
- [208] J. Ugander and L. Backstrom. Balanced label propagation for partitioning massive graphs. In *Sixth ACM International Conference on Web Search and Data Mining, WSDM*, pages 507–516. ACM, 2013. URL: <https://doi.org/10.1145/2433396.2433461>, doi: 10.1145/2433396.2433461.
- [209] S. Umeyama. An eigendecomposition approach to weighted graph matching problems. *IEEE transactions on pattern analysis and machine intelligence*, 10(5):695–703, 1988. doi: 10.1109/34.6778.
- [210] H. Ushijima-Mwesigwa, C. F. A. Negre, and S. M. Mniszewski. Graph partitioning using quantum annealing on the d-wave system. In *Proceedings of the Second International Workshop on Post Moores Era Supercomputing, PMES’17*, page 22–29, New York, NY, USA, 2017. Association for Computing Machinery. URL: <https://doi.org/10.1145/3149526.3149531>, doi: 10.1145/3149526.3149531.

- [211] L. M. Vaquero, F. Cuadrado, D. Logothetis, and C. Martella. Adaptive partitioning for large-scale dynamic graphs. In *2014 IEEE 34th international conference on distributed computing systems*, pages 144–153. IEEE, 2014. doi:10.1109/ICDCS.2014.23.
- [212] J. T. Vogelstein, J. M. Conroy, V. Lyzinski, L. J. Podrazik, S. G. Kratzer, E. T. Harley, D. E. Fishkind, R. J. Vogelstein, and C. E. Priebe. Fast approximate quadratic programming for graph matching. *PLOS one*, 10(4):e0121002, 2015. doi:10.1371/journal.pone.0121002.
- [213] K. von Kirchbach, M. Lehr, S. Hunold, C. Schulz, and J. L. Träff. Efficient process-to-node mapping algorithms for stencil computations. In *IEEE International Conference on Cluster Computing (CLUSTER)*, pages 1–11. IEEE, 2020. doi:10.1109/CLUSTER49012.2020.00011.
- [214] M. von Looz, C. Tzovas, and H. Meyerhenke. Balanced k-means for parallel geometric partitioning. In *Proceedings of the 47th International Conference on Parallel Processing, ICPP*, pages 52:1–52:10. ACM, 2018. URL: <https://doi.org/10.1145/3225058.3225148>, doi:10.1145/3225058.3225148.
- [215] M. von Looz, M. Wolter, C. R. Jacob, and H. Meyerhenke. Better partitions of protein graphs for subsystem quantum chemistry. In *15th International Symposium on Experimental Algorithms, SEA*, volume 9685 of LNCS, pages 353–368. Springer, 2016. URL: https://doi.org/10.1007/978-3-319-38851-9_24, doi:10.1007/978-3-319-38851-9_24.
- [216] C. Walshaw and M. Cross. Multilevel mesh partitioning for heterogeneous communication networks. *Future Generation Comp. Syst.*, 17(5):601–623, 2001. URL: [https://doi.org/10.1016/S0167-739X\(00\)00107-2](https://doi.org/10.1016/S0167-739X(00)00107-2).
- [217] C. Walshaw and M. Cross. JOSTLE: Parallel Multilevel Graph-Partitioning Software – An Overview. In *Mesh Partitioning Techniques and Domain Decomposition Techniques*, pages 27–58. Civil-Comp Ltd., 2007.
- [218] C. Walshaw, M. Cross, and M. G. Everett. Parallel Dynamic Graph Partitioning for Adaptive Unstructured Meshes. *Journal of Parallel and Distributed Computing*, 47(2):102–108, 1997.
- [219] C. Walshaw. *An Exploration of Multilevel Combinatorial Optimisation*, pages 71–123. Springer US, 2003. doi:10.1007/978-1-4757-3748-6_2.
- [220] C. Walshaw, M. Cross, and M. G. Everett. Parallel dynamic graph partitioning for adaptive unstructured meshes. *J. Parallel Distributed Comput.*, 47(2):102–108, 1997. URL: <https://doi.org/10.1006/jpdc.1997.1407>, doi:10.1006/jpdc.1997.1407.
- [221] L. Wang, Y. Xiao, B. Shao, and H. Wang. How to Partition a Billion-Node Graph. In *IEEE 30th International Conference on Data Engineering*, pages 568–579. IEEE Computer Society, 2014. URL: <https://doi.org/10.1109/ICDE.2014.6816682>, doi:10.1109/ICDE.2014.6816682.
- [222] F. Wilcoxon. Individual Comparisons by Ranking Methods. In *Breakthroughs in Statistics*, pages 196–202. Springer, 1992. doi:10.1007/978-1-4612-4380-9_16.
- [223] R. D. Williams. Performance of Dynamic Load Balancing Algorithms for Unstructured Mesh Calculations. *Concurr. Pract. Exp.*, 3(5):457–481, 1991. URL: <https://doi.org/10.1002/cpe.4330030502>, doi:10.1002/cpe.4330030502.
- [224] N. Xu, L. Chen, and B. Cui. Loggp: a log-based dynamic graph partitioning method. *Proceedings of the VLDB Endowment*, 7(14):1917–1928, 2014. doi:10.14778/2733085.2733097.
- [225] H. Yang and D. F. Wong. Efficient network flow based min-cut balanced partitioning. In *Proceedings of the 1994 IEEE/ACM International Conference on Computer-Aided Design, ICCAD*, pages 50–55. IEEE Computer Society / ACM, 1994. doi:10.1109/ICCAD.1994.629743.
- [226] W. Yang, G. Wang, K. R. Choo, and S. Chen. Hepart: A balanced hypergraph partitioning algorithm for big data applications. *Future Gener. Comput. Syst.*, 83:250–268, 2018. doi:10.1016/j.future.2018.01.009.
- [227] L. Yao, P. Hong, W. Zhang, J. Li, and D. Ni. Controller placement and flow based dynamic management problem towards SDN. In *IEEE International Conference on Communication, ICC*, pages 363–368. IEEE, 2015. doi:10.1109/ICCW.2015.7247206.
- [228] A. Yasar, M. F. Balin, X. An, K. Sancak, and Ü. V. Çatalyürek. On symmetric rectilinear matrix partitioning. *CoRR*, abs/2009.07735, 2020. URL: <https://arxiv.org/abs/2009.07735>, arXiv:2009.07735.
- [229] A. Yasar and Ü. V. Çatalyürek. Heuristics for symmetric rectilinear matrix partitioning. *CoRR*, abs/1909.12209, 2019. URL: <https://arxiv.org/abs/1909.12209>, arXiv:1909.12209.
- [230] G. A. Young, T. A. Severini, G. A. Young, R. Smith, R. L. Smith, et al. *Essentials of Statistical Inference*, volume 16. Cambridge University Press, 2005.
- [231] X. Yu, H. Ma, Z. Qu, J. Fang, and W. Liu. Numa-aware optimization of sparse matrix-vector multiplication on armv8-based many-core architectures. In *IFIP International Conference on Network and Parallel Computing*, volume 12639 of LNCS, pages 231–242. Springer, 2020. doi:10.1007/978-3-030-79478-1_20.
- [232] M. Zaslavskiy, F. Bach, and J.-P. Vert. A path following algorithm for the graph matching problem. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 31(12):2227–2242, 2008. doi:10.1109/TPAMI.2008.245.
- [233] W. Zhang, Y. Chen, and D. Dai. Akin: A streaming graph partitioning algorithm for distributed graph storage systems. In *2018 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*, pages 183–192. IEEE, 2018. doi:10.1109/CCGRID.2018.00033.

- [234] D. Zheng, X. Song, C. Yang, D. LaSalle, Q. Su, M. Wang, C. Ma, and G. Karypis. Distributed hybrid CPU and GPU training for graph neural networks on billion-scale graphs. *CoRR*, abs/2112.15345, 2021. URL: <https://arxiv.org/abs/2112.15345>, [arXiv:2112.15345](https://arxiv.org/abs/2112.15345).
- [235] X. Zhu and Z. Ghahramani. Learning from labeled and unlabeled data with label propagation. *Technical Report*, 2002.

最新的（超）图划分

摘要

近年来，平衡超图划分算法的设计和评估取得了重大进展。我们调查了过去十年平衡超图划分实用算法的趋势以及未来的研究方向。我们的工作是对之前有关该主题的调查的更新，特别是该调查不仅扩展了之前的调查，还涵盖了超图划分和流算法，并额外关注并行算法。

第 1 章 简介

图是一种通用且广泛使用的对对象之间关系进行建模的方法。它们可用于社交网络、模拟网格、道路网络/路线规划、（图）神经网络等广泛的应用。随着数据集的爆炸式增长，图处理方法的可扩展性已成为越来越大的挑战。随着该领域的巨大问题实例变得越来越丰富，需要可扩展的算法来执行分析。通常（超）图划分是使各种算法在实践中扩展的关键技术。更准确地说，适用于各种应用程序的可扩展算法通常需要一个子例程，将（超）图划分为大小大致相等的块，以使块之间运行的边数最小化。平衡最常使用平衡约束来建模，该约束要求所有块权重低于给定的上限。需要这种（超）图划分作为关键子例程的应用程序数量巨大。社交网络运营商使用图划分技术来平衡其操作的负载并确保站点响应时间较短；有效的路线规划算法依赖于需要对道路网络进行划分的预计算阶段；在超级计算机上运行的科学模拟使用（超）图划分来平衡负载并最大限度地减少处理器之间的通信；划分（图）神经网络有助于加快训练时间；（超）图划分有助于改善 VLSI 设计中的芯片布局。另一方面，即使对于最大度数为三的未加权树，该问题也是 NP 困难的，并且不存在常数因子近似算法。因此，在实践中使用启发式算法。本文的目的是对丰富的文献进行结构化概述，明确强调解释关键思想并讨论过去十年中发表的、其他概述中缺失的工作。更准确地说，我们的工作是对之前有关该主题的一般调查的更新；特别是，我们还将调查范围扩展到超图划分算法。

还有其他有关该主题的（较旧的）调查。Bichot 和 Siarry 的书涵盖了图划分技术，例如多级方法、元启发法、并行方法和超图划分，以及图划分的应用。Schloegel 等人的调查于世纪之交出版，重点关注科学计算技术，包括自适应和动态模拟算法以及过程映射算法。还讨论了静态算法以及具有多个目标和约束的公式。莫尼恩等人讨论多级算法。他们的描述重点是基于匹配的粗化和使用顶点交换启发式的局部搜索。金等人涵盖模因算法。阿巴斯等人根据流图划分技术的假设、目标和成本对流图划分技术进行描述和分类，并对不同应用和数据集的算法进行实验比较。Buluç 等人关于该主题的最后一次一般调查涵盖了 2013 年或之前发布的广泛技术和实用算法。对于超图划分，存在两项较旧的调查。Alpert 和 Kahng 讨论了最小切割和比率切割双划分公式以及多路扩展、约束驱动划分和复制划分。Papa 和 Markov 讨论了超图划分、精确算法和各种本地搜索启发式问题的实际应用以及软件包和基准测试。Schlag 在他的论文中介绍了超图技术的最新概述。

我们的调查结构如下。我们首先在第 3 节中介绍问题和其他预备知识。然后在第 4 节中，我们重点关注过去十年中出现的新应用程序。我们在第 5 节中继

续介绍新颖的顺序技术。然后我们在第 6 节中继续介绍并行算法。我们在第 7 节中描述实验方法。我们在第 8 节中总结未来的挑战。

第 2 章 绪论

目前该领域非常活跃。一方面，流算法速度非常快，消耗很少的内存，但只能产生低质量的解决方案。这些算法假设单台机器的内部存储器与顶点数量成正比，并且通常会一一加载节点及其邻域以直接对块进行分配。因此，图的边不必适合机器的内存。另一方面，有多种顺序内存算法可以解决划分问题，这些算法适合单台机器的主内存，而不使用并行性。该领域的最新进展提供了一组丰富的算法，能够计算非常高质量的划分。这些高质量算法通常还具有共享内存并行算法。虽然该领域的顺序内存/并行共享内存非元启发式算法对于大多数应用程序使用合理的时间，但也有模因或进化算法投入大量资源来实现更高的质量。请注意，模因或进化算法可以是顺序内部存储器、共享存储器甚至分布式存储器并行的。由于其运行时间较长，此类算法通常仅用于具有数十万个节点的图。精确求解器目前只能求解非常小的实例，同时还需要大量时间来求解实例（即使是几个块）。对于大量分块，精确求解器目前效果不佳。分布式并行算法可以很好地扩展到大型实例，但如果这些工具不实现多级策略，它们的质量通常比顺序内存划分算法低得多。即使此类算法使用多级方案，在此计算模型中运行的算法的解质量通常也比其内部存储器/共享内存对应物更差。然而，这种计算模式的优点是可以非常快速地划分巨大的实例，并且使研究人员能够在廉价的机器上划分巨大的图。最后，研究人员还致力于研究 GPU 等其他平台的算法，以便能够充分利用现有硬件的功能。然而，GPU 的内存大小有限，并且当前在 GPU 计算划分上运行的算法比高质量内部内存方案切割的边缘要多得多。

第 3 章 预备知识

3.1 符号说明

图与超图 定义一个有权无向超图 $H = (V, E, c, \omega)$ 由 n 个顶点集 V ， m 条超边/网集 E ，超边权重 $c: V \rightarrow R_{>0}$ 与网权重 $\omega: V \rightarrow R_{>0}$ ，其中网 e 是顶点集 V 的子集（即 $e \subseteq V$ ）， R 为实数集。网的顶点称为引脚。我们以自然的方式将 c 和 ω 扩展到集合，即 $c(V) := \sum_{v \in V} c(v)$ ， $\omega(E) := \sum_{e \in E} \omega(e)$ 。称顶点 v 与网 e 相关，若 $v \in e$ 。 $I(v)$ 表示所有与顶点 v 相关的网。集合 $\Gamma(v) := \{u \mid \exists e \in E: \{v, u\} \subseteq e\}$ 表示所有顶点 v 的邻接顶点。定义顶点 v 的度为 $d(v) := |I(v)|$ 。我们假设超边是集合而不是多重集，即一个顶点只能包含在超边中一次。大小为 1 的网称为单点网。给定子集 $V' \subset V$ ，定义子超图 $H_{V'}$ 为 $H_{V'} := (V', \{e \cap V' \mid e \in E: e \cap V' \neq \emptyset\})$ 。

定义一个有权无向图 $G = (V, E, c, \omega)$ 由 n 个顶点集 V ， m 条超边/网集 E ，超边权重 $c: V \rightarrow R_{>0}$ 与网权重 $\omega: V \rightarrow R_{>0}$ ，其中网 e 是顶点集 V 的子集（即 $e \subseteq V$ ）， R 为实数集。与超图不同，图的边的大小限制为两个。令 $G = (V, E, c, \omega)$ 为一个有权（有向）图。当提及超图时，我们使用超边/网；当提及图时，我们使用边。然而，我们使用相同的符号来指代顶点权重 c 、边权重 ω 、顶点度数 $d(v)$ 和邻接集合 $\Gamma(v)$ 。在无向图中，由边 $(v, u) \in e$ 可得边 $(u, v) \in e$ ，且 $\omega(v, u) = \omega(u, v)$ 。

划分和集群 （超）图 H 的 k 路划分指将顶点集分配到 k 个分块 $\Pi =$

$\{V_1, \dots, V_k\}$ 的划分, 使得 $\bigcup_{i=1}^k V_i = V, V_i \neq \emptyset, 1 \leq i \leq k$ 且 $V_i \cap V_j = \emptyset, i \neq j$ 。我们称 k 路划分 Π 为 ε -平衡划分, 若每个分块 $V_i \in \Pi$ 满足平衡条件: 对于范围 ε , $c(V_i) \leq L_{max} := (1 + \varepsilon) \left\lceil \frac{c(V)}{k} \right\rceil$ 。我们称一个分块 V_i 是过载的, 若 $c(V_i) > L_{max}$ 。对于任意网 e , $\Lambda(e) := \{V_i \mid V_i \cap e \neq \emptyset\}$ 表示所有与网 e 的连通分块。网 e 的连通度 $\lambda(e)$ 指其连通分块集合的基数, 即 $\lambda(e) := |\Lambda(e)|$ 。称一个网为割网, 若 $\lambda(e) > 1$, 否则, 称其为内网。与至少一个割网相关的顶点称为边界顶点。网 e 在分块 V_i 的引脚数定义为 $\Phi(e, V_i) := |\{V_i \cap e\}|$ 。称分块 V_i 与顶点 $v \notin V_i$ 相邻, 若 $\exists e \in I(v) : V_i \in \Lambda(e)$ 。我们用 $B(v)$ 来表示所有与顶点 v 相邻的分块的集合。给定一个 H 的 k 路划分 Π , 商图 $Q := (\Pi, \{(V_i, V_j) \mid \exists e \in E : \{V_i, V_j\} \subseteq \Lambda(e)\})$ 包含两两相邻的分块间的一条边。超图的聚类 $C = \{C_1, \dots, C_l\}$ 是其顶点集的一个划分。与 k 路划分不同, 聚类的路数目前未给定, 且不存在平衡约束。

(超) 图 H 的 k 路(超)边划分指将边集分配到 k 个分块 $\Pi = \{E_1, \dots, E_k\}$ 的划分, 使得 $\bigcup_{i=1}^k E_i = E, E_i \neq \emptyset, 1 \leq i \leq k$ 且 $E_i \cap E_j = \emptyset, i \neq j$ 。我们称 k 路划分 Π 为 ε -平衡划分, 若每个分块 $E_i \in \Pi$ 满足平衡条件: 对于范围 ε , $\omega(E_i) \leq L_{max} := (1 + \varepsilon) \left\lceil \frac{\omega(E)}{k} \right\rceil$ 。称一个顶点为割顶点, 若该顶点包含不同块中的两条或多条边。一路(超)边划分的(有权)顶点切割是割顶点的总权重。

3.2 (超) 图的 k 路划分问题

问题定义 (超) 图的 k 路划分问题是要求找到(超)图 $H = (V, E, c, \omega)$ 的一个 ε -平衡划分 Π 从而最小化割网上某个值为 ε 的目标函数。当我们处理超图时, 两个最常用的成本函数是割网度量 $f_c(\Pi) := \sum_{e \in E'} \omega(e)$ 和连通度量 $f_\lambda(\Pi) := \sum_{e \in E'} (\lambda(e) - 1) \omega(e)$, 其中 E' 是所有割网的集合。割网度量是划分 Π 中所有连接了不只一个分块的网的权重的求和, 连通度量还考虑了割网连接的分块的具体数量 λ 。当进行图的划分时, 目标通常是减小 $\sum_{i < j} \omega(E_{ij})$, 其中 $E_{ij} := \{\{u, v\} \in$

$E \mid u \in V_i, v \in V_j\}$ 。注意到目标函数与顶点位于不同块中的边的权重之和有关。

除了这些常见的成本函数之外, 还存在其他专门的目标函数。图 1 显示了图的划分示例。在整篇论文中, 如果相应的目标函数与其他工具相比较小, 我们使用“高质量”一词, 如果情况相反, 则使用“低质量”一词。

第 4 章 应用

图划分算法的研究一直受到其众多应用的推动。一些传统的应用是并行处理(例如, 在科学计算中)、VLSI 设计、路线规划和图像分割; 另请参阅之前的调查。然而, 近年来出现了一些新的应用。

分布式数据库和查询优化 分布式数据库和查询优化。一个突出的应用是分布式数据库分片。顶点代表数据记录, 超边代表访问记录的查询, 而分块对应于分片(机器)。因此, 最小化连接性相当于最小化查询中涉及的平均分片数量, 从而优化延迟和查询处理时间。

类似的应用是通过提高缓存利用率来提高 Google 搜索引擎后端的 IO 吞吐量。Archer 等人初始化一个分配搜索请求的投票表，使用二分图上的图分区，其中顶点是搜索词和查询，并且查询中包含的每个词都存在一条边。随后的模拟和细化进一步提高了投票表的预测准确性（缓存中的项目）。

Li 等人使用图划分来构造集合相似性搜索索引。该方法首先使用索引过滤候选集，然后暴力检查剩余的候选集。顶点表示集合，边连接最近邻的集合（或在相似性阈值内，取决于查询类型），并且切割大小与修剪效率相关。

数据分发与调度 Djidjev 使用图分区来加速和并行化分子动力学模拟的密度矩阵计算。格子玻尔兹曼流体模拟属于图分区的经典负载平衡和通信最小化应用类别。Yu 等人考虑稀疏矩阵向量乘法，但是关注于具有许多 NUMA 节点的共享内存机器而不是分布式系统。

Pregel 或 Giraph 等并行图处理系统已成为网络分析任务的广泛使用工具。这些系统采用类顶点思考或类边思考编程范例，需要跨机器对顶点或边进行平衡划分。基于标签传播的流方法比传统的基于范围或基于哈希的分区具有更好的性能。

量子电路模拟 有几篇论文在超图上采用嵌套剖析来寻找良好的收缩树，以加速经典非量子机器上量子电路的模拟。这可以用来通过实验验证量子电路的正确性，并驳斥已经实现量子霸权的过早说法。

解决 SAT 布尔可满足性 (SAT) 公式可以表示为超图。一种表示是双重表示，其中子句作为顶点，变量作为超边，由包含变量（否定或非否定）的子句组成。Mann 和 Papp 使用平衡二分法来识别分支求解器应首先关注分配的变量，这样一旦分配了剪切中的变量，公式就会分为两个。两个子公式可以独立求解，预计速度会更快。大多数 SAT 求解器采用启发式 (VSIDS)，将优先级（观察到的冲突子句中的频率）分配给变量，选择最高优先级进行下一个分支。作者考虑了两种变体，后者更成功：首先强制拆分公式并使用 VSIDS 进行平局打破，或者使用分区初始化优先级并让 VSIDS 否决决策。

其它 Lamm 等人在基于分离器分解和图划分的独立集的进化框架中制定重组算子。给定两个独立集，它们在分离器分解的两个块中的顶点被交换，产生使用局部搜索细化的两个后代独立集。

Kumar 等人使用聚类公式来计算视频中移动对象的轨迹。帧中检测到的对象对应于顶点，每个块对应于一个对象。作者强制执行特殊约束，使得每个分区每帧仅包含一个对象，并且对象不会在帧之间跳转。

Yao 等人考虑在软件定义网络中放置控制单元，以最小化从交换机（节点）到控制器的消息的平均延迟。首先，根据消息容量估计必要的控制器数量，然后对开关图进行分区，并在每个块中放置一个控制单元，使用单独的例程在块内执行放置。

密度泛函理论等量子化学模拟表现出二次时间复杂度，这就是为什么使用很大程度上独立的子系统上的计算来近似和加速该过程的原因。Von Looz 等人建议使用图分区来自动化子系统构建过程，其中小边缘切割对应于引入的小计算误差。

第 5 章 顺序式技术

5.1 经典（超）图划分技术

本节讨论本次调查中重复使用的分区技术。我们简要概述了多级范式和最常见的本地搜索算法。这些算法根据增益值移动顶点。增益值 $g_u(V_j)$ 反映了当我们

将顶点 u 从当前块 V_j 移动到目标块时目标函数的变化（例如，边缘切割的减少）。

多层次范式 解决（超）图划分问题最成功的方法是多级范式。它由三个阶段组成。在粗化阶段，通过收缩匹配或顶点簇来创建连续较小且结构相似（超）图的层次结构。一旦（超）图足够小，初始分区算法就会获得最粗糙（超）图的分区。在去粗化阶段，分区被投影到层次结构中的下一个更大的（超）图，并且在每个级别，局部搜索算法改进目标函数（例如，边缘切割）。图 3 说明了多级范例。

将多个顶点收缩为超顶点会在超顶点中聚合它们的权重。为了进一步减小较粗糙（超）图的大小，还可以删除除一条之外的所有变得相同的边，在该边聚合它们的权重（丢弃自循环）。这样，当将分区投影到层次结构中的下一个较大的超图时，可以获得具有相同平衡和切割属性的划分。

标签传播算法 如图 2 所示，标签传播算法最初被提出用于检测大规模网络中的社区结构，但也被用作分区上下文中的细化技术。标签传播算法按轮进行，每个顶点 u 都与一个标签 $L[u]$ 相关联。最初，每个顶点都被分配了自己的标签（即 $L[u] = u$ ）。在每一轮中，顶点都按某种顺序被访问，并且每当访问一个顶点时，它都会将其标签调整为在其邻域中最频繁出现的标签（关系被随机断开）。该算法将继续进行，直到达到预定义的轮次，或者没有一个顶点在一轮中改变其标签。该算法可以用作粗化阶段的聚类算法，或者当当前路分区用于初始标签分配时用作局部搜索算法。

Fiduccia-Mattheyses 算法 Fiduccia 和 Mattheyses 提出了第一个平衡二分问题的线性时间启发式算法（称为 FM 算法）。该算法按遍运行，并且顶点在一次遍中最多可以更改其块一次。FM 算法使用两个优先级队列（每个块一个）。最初，它将每个边界顶点插入到优先级队列中，以及将顶点移动到相反块的增益。每个步骤重复执行具有最高增益且不违反平衡约束的移动，随后更新所有未移动邻居的增益。当所有顶点都移动或平衡约束阻止进一步移动时，遍历结束。FM 算法还执行负增益移动，因此能够摆脱局部最优。最后，它恢复到传递过程中最佳看到的解决方案。

5.2 最新多级划分

本节讨论多级分区上下文中的最新发展。我们从两种新的多级分区方案开始，这两种方案在块数量很大或需要高解决方案质量的情况下更适用。然后，我们仔细研究多级方案的不同阶段，并重点介绍最近的算法改进。

深度多级划分 （超）图的 k 路分区可以通过递归二分区（RB）或直接 k 路分区来获得。前者计算输入（超）图的二分，然后在两个块上递归，直到（超）图被划分为所需数量的块。后者将（超）图直接划分为块，并应用单向局部搜索算法来改进解决方案。

最近，这些方法已被推广到深度多级划分。该方法继续粗化，直到只剩下 $2X$ 个顶点（其中 X 是输入参数），并计算最粗（超）图的初始二分。在去粗化阶段，当块的大小大于 $2X$ 个顶点时，只要块数小于 k ，它就会将块二分。

深度多级方法结合了 RB 和 direct 路方案的优点：它在粗化阶段递归地对

大小为 $O(1)$ 的块进行二分，从而能够使用 k 路局部搜索算法，而 RB 方案递归地对大小为 $O(k)$ 的块进行二分，而 direct 路方案则恢复为 RB 进行初始分区。图 3 说明了多级方案的不同实例。

n 级（超）图划分 多级层次结构的深度提供了多级算法的运行时间和解决方案质量之间的权衡。更多级别提供了在不同粒度上“改进当前解决方案的更多机会”，但需要快速的本地搜索算法来实现合理的运行时间。多级范式的最极端版本是在每个级别仅收缩一个顶点，这会产生几乎具有 n 级别的层次结构。这项技术首先由 Osipov 等人研究用于图分区，后来由 Schlag 等人进一步改进用于超图划分。该方法通过使用经典 FM 算法的高度本地化变体而变得可行，该算法仅使用未契约节点初始化优先级队列，并将搜索扩展到移动节点的邻居。此外，该实现使用增益缓存来避免昂贵的移动增益重新计算（将 FM 算法的运行时间减少 45%），并使用自适应停止规则，如果不太可能找到进一步的改进，则提前终止搜索（将运行时间减少一个数量级）。存在 n 层方案的并行版本，它在每个层上并行解包固定数量的顶点（而不是单个顶点）。

粗化 粗化阶段的目的是计算输入（超）图的连续较粗的近似值，从而保持其结构属性，并在某种意义上充当过滤器，从搜索空间中删除尽可能多的不必要的信息。在过去的几年中，研究重点是复杂网络的聚类技术以及利用有关（超）图的社区结构的信息来增强粗化过程。

Meyerhenke 等人使用大小约束标签传播算法来计算顶点聚类，然后将其收缩以形成多级层次结构。与以前使用的基于匹配的方法相比，聚类算法可以更有效地减小复杂网络的大小（幂律度分布）。后者存在的问题是，与高度顶点关联的顶点常常保持不匹配。因此，Davis 等人提出了几种减少不匹配顶点数量的技术。如果两个不相邻的不匹配顶点共享一个公共邻居（也称为 2 跳匹配）或者它们与已匹配的两个顶点相邻，则它们被匹配。此外，可以使用不匹配的邻居扩展两个匹配的顶点（三路匹配）。

以下两种技术重新计算输入（超）图的（超）边权重，并将它们用作多级算法的输入。新的权重编码了有关（超）边重要性的更多信息，并且应该防止粗化算法将（超）边折叠成良好分区的切割集中的单个顶点。Glantz 等人将边 $e \in E$ 的权重定义为由其中有割边 e 的生成树引起的所有二分的 $\min$ 电导值。二分法

(V_1, V_2) 的电导值为 $cond(V_1, V_2) := \frac{\omega(E'(V_1, V_2))}{\min\{vol(V_1), vol(V_2)\}}$ ，其中 $vol(V_i)$ 是与 V_i 中的顶点

相关的所有边的权重。Chen 和 Safro 使用超边的两个引脚之间的最大代数距离作为其权重。

Lotfifar 和 Johnson 计算超边缘聚类，并将一个顶点分配给包含大部分关联网络的聚类。他们基于匹配的粗化算法使用匹配顶点必须属于同一聚类的限制。Heuer 和 Schlag 还使用顶点聚类来限制超图的密集耦合区域的收缩。他们的算法将超图转换为其二分图表示，然后使用 Louvain 算法最大化模块化目标函数以利用其社区结构。

Shaydulin 和 Safro 使用代数多重网格以及稳定匹配的思想，其中偏好列表确定了基于聚合的粗化的良好节点组合。作者将他们的方法集成到 Zoltan 工具中。他们使用 Zoltan、hMetis 和 PaToH 的实验结果表明，在同样的改进下，所提出的方案至少与传统的基于匹配的方案一样有效，同时许多情况下优于它们。

初始划分 粗化通常会持续进行，直到保留 $\Omega(k)$ 顶点。基于直接 k 路方案的分区器通常使用多级递归双向分区来获得最粗（超）图的初始路划分。Heuer

表明, 这比平面分区技术产生更好的初始划分。为了获得初始 k 划分, 许多分区器多次运行不同平面 k 划分算法的组合 (例如, 贪婪图增长、随机或频谱划分), 然后进行标签传播或 FM 局部搜索, 并使用这些运行中的最佳二分区继续去粗化。Preen 和 Smith 表明, 何时停止粗化的决定通常取决于实例, 并提出了自适应停止标准 (而不是对所有实例使用相同的恒定数量的顶点)。

细化 Sanders 和 Schulz 提出了一种基于流的双分区细化技术, 该技术被调度在用于 k 路划分的块对上。这个想法是选择二分 $\Pi = \{1, 2\}$ 割线周围的顶点子集 R 。通过将 $1 \setminus R$ 收缩到源点并将 $2 \setminus R$ 收缩到汇点来构建流网络。由此产生的最大流量导致可能改进的切割, 这可能违反平衡约束。因此, $|R|$ 自适应地选择, 并使用强连通分量计算来表征所有最小割集, 然后可以搜索平衡二分。Heuer 等人将这种方法推广到超图。Gottesbüre 等人通过直接在超图上运行流算法来加速基于流的超图细化。通过结合 FlowCutter (它解决了增量流问题, 以权衡切割尺寸以获得更好的平衡), 他们更有效地探索解决方案空间, 从而导致具有较小切割/连接的分区。示例如图 4 所示。FlowCutter 是一种迭代算法, 从给定源和接收器之间的最小切割 (以及相应的二分) 开始。如果二分不平衡, 则通过将其所有顶点以及附加顶点 (穿透节点) 收缩到相应的终端来增长较小的块。该顶点被选择为与切割相关, 并且如果可能的话, 在接下来的迭代中不增加切割尺寸。由此产生的流问题是嵌套的, 即终端集只会增长, 这使得来自先前迭代的流分配可行, 但违反了其最大值。

Henzinger 等人使用整数线性规划 (ILP) 直接细化 k 路分区。与基于流的细化类似, 选择允许移动的一小部分顶点区域。其余顶点收缩为每个块一个超级顶点。然后在该模型图上运行 ILP 公式。图 5 显示了一个示例。加速 ILP 的技术是通过将足够重的超级顶点固定到相应的块来为求解器提供启发式解决方案和对称性破缺。Ugander 和 Backstrom 使用线性规划从一组可能的移动中选择一个子集, 该子集在满足平衡约束的同时产生最大的削减减少。

在实验中, 基于流的 (超) 图细化被证明是高质量分区的关键要素, 即, 它比其他提高质量的方法 (例如使用 ILP、进化技术、重新启动、V 循环等) 具有更好的成本效益比。

顶点权重分布不均匀的划分 尽管现实世界的应用程序通常使用顶点和边权重来准确建模潜在问题, 但 (超) 图划分研究社区通常使用未加权的实例。多级算法结合了防止形成重顶点的技术 (限制最大允许顶点权重或将顶点权重作为惩罚项纳入粗化评级函数中), 但如果输入中已存在此类顶点, 那么就会相当困难, 就像许多现实世界实例的情况一样。

有两种方法可以在多级分区器中的严格平衡约束下计算平衡 k 路分区: (i) 确保初始分区找到平衡分区, 并且细化仅在满足平衡约束时才将节点移动到分区, 或者 (ii) 允许中间平衡违规并使用重新平衡技术来确保最终分区是平衡的。鉴于找到平衡路分区是一个 NP 困难问题 [70] (可简化为作业调度问题的最常见版本), 两种方法都不能保证平衡。

最近, Heuer 等人提出了一种技术, 使基于递归双分区的分区器能够在实践中可靠地计算平衡分区。这个想法是将最重顶点的一小部分预先分配给两个块之一 (视为固定顶点) 并优化剩余顶点的目标函数。

5.3 进化计算

遗传或进化算法 (GA) 从个体群体 (在我们的例子中, (超) 图的分区) 开

始，并在几个代周期或轮次中进化群体。如果遗传算法与局部搜索相结合，则称为模因算法（MA）。在每一轮中，遗传算法根据种群个体的适应度，使用选择规则来选择优良个体，并将它们组合起来以获得改进的后代。组合步骤通常使用重组操作来完成。当产生后代时，驱逐规则用于选择群体中的成员以被新的后代取代。对于进化算法来说，保持种群的多样性非常重要；即个体不应变得过于相似，以避免算法过早收敛。这通常是通过使用突变操作和使用考虑个体相似性的驱逐规则来实现的。文献中的所有算法都遵循一个相当通用的总体方案。因此，我们重点关注重组操作的描述。

Benlic 和 Hao 将大量顶点聚集在一起，这些顶点已分配给每个个体中的同一块，以执行重组操作并将其运算符与禁忌搜索相结合。重组操作的动机是观察到给定许多高质量的解决方案，在这些解决方案中总是有大量的顶点聚集在一起。

Sanders 和 Schulz 介绍了一种分布式进化算法 KaFFPaE（KaFFPaEvolutionary）。重组操作使用 KaHIP [176] 中多级图分区求解器的修改版本，该版本不会收缩在输入分区之一中切割的边。因此，两个输入个体中较好的一个可以用作初始划分，并且局部搜索可以在多级层次结构的多个级别上有效地交换解决方案的良好部分。此外，重组操作保证后代至少与两个输入个体中较好的一个一样好。

Ruiz 和 Segura 提出了一种模因算法，使用基于加权匹配的重组和多样性保留，类似于 Benlic 和 Hao 的算法。更准确地说，给定两个个体，重组操作计算二分图中的匹配，其中左侧代表第一个个体的块，右侧代表第二个个体的块。顶点之间的边根据相应块的交集的大小进行加权。在此二分图中，计算匹配，并将相应的交集用作新分区（后代）中集合的核心。此外，该算法使用带有负循环检测的局部搜索来进一步减少计算分区的边缘切割。当后代被插入群体时，具有最高相似性的个体将被驱逐。这里，两个个体之间的相似性基于上面定义的二分图中的匹配权重。

Henzinger 等人提供了一个基于整数线性规划的重组算子，并将其集成到 KaFFPaE 中。新的重组操作从群体的多个个体构建并求解整数线性程序。粗略地说，作者采用个体（分区）并通过收缩每个分区中同一块中的节点对来构建重叠图。然后使用整数线性规划在（小得多的）重叠图上解决原始分区问题，该整数线性规划根据具有最低切割值的分区使用块从属关系进行初始化。当多个分区具有相同的切割值时，随机选择一个。

Andre 等人将 KaFFPaE 推广到多级模因超图分区器 KaHyParE。他们还引入了适用于最佳个体的多级多点重组算子。该运算符会惩罚出现在许多父级中切割的网络中的顶点的收缩。Preen 和 Smith 使用自适应方案改进 KaHyParE 以停止粗化。

除了用于超图和图划分的模因算法之外，还有用于其他问题变体的算法。例如，Moreira 等人 and Popp 等人给出了非循环（超）图分区的模因算法，其中输入是非循环（超）图并且分区必须满足非循环性约束。此外，Schulz 和 Sanders 给出了一个分布式进化框架来计算图中的 k 路顶点分隔符。

5.4 流

如果（超）图分区算法满足以下条件，我们将其视为流式算法：(i) 它以某种顺序接收图的顶点或（超）边作为输入，(ii) 它不能将整个（超）图保留在内存中（通常它只能在顶点和块的数量上花费线性内存），以及 (iii) 它根据（超）

图的部分信息动态执行永久分区决策。更具体地说,流分区算法通常遵循图 6 中所示的迭代加载-计算-存储操作序列。该操作序列可用于分区顶点流或(超)边流。此外,它批量运行,一次可以包含单个顶点/(超)边或多个顶点/(超)边。

最常见的流模型是一次性模型,其中一次加载一个顶点及其邻接列表,然后应用一些逻辑将它们永久分配给块。算法逻辑可以像散列函数一样简单,也可以像基于某个目标对所有块进行评分然后将顶点分配给得分最高的块一样复杂。还有其他流模型,例如滑动窗口模型,其中包含下一个顶点的固定大小窗口保存在内存中以帮助单遍决策,以及缓冲流模型,其中连续加载缓冲区或批量顶点,立即分区,然后永久分配给块。

除了将顶点分配给块的传统流图划分方法之外,许多流图划分工作采用基于边的方法。为了消除歧义,我们将这种替代方法称为流式边缘分区,并将传统方法称为流式(超)图分区或流式顶点分区。在流式边划分中,边被一条一条地加载并直接动态分配给块,以最小化顶点切割。从应用程序的角度来看,流式边缘分区可用于分配由计算以边缘为中心的图表示的应用程序的工作负载。在这种情况下,分区的顶点切割表示将在多个块中复制或镜像的顶点数量,这通常与通信开销相关。另一个常见的指标是复制因子,它定义为复制顶点的数量除以图中的顶点总数。

5.4.1 一次性流式顶点划分

Stanton 和 Kliot 提出启发式方法来解决流模型中的图划分问题。他们最突出的启发式方法是一次性方法线性确定性贪婪(LDG),它产生具有最佳整体边缘切割的解决方案。在该算法中,顶点分配优先考虑包含更多邻居的块,并使用惩罚乘数来控制不平衡。特别地,顶点 v 被分配给最大化 $|V_i \cap \Gamma(v)|\Phi(i)$ 的块 V_i , 其中 $\Phi(i) := 1 - \frac{V_i}{L_{max}}$ 。直觉是,降级因素避免了使已经很重的块过载。如果目标函数存在联系,LDG 会将顶点移动到顶点较少的块。总体而言,LDG 在 $O(m + nk)$ 时间内对图进行分区。此外,作者还提出了一种基于散列的简单一次性方法,该方法具有运行时间 $O(n)$ 并且产生较差的边缘切割。

后来,Stanton 从更理论的角度研究了流顶点划分问题。作者证明,没有算法可以通过随机或对抗性流排序获得 $o(n)$ 近似。接下来,通过使用与有限 Polya Urn 过程的新颖耦合来分析随机贪婪算法的两个变体,这直观地解释了比较算法的性能。

Tsourakakis 等人提出了 Fennel, 一种简单的一次性划分算法,它采用了众所周知的聚类目标函数模块化。粗略地说,Fennel 将一个顶点 v 分配给一个块 V_i , 以便最大化 $|V_i \cap \Gamma(v)| - f(|V_i|)$ 类型的表达式,其中 $f(|V_i|)$ 是加性降级因子。更具体地说,作者使用 $f(|V_i|) = \alpha\gamma |V_i|^{\gamma-1}$ 定义了 Fennel 目标函数,其中 γ 是自由参数并且 $\alpha = m \frac{k^{\gamma-1}}{n^{\gamma}}$ 。经过作者的参数调整后,Fennel 使用 $\gamma = \frac{3}{2}$ 。尽管目标函数会惩罚不平衡的分区,但作者定义了强制平衡的硬约束的可能性。在他们的实验中,Fennel 切割的边缘比 LDG 少,并且在某些情况下,它切割的边缘数量与离线分区算法大致相同。

Zhang 等人提出 AKIN, 一种用于分布式图存储系统的流式顶点分区算法。AKIN 能够通过允许块之间的顶点随时间迁移来对预先未知节点数量 n 的图进行划分。分配决策主要基于顶点之间的相似度,用杰卡德相似系数来评估。给定

两个顶点的（部分）邻域，该系数定义为它们的交集与并集的比率。最初，AKIN 为每个顶点假定一个基本块，该基本块由哈希函数给出。该基本块被视为初步顶点分配以及用于获取分布式图存储中的信息的恒定时间索引。更具体地说，对于每个顶点，AKIN 在其基本块中存储一个固定长度的列表，其中包含其加载的具有最大度数的邻居。该列表用于计算顶点之间的相似度。一旦加载了一条边，AKIN 就会将其及其两个端点分配并迁移到块，从而最大限度地提高基于相似性的启发式算法。在实验评估中，与 Fennel 相比，每个顶点保留多达 100 个邻居的 AKIN 版本减少了更少的边，同时保持了同等的不平衡性，并且运行时间增加了 10%。

5.4.2 重新流式顶点划分

Nishimura 和 Ugander 引入了一种划分顶点的重新流方法。他们的方法是受同一张图多次流式传输的场景启发的。在他们的模型中，一次性分区算法可以多次遍历整个输入，同时迭代地减少边缘切割。作者提出了 ReLDG 和 ReFennel，它们分别是线性确定性贪婪（LDG）和 Fennel 的重新流改编。一方面，ReLDG 修改了 LDG 的目标，以仅考虑在计算块权重时当前通道期间执行的顶点分配。另一方面，ReFennel 在重播期间使用与 Fennel 相同的目标，但其附加平衡降级因子在每次通过后都会增加，以强制平衡。此外，作者证明，即使不增加降级因子，ReFennel 在有限次数的重播后也会收敛。他们的实验证实，他们的重新流媒体方法可以迭代地减少边缘切割。

Awadelkarim 和 Ugander 研究了顶点流式传输的顺序如何影响一次性顶点分区。作者引入了优先流的概念，其中（重新）流传输的顶点根据某些预定义的优先级静态或动态地重新排序。他们的方法是 ReLDG 的优先版本，使用重流算法的乘法权重，并根据平衡标签传播的启发调整流过程的顺序。在他们的实验中，作者考虑了广泛的流排序。最小整体边缘切割是使用基于其自身度量矛盾的动态顶点排序来获得的。该方法紧随其后的是基于顶点度数的静态排序。

5.4.3 缓冲流式顶点划分

Patwary 等人提出了 WStream，一种简单的流式顶点分区算法，在内存中保留滑动窗口。作者允许滑动窗口中有数百个顶点，以便在基于贪婪函数将顶点永久分配给块之前获得有关顶点的更多信息。一旦将一个顶点分配给一个块，就会将另一个顶点从输入流加载到滑动窗口中，从而保持窗口大小不变。在他们的实验中，对于大多数测试图来说，WStream 比 LDG 切割的边更少，比离线多级分区切割的边更多。

Jafari 等人使用缓冲流计算模型执行图分区。作者提出了一种共享内存算法，该算法从流输入中重复加载一批顶点，使用多级方案对其进行分区，然后将顶点永久分配给块。他们的多级方案基于一种简化的结构，其中单遍算法 LDG 用于粗化、计算初始分区并对其进行细化。他们通过简单地在处理器之间分割顶点，以以顶点为中心的方式并行化 LDG，从而产生多级方案的三个步骤的并行化。在他们的实验中，他们的算法比 LDG 切割更少的边缘，同时比离线分区算法更好的缩放。

Faraj 和 Schulz 提出了 HeiStream，一种在缓冲流模型中对顶点进行分区的算法。他们的算法加载一批顶点，构建图模型，然后使用多级算法对该模型进行分区。在他们的图模型中，分配给每个块的先前批次的顶点表示为固定到相应块

的单个大顶点。类似地，当前批次 b 的顶点 v 与前一批次 $\bar{b}$ 的顶点 $\bar{v}$ 之间的边由边 $(\bar{b}, b)$ 表示。此外，当当前批次的顶点具有来自未来批次（即尚未流式传输）的邻居时，他们的模型以收缩形式紧凑地表示该邻居。他们的多级算法具有传统的结构和组件，不同之处在于初始分区是 Fennel 的单遍执行，并且标签传播细化还优化了 Fennel 用于将顶点分配给块的目标函数。特别是，茴香目标扩展到加权图。在实验中，HeiStream 比 LDG、Fennel 和 Jafari 等人提出的缓冲流算法切割的边缘更少，同时对于大量块来说比 Fennel 更快。

5.4.4 超图的流式顶点划分

Alistarh 等人提出了 Min-Max，一种将超图的顶点分配给块的一次性流算法。对于每个块，该算法都会跟踪其中包含引脚的网络。这意味着 $O(mk)$ 的内存消耗，这比图分区的流式算法的典型内存消耗要多。加载顶点时，最小-最大将其分配给包含与其网络最大交集的块，同时遵守负载平衡的硬约束。作者从理论上证明，他们的算法能够以高概率恢复隐藏的共聚类，其中共聚类被定义为顶点和超边的同时聚类。在实验评估中，Min-Max 在负载不平衡方面优于五种直观的流方法，同时产生的解决方案的不平衡程度比 hMetis 等内存算法高出五倍。

Taşyaran 等人提出对 Min-Max 算法的改进。首先，作者提出了 Min-Max 的修改版本，其中对于每个网络，存储包含其引脚的块，而不是像 Min-Max 中那样存储每个块的网络。在他们的实验中，这个修改后的版本比最小-最大快三个数量级，同时保持相同的割网，但代价是内存增加了两倍。作者还介绍了三种使用比 Min-Max 更少内存的替代适应方法：限制内存使用的恒定上限、使用布隆过滤器来回答成员资格查询以及使用哈希函数来替换块和网络之间的连接信息。在他们的实验中，与最小-最大算法相比，他们的三种算法减少了运行时间，尤其是上限和散列方法，速度快了四个数量级。所有三种算法的内存消耗也都减少了，特别是布隆过滤器和散列方法，它们分别需要减少一个和两个数量级的内存。另一方面，这三种算法生成的解决方案的割网比最小-最大差，尤其是散列方法，它将割网度量提高了一个数量级。此外，作者提出了一种缓冲方法，其中最小-最大的修改版本在将顶点永久分配给块之前多次遍历顶点缓冲区。他们的实验结果表明，与 Min-Max 的快速修改版本相比，缓冲方法将分区质量提高了 5% 到 20%，但代价是显着的开销。

5.4.5 流式边划分

Bourse 等人从理论和实践的角度研究流边缘划分问题。作者首先为顶点划分提供了两种替代公式：一种基本公式，其目标是最小化传统的边缘切割度量；以及一种聚合公式，其目标是最小化修改的边缘切割度量，其中，对于每个顶点，每个块仅计算一个入射切割边缘。作者提供了两种类似的边划分公式，假设每个切割顶点都分配给一个主块并复制到包含其剩余边的块。在他们的基本公式中，目标是最小化与复制顶点相关的边的数量，即未分配给其端点之一的主块的边。在他们的聚合公式中，目标是最小化复制顶点的数量。假设顶点和边的均匀随机分配，作者表明基于聚合的边划分的预期成本始终小于或等于基于聚合的顶点划分的成本。此外，证明了顶点划分和边划分在其基本公式中存在一一对应的关系。作为推论，存在一种边缘划分的近似算法，该算法与顶点划分的最著名的近似比率相匹配。最后，作者提出了顶点分区和边分区的流版本的贪婪启发式方法。他们的实验证明了边缘划分相对于顶点划分的实际优势，并验证了他们的近似算法。

Petroni 等人提出了用于流边缘分区的“高度复制优先”(HDRF)算法。HDRF 算法旨在最大限度地减少顶点切割,通过显式地利用高度倾斜的幂律度分布来进行边缘分配决策。该算法背后的直觉是,高度顶点最好被切割,即被复制到几个块中,因为幂律图中的顶点很少。作者得出了其复制因子的平均情况上限。在他们的实验中, HDRF 在复制因子方面优于所有竞争对手的算法。

Sajjad 等人提出了 HoVerCut, 一个用于流式边缘分区算法的平台,它可以通过将状态与分区算法解耦来在多线程和分布式系统中扩展。每个运行 HoVerCut 的线程都会接收流式边缘的唯一部分,并使用任何流式边缘分区算法(例如 HDRF)对其进行分区。线程保留包含流算法所需的特定信息的本地状态,例如处理顶点的部分度、部分块权重和顶点分配。HoVerCut 使用缓冲技术,其中这些本地状态通过包含在所有线程可访问的存储中的全局状态按需共享。所有线程异步重复以下步骤序列,直到整个图被分区:收集边缓冲区,从全局状态中提取相关信息,分区其缓冲区,并将本地状态推送到全局状态。在他们对真实世界和合成图的实验中, HoVerCut 持续加速了 HDRF 等算法的划分过程,同时大致保持了原始划分算法的顶点切割。

Mayer 等人提出 ADWISE, 一种基于窗口的边缘分区流算法。该算法保留一个动态的边缘窗口,并始终选择窗口中的最佳边缘并将其分配给一个块。更具体地说,窗口一开始只包含一个边缘;然后其大小动态增加或减少,以提高分区质量,同时大致坚持给定的延迟上限。这里,延迟被定义为分区算法将当前窗口中的边缘分配给块所需的时间。因此, ADWISE 通过保持平均延迟来坚持它,以在整个执行过程中控制分配边缘。

Hoang 等人提出了 CuSP, 一种分布式并行流算法,用于根据用户定义的策略来划分边缘。作者将 CuSP 构建为一个可编程框架,能够表达文献中使用的最常见的流边缘分区策略。这种方法的动机是可用的分区策略多种多样,并且缺乏能够处理这些策略的集成分区器。在他们的实验中, CuSP 可以在几分钟内对具有数十亿个顶点和数千亿个边的巨大图进行分区,同时允许不同的分区策略。

5.4.6 实验研究

Zainab 等人对用于顶点划分和边划分的主要单通道流算法进行了理论和实验研究。在根据这些算法的要求、公式和复杂性对这些算法进行分类后,作者根据不平衡、边缘切割和顶点切割等离线指标对它们进行了实验评估。他们的结果表明, Fennel 实现了最低的边缘切割,而 HDRF 获得了最低的复制因子。对于边缘切割分区算法,作者还根据通信成本和总执行时间等应用指标进行了实验比较。在他们的应用程序性能实验中, HDRF 和 Greedy 产生了更高的分区成本,这是通过迭代应用程序的较低计算时间和单通道流处理应用程序的更好的端到端延迟来补偿的。另一方面,散列具有最短的分区时间,这提高了应用程序的整体性能,其中数据局部性不会显着影响通信成本。

Pacaci 和 Özsu 提出了用于顶点和边划分的流算法的实验调查。他们研究算法的性能、资源使用和可扩展性。作者研究了基于离线图度量和在线图查询的顶点和边划分算法。Pacaci 和 Özsu 得出的结论是,主要最小化边缘切割或顶点切割无法代表实践中的工作负载性能,因为这些指标未解决某些工作负载特征和图形结构。例如,所有测试的算法在实践中都会遇到工作负载分配不佳的问题,即使这些算法理论上应该提供平衡的分区。此外,他们的实验表明,边缘切割和顶点切割是网络通信的可靠指标,尽管这并不总能提高实践中的应用程序性能。

另一方面，实验还表明，哈希对于横向扩展图数据库来说是一种有效的策略，因为它提供了良好的性能。

5.5 再划分

图重新分区是图分区的扩展，它处理动态图，即顶点和边集随时间修改的图。假设图是分区的，其组件的动态变化可能会使其块不平衡，并且其边割或顶点割变大。图 7 说明了图随着时间的推移以及重新分区工作的演变。一般来说，通过每次修改整个图时从头开始分区，可以获得重新分区质量的粗略上限。由于这很昂贵，因此许多重新分区程序使用更快的方法。

Vaquero 等人提出了一种用于大规模图处理系统的分布式重新分区算法。在他们的算法中，散列被用作所有顶点的初始分区。基于局部信息，标签传播方案迭代地将顶点迁移到其大多数邻居所在的块，直到实现收敛。此过程本身处理图中的修改。他们的实验表明，与散列算法相比，他们的方法能够在图形变化时保持更好的边缘切割和应用程序性能。作者没有将他们的算法与其他最先进的重新分区算法进行比较。

Xu 等人提出了 LogGP，一种用于图处理系统的集中协调的重新分区算法。他们的算法分析并重用历史信息来改进分区。特别是，LogGP 通过将图与代表先前历史分区的超边相结合来构建超图。然后通过一种新颖的流引脚分区算法对该超图进行分区。在执行过程中，系统使用运行日志的统计推断来优化分区过程。在他们的实验中，LogGP 在边缘切割和应用程序运行时方面优于最先进的流和重新分区算法。

Nicoara 等人提出了一种轻量级重新分区算法并将其集成到他们自己的分布式社交网络数据库系统 Hermes 中。在 Hermes 中，初始分区是通过哈希或 Metis 执行的，而它们的重新分区算法是在图的顶点集发生变化时触发的。重新分区使用平衡和边缘切割的迭代局部改进。仅当改进过程完成时才会移动关联数据。在他们对现实世界工作负载的实验中，他们的算法产生了大致相同的边缘切割，同时与 Metis（从头开始重新分区）相比，迁移的数据少了一个数量级。此外，他们的算法在总吞吐量方面比散列算法提高了约 1.7 倍。

Huang 和 Abadi 提出了 Leopard，一种解决图重新分区同时复制顶点的算法。Leopard 可以利用任何一次性顶点分区算法。这是可能的，因为这些算法是以顶点为中心的，因此它们可以随时在动态图上本地使用。此外，Leopard 是第一个将顶点复制与重新分区集成在一起的算法。这种复制提供了容错能力并提高了局部性，甚至提高了分区质量，因为它主要基于本地信息。他们的实验表明，没有顶点复制的 Leopard 可以随时保持与从头开始运行 Metis 相当的分区质量。另一方面，具有顶点复制功能的 Leopard 进一步将边缘切割减少了一个数量级。

Kiefer 等人提出了一种针对性能不随输入大小线性扩展的系统进行优化的重新分区算法。作者首先通过将块的权重定义为其顶点的权重加上随块的基数单调增长的惩罚函数之和，来制定图分区问题的惩罚版本。在实践中，这种公式会惩罚资源消耗，从而模拟一些现实世界系统性能的非线性行为。作者修改了用于图分区的静态多级算法来解决问题的惩罚版本，然后在混合重新分区策略的改编版本中使用该多级算法。他们的混合重新分区算法的工作原理是，只要违反平衡约束，就会触发局部细化，并在给定数量的细化后在后台计算全新的分区。更具体地说，这个新分区是从头开始计算的；然后，它的块被映射到前一个分区的块

上, 以最小化迁移成本 (有关进程映射的更多详细信息, 请参阅第 5.6.5 节)。然后, 仅当新的边缘切割补偿了迁移开销时, 它才会替换当前分区。作者在 Metis 框架上实现了他们的算法, 并表明他们的静态算法平均比 Metis 多花 28% 的时间, 同时保持顶点、边和块数量的线性复杂度。最后, 他们的重新分区算法能够在整个图更新过程中保持不平衡、边缘切割和较低的迁移时间。

Fan 等人研究重新划分顶点 (边) 的问题, 以便同时最小化边切割 (vertex-cut) 和对初始划分的修改。作者证明了即使对于问题的有限情况也具有很强的棘手性。此外, 他们提供了重要的证明, 证明这个问题对于边缘切割和顶点切割来说是无界的。基于所提出的定理, 作者推导出一种策略, 将成功的分区算法转换为重新分区算法, 同时确保较小的迁移成本并保持与原始算法相同的分区质量范围。作为概念证明, 作者将他们的策略应用于 Fennel (顶点分区) 和 HDRF (边缘分区) 等算法。最后, 作者根据重新分区算法 Hermes 和 ParMetis 等从头开始分区的算法通过实验验证了他们的算法。在他们的实验中, 他们的算法比原始算法快一个数量级, 同时保持相当甚至更好的分区质量。

5.6 目标函数和问题变体

本次调查大部分研究的简单平衡 (超) 图划分问题并未涵盖与应用程序相关的所有方面。因此, 诸如多重约束、有向边、矩阵划分、流程映射以及通信成本建模的各种方法等附加问题都得到了考虑。

5.6.1 目标函数

Kaya 等人提出了关于不同分区模型和度量对并行稀疏矩阵向量乘法 (SpMxV) 性能影响的实验研究。为此, 使用多种矩阵划分模型对现实生活和人工稀疏矩阵进行划分。然后, 作者使用三种不同的 SpMxV 实现来测量矩阵向量乘法的运行时间, 并通过执行回归分析来确定不同分区指标对运行时间的影响。实验评估表明, 正确的划分模型和度量取决于所使用的处理元件的数量、具体的 SpMxV 实现以及矩阵的大小和结构。更具体地说, 最小化发送的消息总数比最小化每个处理元素的发送量更重要, 特别是如果处理元素的数量 ≥ 64 。棋盘分区模型减少了大部分分区指标, 通常实现了最低的运行时间。

最初由 Schloegel 等人提出, 多目标方法试图同时最小化多个目标函数。为此, Deveci 等人最近介绍了多目标超图划分器 UMPa。虽然之前关于此问题变化的工作最小化了多个阶段的多个目标, 其中第一阶段优化了主要目标, 而后续阶段可能不会恶化该目标, 但 UMPa 使用单阶段方法结合二级和三级指标来优化主要指标。这是通过贪婪地移动边界顶点来优化主要指标, 同时如果多个符合条件的目标块最大化主要 (或次要) 指标, 则使用次要 (和第三) 指标来打破平局。边界顶点的最佳目标块的选择取决于其增益值, 即如果顶点移动到另一个块则度量的变化。为此, 作者描述了如何有效地计算通信量和消息数量指标以及每个块的指标的增益值。

5.6.2 多约束图划分

平衡图分区通常受到每个块权重上限的限制。在 Karypis 和 Kumar 引入的多约束图划分模型中, 图的每个顶点都与权重向量相关联, 并且对每个权重施加平衡约束。多约束分区扩展到超图, 从而实现稀疏矩阵/图的二维粗粒度分区。最近, Recalde 等人引入了一种针对多约束图划分问题的精确算法。提供了两个整

数规划公式，作者证明了与各个多面体相关的几个不等式族。Recalde 等人使用基于分支定界的方法。可以在大约半分钟的 CPU 时间内解决具有 30 个顶点的现实世界实例。此外，几个已证明的不等式族显著减少了分支定界顶点的数量和最优化差距。

5.6.3 有向无环（超）图划分

如果输入实例是有向无环图 (DAG)，人们通常会对找到具有无环商图的分图感兴趣。Moreira 等人表明，即使在额外的非循环约束下，完美平衡的图划分也是 NP 完全的，并且不存在 $k \geq 3$ 的常数因子近似算法。Herrmann 等人 and Moreira 等人都提出了 DAG 分区的多级启发式方法。Popp 等人使这些技术适应有向非循环超图划分。Moreira 等人使用大小约束标签传播来粗化图。由于该算法不适合 DAG，因此收缩计算的集群会创建带有循环的粗略图。因此，分区器在粗化之前计算初始非循环分区。将簇限制为单个块可以确保输入图的分图也可以用作最粗糙图的分图。Herrmann 等人提出了一种非循环聚类算法，确保粗图是非循环的。DAG 顶点 v 的顶层定义为从 DAG 的任何源到 v 的最长距离。基于相邻顶点的顶层，制定了簇间和簇内边缘的条件，并且表明，尊重这些条件的聚类产生非循环粗图。Moreira 等人。通过将图的拓扑有序顶点切割成 k 个大小相等的块来计算初始分区。Herrmann 等人。使贪婪图增长适应初始双向划分和初始单向划分。此外，将图像无向一样进行二划分（例如使用 Metis）并随后修复非循环约束似乎很有希望。为了细化，Herrmann 等人。和 Moreira 等人。通过将可能的顶点移动集合限制为不违反非循环约束的顶点移动，使 FM 算法适应 DAG 分区。Herrmann 等人注意到，如果分区只有两个块，则可以有效地检查此条件。

5.6.4 对称直线矩阵划分

Yaşar 等人考虑对称直线稀疏矩阵划分问题。这里，稀疏矩阵的列和行使用相同的分区向量进行分区（与更一般的直线矩阵分区问题相反，其中使用不同的向量）。这会产生稀疏矩阵的平铺，对角线上有方形平铺。目标是找到一个分区，使最重图块的权重（即分配给图块的所有非零矩阵条目的总和）最小化，或者在给定的图块权重上限的情况下，找到受最小切割次数约束的分区。作者表明，这两个问题的变体都是 NP 完全的，提出启发式方法来优化任一目标，并提供其有效的实现。通过稀疏矩阵来减少运行时间。使用这些技术，该算法可以在现代 24 核系统上在不到 3 秒的时间内对代表 Twitter 社交网络的图进行分区，该图具有大约 15 亿条边。

5.6.5 流程图

在高性能系统的处理元件 (PE) 之间分配一组并行和通信的进程是图分区的主要应用之一。假设这些进程及其通信分别由应用程序图的顶点和边表示，可以对应用程序图进行分区，以便在 PE 之间分割进程。当不同对的 PE 以不同的速度相互通信时，边缘切割并不能反映总的通信成本。进程映射是图分区的推广，旨在将一组进程划分为大小大致相等的块，并将这些块映射到 PE 上，以最小化总通信成本或相关目标函数。

进程到 PE 的给定映射的确切总通信成本取决于带宽和延迟的组合，而这取决于实践中的多种因素。在计算上，总通信成本通常基于包含 PE 之间距离的

(隐式) 拓扑矩阵进行建模。给定应用图到 PE 的映射, 总通信成本定义为切割边的权重之和乘以包含其各自端点的 PE 之间的距离。分层拓扑是一种特殊情况, 其中 PE 包含在模块和子模块的多层分层结构中。基于延迟的拓扑是另一种特殊类型的拓扑, 其中 PE 包含在拓扑图中, 使得一对 PE 之间的距离定义为它们之间的最短路径中的边数。基于延迟的拓扑中的总通信成本也称为 *Coco(.)* 或跳字节目标函数。总通信成本的另一个目标函数是基于带宽的度量最大拥塞, 它被定义为通过拓扑图的任何链路的消息交换的最大数量。另一个可能的目标是最大扩张, 它被定义为与给定映射的一对 PE 直接相关的最大通信成本。

流程图的算法可以分为两类。一方面, 单阶段算法将流程映射与图分区结合起来, 这样分区的目标 (通常是边缘切割) 通常被测量通信成本的函数所取代。另一方面, 两阶段算法解耦了分区和映射。在两阶段算法中, 使用默认图划分算法将通信图划分为 k 块, 同时通常最小化边缘切割。然后, 将划分的通信图的商图映射到 PE 上, 以最小化总通信成本或其他目标函数。读者可以参考[26, 34]来了解有关流程图的较早作品。

Deveci 等人提出了一种贪婪构造算法和两种细化算法, 以基于拓扑图将进程映射到 PE。构造算法迭代地选择与已映射的进程具有最高连接性的进程。然后, 这个过程被分配给最小化跳字节的模块, 这是通过在拓扑图上执行广度优先搜索找到的。它们的两种细化方法都基于进程交换, 但其中一种旨在最小化跳字节, 而另一种旨在最小化拥塞。作者在基于环面的系统上通过实验将他们的算法与系统的默认映射器以及 *Scotch* 和 *LibTopoMap* 进行比较。与系统的默认映射器相比, 他们的算法在纯通信应用和稀疏矩阵向量乘法方面的性能分别提高了 43% 和 23%, 并且在竞争对手中具有最佳的总体结果。

Deveci 等人还提出了一种利用几何分区将处理器映射到 PE 的方案。使用 *MultiJagged* 几何分区器分别对应用程序数据和 PE 进行分区。然后, 数据分区中的块被映射到 PE 分区中的相应块, 从而有效地将相互依赖的数据分配给网络中“附近”的 PE。该方法适用于具有几何坐标和图形数据的应用程序。

Vogelstein 等人解决了一个与应用程序无关的问题, 其中有两个具有相同数量顶点的图, 它们双射映射到彼此上, 以便最大限度地减少引起的边缘不一致的数量。作者将此问题称为图匹配问题, 并将其表述为二次分配问题 (QAP)。他们用基于梯度向量的非线性近似算法来解决这个问题。他们的算法具有复杂性 $O(k^3)$, 并且在超过 80% 的 QAPLIB 基准库的运行时间和目标函数方面比之前最先进的表现更好。

Glantz 等人提出了两种贪婪算法来将块双射映射到假设通信图的 PE 上。他们最成功的算法是对 Müller-Merbach 提出的贪婪映射算法的改编。Müller-Merbach 的算法最初计算每个进程的总通信量以及每个 PE 到所有其他进程的距离总和。然后迭代地将通信量最大的进程分配给总距离最小的 PE。Glantz 等人通过根据要完成的通信量缩放距离来修改该算法。此修改提高了相对于质量测量最大拥塞和最大扩张的整体映射质量。

Glantz 等人提出了一种用于一对一过程映射的局部改进算法, 其中硬件拓扑是部分立方体, 即超立方体的等距子图。作者利用这些拓扑的规律性来标记 PE 以及沿凸切割使用位串的进程。这些位串允许快速计算 PE 之间的距离并实施有效的分层细化方法以改进由标签引起的映射。他们的实验结果表明, 他们的算法将最先进的映射算法生成的映射的总通信成本降低了 6% 到 34%, 同时映射时间保持在相同的数量级内。

Schulz 和 Träff 使用两阶段方法解决了分层拓扑的流程映射。他们的算法使用最先进的分区器 KaHIP 进行分区阶段,然后使用多部分进行一对一映射阶段。所提及的多部分算法以自上而下的方向在通信拓扑的各层中递归地划分先前获得的划分的商图。该算法获得一对一映射后,对 Brandfass 等人提出的基于交换的细化方法进行了轻量级修改。执行是为了进一步最小化通信成本。他们的实验表明,与将相同的基于 KaHIP 的第一阶段与最先进的一对一映射算法相结合的替代方法相比,他们的方法产生的映射具有较低的总通信成本。基希巴赫等人。进一步改进了 Schulz 和 Träff 的两遍方法。特别是,作者专门在分区阶段多次应用 KaHIP,即在整个分层拓扑中。Kirchbach 等人通过实验表明,与 Schulz 和 Träff 的算法相比,他们的最佳算法速度更快,同时还降低了总通信成本。

Faraj 等人提出了一种单相多级算法来解决分层拓扑的过程映射。他们的多部分方案使用自上而下的递归多部分算法来进行初始映射。然后,在去粗化过程中,使用经过专门设计的标签传播、k 路 FM、多尝试 FM 和商图细化的修改版本来改进该初始映射,这些都是专门为最小化总通信成本而设计的。此外,他们的算法隐式地表示了 PE 的压缩二进制标记中的分层拓扑,它使用 PE 数量的线性空间,并提供恒定时间内 PE 之间的距离。实验上,他们的算法比 Schulz 和 Träff 以及 Kirchbach 等人提出的算法改善了总通信成本同时还具有较短的映射时间。

Predari 等人提出了一种单相分布式算法来解决分层拓扑的过程映射。作者将系统层次结构建模为隐式标记树,并标记通信图的顶点,以便隐式诱导映射。该算法通过使用并行标签传播的改编版本来优化映射,其中标签方案用于快速增益计算。在他们的实验中,所提出的算法对于多达数千个 PE 具有良好的可扩展性,同时实现的总通信成本小于分布式算法 ParMetis 和 ParHIP,并且与 Faraj 等人提出的最先进的顺序映射算法的成本相当。

Kirchbach 等人假设进程以稀疏模板模式通信并且 PE 以笛卡尔网格组织,则解决了进程映射问题。首先,作者证明笛卡尔映射对于二维笛卡尔网格和一维模板来说已经是 NP 困难的。然后提出了三种利用问题规律性的快速构造算法。他们具有最佳总体结果的算法是模板条算法,该算法将网格划分为长度接近目标模板坐标的最佳边界矩形的缩放长度的条。在使用 MPI 的 MPI_Neighbor_alltoall 例程进行的实验评估中,他们的算法比 VieM 等通用图形映射工具快两个数量级,同时产生类似的通信性能。此外,他们的算法比其他笛卡尔网格映射算法快三倍,同时带来更好的通信效果。

第 6 章 并行算法

图分区的大部分初始工作都涉及顺序算法。这些算法已扩展到在分布式内存环境中工作,特别是用于平衡并行应用程序中的处理器工作负载。在这个用例中,分布式内存应用程序已经具有图的分布;为了内存可扩展性,整个图并不存储在每个处理器中。因此,分布式内存分区算法通常不具有整个图的全局视图;他们经常根据本地图数据的部分视图做出分区决策。因此,它们的解决方案质量可能低于顺序解决方案的质量。尽管如此,分布式内存算法对于太大而无法容纳在单个内存空间中的图形以及希望动态分区数据以适应不断变化的计算工作负载的应用程序至关重要。

近年来,共享内存算法取得了进展,因为共享内存架构比分布式内存架构提供了更大的灵活性。例如,与分布式内存机器相比,随机内存访问或原子更新可

以快几个数量级。由于共享内存算法具有图形的全局视图，因此它们可以实现与顺序前辈相同的解决方案质量。然而，对于无法容纳在单个内存空间中的极大图来说，它们是不可行的。

6.1 共享内存

本节讨论的大多数算法都是多级算法。因此，我们将讨论分为每个阶段的单独贡献。我们考虑的多级算法是 Lasalle 和 Karypis 的 Mt-Metis、Akhremtsev 等人的 Mt-KaHiP, Mt-KaHyPar, Gottesbüren 等人(超图), KaMinPar, Gottesbüren 等人(深度多级)，以及 Maleki 等人的 BiPart。接下来的工作重点是尽可能忠实地将顺序方法转移到共享内存，同时有时会结合从分布式内存中吸取的经验教训。

6.1.1 粗化

大多数粗化算法基于贪婪匹配或贪婪聚类。这些算法以某种顺序（例如，随机）访问顶点，计算用于加入相邻簇（或与邻居匹配）的评级（例如，heavyedge），然后加入评级最高的簇。并行访问顶点会产生顺序方法的忠实并行化。挑战在于防止线程之间的集群决策不一致。

Çatalyürek 等人除了在第二遍中解决冲突的贪婪匹配的无锁版本之外，还提出了使用锁定的凝聚聚类 and 贪婪匹配的并行方案。基于锁的算法首先尝试锁定访问的顶点，计算评级，然后迭代候选顶点。当找到更好的候选者时，就会测试其锁定。如果成功，旧候选者将被替换并解锁。对于无锁解析方案，匹配项存储在全局数组 M 中，不保护写访问。经过一轮后，带有 $M[M[u]] \neq u$ 的顶点 u 将与其自身匹配 $M[u] \leftarrow u$ 。这些顶点的匹配被同时访问并匹配到不同的顶点。在他们的评估中，基于锁的算法在基数和重边度量方面与顺序版本表现相同，而基于分辨率的算法在重边度量方面下降了 10%。

LaSalle 和 Karypis 在 Mt-Metis 中也使用了基于分辨率的匹配方案。对于具有倾斜度分布的输入（例如复杂网络），最大匹配很小，因此粗化收敛太慢。如果一次遍历后匹配的顶点太少，则会匹配具有相同邻居和低度数的非相邻顶点对。如果匹配的顶点仍然太少，则可以匹配共享公共邻居的任何顶点对。这种技术被称为两跳匹配。

Akhremtsev 等人相反，使用并行大小受限的标签传播聚类来更有效地粗化倾斜输入。没有使用锁，因此可能会发生循环集群连接。簇大小约束确保初始划分能够找到可行的划分。簇大小通过原子指令更新。在更新之前检查大小约束，但这不提供原子一致性。因此，检查指令的结果以保证大小限制（如果超出则恢复）。对于粗化，不需要严格遵守尺寸约束（而对于细化，则需要保证平衡）。KaMinPar 中也使用了标签传播，但是为了粗化（而不是为了细化）省略了原子检查。

对于 Mt-KaHyPar, Gottesbüren 等人使用并行凝聚聚类，但在评级和目标簇选择之后推迟锁定，仅锁定访问的顶点和目标簇。尝试循环相互连接的顶点会被动态检测和解析，从而递归地合并关联的簇。

6.1.2 初始划分

对于初始分区，LaSalle 和 Karypis 使用递归二分区。每个线程顺序计算一个二分区，从中选择最好的一个。对于递归，线程静态地分为两组，分别处理两

个单独的子图。如果图足够大，而不是独立的顺序试验，这一点后来被改进，以在一个二分区上合作的线程。Akhremtsev 等人使用顺序 KaHiP 计算独立路分区。这两种方法的缺点是潜在的严重负载不平衡（递归二分区）或成为顺序瓶颈。为了克服这个问题，Gottesbüren 等人使用工作窃取任务调度程序而不是通过并行（非）粗化来拆分线程来进行递归双分区。对于平面二分区，一系列顺序算法是独立并行运行的。

深度多级方法通过不同粗化级别的重复（分叉）提供额外的并行性。在这些阶段，图可能太小而无法有效利用并行（非）粗化，因此为重复而拆分线程几乎不会造成运行时间损失。因此，使用随机组件（粗化、初始二分、细化）提供了多种多样化的解决方案，从中选择最好的解决方案。

Slota 等人使用 k 个并行源 BFS 来计算 k 路划分。顶点连接其父母的块。他们的方法不使用粗化，这证明在这个阶段使用平面并行算法是合理的。

Maleki 等人并行化贪婪超图生长以进行二分。所有顶点都分配给块 V_0 。然后计算将块 V_0 中的顶点移动到块 V_1 的增益。 $\sqrt{|V|}$ 在重新计算块 V_0 中剩余顶点的增益之前，将移动评级最高的顶点。重复此操作直到二分平衡。

6.1.3 细化

在细化阶段必须解决的两个最关键的问题是如何保持平衡的分区以及如何确保并发移动不会导致更糟糕的削减。一些细化算法比其他算法更难并行化。毫不奇怪，更复杂的技术，特别是那些能够逃脱局部最小值的技术，更难以并行化，因为需要中间的负增益移动。特别是 FM 和 KL 被认为是 NP 完全的，这使得多对数深度算法不太可能存在。

标签传播 最直接的并行方法是标签传播：只需并行访问顶点。虽然增益可能由于邻近区域的并发移动而不正确，但这在实践中不会对剪切优化产生太大影响。可以通过使用原子指令更新块权重，或者首先收集移动并在第二步中批准来确保平衡。在细化过程中，重要的是通过检查原子指令的结果来强制执行大小约束，以保证平衡分区。

贪婪算法 LaSalle 和 Karypis 通过静态地将顶点分配给线程并在局部顶点上运行顺序算法来并行化 Metis 的贪婪细化。顺序算法在边界顶点上执行 FM，但一旦没有正增益移动就停止。这简化了并行化，因为无需恢复任何移动，因此无需观察串行移动顺序。然而，它确实消除了逃避局部最小值的能力。移动通过消息缓冲区传达到相邻顶点的其他线程。线程经常检查它们的缓冲区以更新本地增益。此外，细化被分为上游和下游通道，其中仅允许顶点移动到具有比当前块更高（或更小的）标识符的块中。这避免了意外的剪切降级并发移动，但限制了搜索空间，因为某些块很快就会接近过载。平衡检查是通过本地更新的块权重乐观地完成的。由于这可能会导致平衡违规，因此通过后的同步点用于恢复某些移动以恢复平衡。

并行本地化 FM 局部 FM 是 FM 的一种变体，它从几个边界顶点开始，并将其搜索空间扩展到移动顶点的邻居。由于允许负增益移动，它擅长逃离局部最小值，但不会通过短暂的搜索冲刺在没有希望的区域浪费太多时间。这种方法可以通过执行多个独立的局部搜索来并行化，这与难以有效并行化的标准 FM 不同。Akhremtsev 等人将边界顶点组织在随机打乱的队列中。线程重复从队列中轮询种子顶点并围绕其种子执行局部 FM。移动不会传递给其他线程。相反，每

个线程维护一个本地分区——一个用于块权重的数组和一个用于分区 ID 的哈希表。将移动保持私有的基本原理是，在本地化搜索结束时恢复（负增益）移动会混淆其他搜索。搜索可能在其本地顶点中重叠，但每个顶点仅移动一次（原子测试和设置）。一旦全局队列为空，线程的本地移动序列就会连接成一个序列。为了确保平衡划分并且没有切割退化，按顺序重新计算该序列的增益，并应用产生受平衡约束的最小边缘切割的前缀。

在 Mt-KaHyPar, Gottesbüren 等人一旦找到局部最优值，就将局部移动序列应用于全局分区。这为其他线程提供了更准确的信息。另外，所得到的移动顺序更充分地反映了局部搜索的移动决策所基于的分区状态。最后，它减少了哈希表的内存，同时保持本地化搜索结束时的负增益移动私有。这对于超图很重要，因为本地分区数据结构中存储的信息比图分区中存储的信息更多。如果应用于全局分区的移动违反了平衡约束，则被拒绝，并且块权重由原子指令维护。

6.1.4 基于并行流的细化

第 5.2 节中基于流的细化提供了两个并行源，这些并行源在中进行了研究。核心例程适用于二分区，因此它可以并行应用于单路分区不同块对。基本版本仅并行运行非重叠块对，但作者表明重叠搜索在某些限制下是可行的，并且对于更好的并行性是必要的。第二个并行源是流算法，其中众所周知的推送重新标记算法非常适合并行化。该方法集成在最先进的并行多级框架 Mt-KaHyPar 中。实验表明，新算法的分区质量与最高质量的顺序代码 (KaHyPar) 相当，同时在使用 10 个线程时速度提高了一个数量级。

山丘扫描 LaSalle 和 Karypis 通过采用局部搜索的简化变体，在某种程度上扩展了他们的贪婪细化以逃避局部极小值。每当线程本地优先级队列中仅保留负增益移动时，就会增量构造一小组 u 周围的顶点（最多 16 个），希望移动整个组可以减少削减。该组的构建方式与本地化 FM 扩展搜索的方式相同；然而，一旦有可能改善，扩张就会停止。与贪婪算法相反，所选顶点不限于线程本地顶点。一项限制是所有顶点必须移动到同一个块。

再平衡 如果同时移动顶点，则检查每次移动是否保持平衡是不够的，还需要某种同步机制。因此，一些细化算法无法保证平衡分区，因此需要显式的再平衡算法。此外，即使某些细化算法可以保证平衡，但中间的平衡违规可能会导致重新平衡步骤后的较小削减。

Slota 等人使用标签传播，增益乘以 $L_{\max}/c(V_i)$ 有利于进入较轻的区块。Maleki 等人使用与图增长并行化相同的方法来重新平衡双向分区，从逐渐填充的非空较轻块开始。

Lasalle 和 Karypis 将再平衡整合到他们的并行贪婪细化中。线程局部移动缓冲区以相反的顺序遍历，并且恢复到重载块中的移动。每个线程负责恢复与其移入该块的量成比例的多余权重。

对于大数 k 的，Gottesbüren 等人每个过载块使用一个优先级队列，以最高增益（与非过载块）和顶点权重的比率作为关键。每个队列都填充了足够的顶点以消除多余的权重，但会插入同一前一个块中移动顶点的邻居，以防指定的目标块接近过载。并行性是通过并行清空不同的重载块来实现的。

Gottesbüren 等人允许在 Mt-KaHyPar 中局部 FM 的增益重新计算步骤中控制平衡违规。下一级的标签传播通常能够重新平衡，并且在最精细的级别上采用与标签传播类似的显式重新平衡。

并行 n 级（非）粗化 Gottesbüren 等人提出了 Mt-KaHyPar 的 n 级版本。对于顺序级别，一次只有一个顶点（未）收缩，这本质上是顺序的，但通过高度局部化的细化提供了高质量的解决方案。在并行版本中，顶点对与以分层森林数据结构组织的要收缩的顶点异步收缩。森林为异步收缩产生优先条件（自下而上）。这样，一旦子树完成，不同子树中的顶点就可以独立并行收缩。采用细粒度锁定来编辑超图数据结构，并动态维护森林的一致性。

取消粗化通过并行取消收缩 $b > 1$ 个顶点的批次来引入并行性。这些批次是通过自上而下的顺序遍历森林，组装可以在批次中独立恢复的收缩来构建的。取消合同批次可以解决取消合同下一批所需的最后依赖项。当前批次的顶点充当高度局部化并行细化（标签传播和 FM）的种子。

决定论 几十年来，研究人员一直在宣传确定性并行算法的好处，包括易于调试、性能推理和可重复性。缺点是算法设计选择和同步优化决策的潜在开销方面灵活性较差。例如，Mt-KaHyPar 的确定性版本比等效的非确定性配置慢 1.16 倍，但连接性高出 1.029 倍（均使用几何平均值聚合）。有趣的是，这种退化源于粗化，而不是细化。

Maleki 等人提出 BiPart，一种确定性多级递归二分算法。对于粗化，顶点与它们的最小超边相匹配，并使用 ID 哈希作为平局断路器。与同一超边匹配的所有顶点都会收缩，而不限限制粗略顶点权重。初始分区使用上述并行贪婪图增长。为了细化，并行计算将每个顶点移动到相反块的增益。设 l_0, l_1 表示块 V_0, V_1 中具有正增益的顶点数。然后交换增益最高的 $\min(l_0, l_1)$ 顶点，这种方法已经为 Social Hash 提出。如果超图具有单位顶点权重，则这将保持平衡分区。然而，该算法在多级框架内使用，其中粗化会出现不均匀的顶点权重。因此，采用了上述的图增长再平衡。请注意，邻居移动后，增益就会变得陈旧，因此总体削减量并不是增益的总和。

Gottesbüren 和 Hamann 提出了 Mt-KaHyPar [83] 的确定性版本。用于 Mt-KaHyPar 预处理、粗化和细化的标签传播风格算法使用同步局部移动以确定性方式重新实现。移动决策不依赖于当前回合中的其他移动。轮次进一步分为子轮次，以纳入更多最新信息。在每个子回合之后，一些移动被批准，一些移动被拒绝，例如由于平衡约束或粗化的集群大小约束。为了细化，使用与 BiPart 和 SocialHash 相同的方法：在块对之间交换最高增益前缀。为了合并非单位顶点权重，计算移动序列的所有前缀的累积权重。然后，类似于并行合并，选择最佳前缀组合。

6.2 分布式内存

分布式（超）图分区是处理不适合单机主内存的大型输入的一种方法。在分布式内存模型中，多个处理器（PE）通过通信网络互连，每个处理器都有其私有内存，其他处理器无法访问。每个 PE 上的计算任务通常仅对代表输入的一小分子集的本地数据独立运行。中间计算结果必须通过专用网络通信原语进行交换。

分布式（超）图处理算法要求输入的顶点和边在处理器之间进行分区。由于许多应用程序使用平衡（超）图分区来获得良好的初始分配，因此在分布式分区器中使用更简单的技术。存在基于范围的和基于散列的分区技术。前者将顶点 ID 分割成等距的范围，然后将其分配给 PE。由于这两种技术都不考虑（超）图结构，因此可能导致负载不平衡或高通信开销。然而，随着有关（超）图结构的更多信息可用，人们也可以迁移顶点，例如，当在通过递归二分区获得的子图上递

归时。如果几何信息可用，还可以使用空间填充曲线。

然后，每个 PE 存储分配给它的顶点以及关联到它们的边。存储在 PE 上的边可以与本地顶点或其他 PE 上的顶点（也称为鬼顶点或晕顶点）相关联。如果一个 PE 与另一个 PE 共享公共边，我们就说它们相邻。处理器必须能够识别相邻的 PE 来传播更新，例如，如果我们将顶点移动到不同的块，我们必须将该更改传达给网络中的其他 PE，以便本地搜索算法可以处理准确的分区信息。然而，每个通信操作都会带来开销，从而限制系统的可扩展性。因此，分布式（超）图分区的主要挑战是在某种意义上保持每个 PE 上的全局分区信息是最新的，同时最小化所需的通信。

本节的其余部分描述了该领域最新出版物的算法核心思想，并从顶点的物理位置和分布式（超）图数据结构的实际表示中进行了抽象。然而，我们假设每个顶点都知道其邻居存储在哪个 PE 上。

本地搜索 标签传播启发式是分布式系统中最广泛使用的本地搜索算法。其他方法在相邻块对上并行调度顺序 k 路 FM。然而，这将可用的并行性限制为最多块数。

并行标签传播实现主要遵循批量同步并行模型。在计算阶段，每个 PE 为其本地顶点计算其所需的目标块。在通信阶段，通过个性化的全对所有通信，更新对其他 PE 是可见的。Meyerhenke 等人使用异步通信模型。如果 PE 的计算阶段结束，它会向其他 PE 发送更新并从其他 PE 接收更新，并立即继续下一轮。

在并行设置中，两个相邻顶点的移动增益在单独移动时可能会有所改善，但同时移动两个顶点可能会降低解的质量。因此，一些分区器使用顶点着色或两阶段协议，其中在第一阶段，顶点只能从块 V_i 移动到块 V_j 若 $i < j$ ，反之亦然，在第二阶段。许多系统不使用任何技术来防止移动冲突。这可以被视为一种乐观策略，假设实践中很少发生冲突。

最近发布的其他分布式本地搜索技术基于保持分区平衡的顶点交换技术。Rahimian 等人提出了使用这种方法的 JA-BE-JA。该算法迭代每个 PE 的局部顶点，对于每个顶点，它将所有相邻顶点视为交换候选者。如果没有找到伙伴，它会从样本中随机选择一个顶点作为候选顶点。如果所选顶点被分配给不同的 PE，则实例化 PE 会发送带有所有所需信息的请求，以便接收 PE 可以验证交换操作是否会改善边缘切割。成功后，两个顶点都会更改其块。此外，还使用模拟退火来避免局部最小值。

Aydin 等人实现一个分布式分区器，计算顶点的线性排序，然后将其分割成大小相等的范围以获得初始分区。这个想法类似于空间填充曲线，但不需要几何信息。初始排序是通过将标签分配给通过凝聚层次聚类构建的树来计算的。然后，它对叶子的标签进行排序以获得初始排序。为了进一步改进排序，它解决了试图

优化 $\sum_{(u,v) \in E} |\pi(u) - \pi(v)| \omega(u,v)$ 的最小线性排列问题，其中 $\pi(u)$ 表示 $u \in V$ 在当

前排序中的位置。为此，它使用重复直到收敛的两阶段 MapReduce 算法：首先，每个顶点计算其所需的新位置作为其邻居位置的加权中值。其次，通过使用简单的基于 ID 的排序解决重复项，将最终位置分配给顶点。第二个局部搜索算法执行顶点交换。它将每个块的顶点分割成不相交的集合，并将分区的相邻块配对并随机地配对它们的集合。然后将成对的集合映射到执行以下算法的处理器：如果移动到相反的块，则根据两个集合中的顶点的切割减少对它们进行排序，并交换具有最高组合切割减少的顶点。

平衡约束 标签传播算法只知道每个计算阶段开始时的确切块权重。在计算阶段，块权重仅在本地维护。在通信阶段，所有移动的组合可能会导致违反平衡约束的分区。因此，基于该方案的分区器必须采用技术来确保平衡。

分布式多级图分区器 ParHIP 将标签传播轮划分为子轮，并在每个子轮之后通过 **All-Reduce** 操作恢复精确的块权重。请注意，这并不能保证平衡，但当子圆形中移动的顶点数量较小时，可以很好地近似块权重。

多级算法 尽管众所周知，多级算法比平面分区方案产生更好的分区，但工业中使用的系统（例如 **Google** 或 **Facebook**）主要是非多级算法。其主要原因是多级算法的可扩展性通常仅限于几百个处理器。此外，大多数并行多级系统实现基于匹配的粗化算法，这些算法无法有效地减小当今复杂网络的大小（幂律节点度分布）。最著名的分布式多级算法是用于图的 **Jostle**、**ParMetis**、**PT-Scotch**、**KaPPa**、**ParHIP** 和 **ScalaPart**，以及用于超图划分的 **Parkway** 和 **Zoltan**。

Meyerhenke 等人构建并行多级分区器 **ParHIP**，它使用尺寸约束标签传播算法的并行版本。该算法用于在粗化阶段计算聚类，并在细化阶段用作局部搜索算法。为了获得最粗图的初始划分，它使用分布式进化图划分器 **KaFFPaE**。在复杂网络上，**ParHIP** 计算的边缘切割平均比 **ParMetis** 小 38%，同时速度也快了兩倍多。

Wang 等人使用类似的方法，也利用标签传播算法来计算粗化阶段的聚类。该算法是在微软的 **Trinity** 图形引擎之上实现的。分区器还使用外部存储器技术来对少量机器上的大图进行分区。但是，它不执行多级细化（初始分区投影到输入图）。

几何划分器 许多图源自几何应用并且富含坐标信息（例如，每个顶点与一个维点相关联）。具有坐标信息的网格和基于该坐标的分区如图 8 所示。几何分区技术使用该信息将相应的点集分区为大小相等的簇，同时最小化在簇上定义的目标函数。应选择目标函数，使其隐式优化所需的图划分度量（例如，所有边界框的长度之和近似于总通信量）。由于几何方法忽略（超）图的底层结构，因此与传统的多级算法相比，分区的质量通常较差。然而，这些算法通常更简单，导致算法更快、更具可扩展性。突出的技术使用空间填充曲线将一组维点映射到一条一维线。该曲线的一个基本属性是原始空间中接近的点在线上也很接近。其他方法通过切割平面递归地划分空间，例如基于八叉树的分区[147]、递归坐标二分和递归惯性二分。**Deveci** 等人的 **MultiJagged** 算法使用多重分割而不是二分来减少递归的深度和相对于递归坐标二分的计算速度；它的混合实现使用 **MPI** 和 **Kokkos** 来支持 **PE** 之间的分布式内存消息传递以及 **PE** 内的多线程或 **GPU** 计算。

6.3 GPU

由于其高计算能力，现代 **GPU** 已成为加速数据并行应用程序的重要工具。然而，由于图的结构高度不规则，设计有效利用现代 **GPU** 的 **SIMD** 架构的图算法仍然具有挑战性。

多级图划分 **Goodarzi** 等人提出了两种基于 **GPU** 的多级图分区算法。他们早期的方法使用重边匹配进行粗化，并将最粗的图传输到 **CPU** 上进行初始分区（使用 **Mt-Metis**）。在细化期间，顶点分布在线程之间，并且每个线程找到最大化其分配顶点的增益值的块。为了防止导致组合边缘切割恶化的冲突移动，细化在轮次之间交替进行，其中仅考虑移动到具有增加（或减少）块 ID 的块。对于

每个块，对该块的潜在移动被收集在全局缓冲区中，然后对其进行排序，并执行最高评级的移动。

他们后来的方法带来了一些改进。首先，作者使用 Warp Segmentation 来提高粗化过程中重匹配计算的效率。然后使用贪婪增长技术在 GPU 上执行初始分区。在细化过程中，顶点再次在线程之间划分，每个线程找到最大化其分配的边界顶点增益的块，并将潜在的移动收集在全局缓冲区中。然后，对于一些小的输入常数 ℓ ，算法会在全局缓冲区中找到最高评级的 ℓ 移动。由于移动可能会相互冲突，因此他们的算法将所有 2^ℓ 移动组合分布在线程组之间，并找到最佳组合，然后将其应用于图分区。重复此过程直到全局缓冲区为空。平均而言，他们基于 GPU 的方法大约比 Mt-Metis 快 1.9 倍，而在 16 个基准图集上计算的边缘切割稍差。

Fagginger Auer 和 Bisseling 提出了一种用于图粗化的细粒度共享内存并行算法，并将该算法应用于图聚类的背景下，以获得快速贪婪启发式，以最大化加权无向图中的模块性。该算法同时适用于多核 CPU 和 GPU。后来，Gilbert 等人提出了性能可移植的图粗化算法。作者特别研究了重边缘粗化方法的 GPU 并行化。作者使用多级谱图分区算法作为主要用例来评估他们的粗化方法。

谱图划分 GPU 上高效特征求解器的可用性导致最近在 GPU 系统上重新出现用于图形分区的谱技术。这些技术首先由 Donath 和 Hoffman 以及 Fiedler 在 20 世纪 70 年代开发，用于计算图二等分。随后，这些技术得到了改进并扩展到使用多个特征向量将图划分为两个以上的块。

Naumov 和 Moon 提出了一种针对单 GPU 系统的谱图分区的实现，作为 nvGRAPH 库的一部分，而 Acer 等人提出了多 GPU 实现 Sphynx。两个分区器都对矩阵进行预处理并使用 LOBPCG [124] 特征值求解器。然后使用特征向量将图嵌入到多维坐标空间中，然后使用该空间导出图的分区。在 nvGRAPH 中，这是通过在嵌入图上使用 k-means 聚类算法来完成的，而 Sphynx 使用支持多 GPU 系统的几何图分区器 Multi-Jagged。由于宏碁等人的方法。在分区平衡、切割大小和运行时间（在单 GPU 系统上运行时）方面优于 nvGRAPH，我们重点关注他们的实验评估。为此，当 Sphynx 与 ParMETIS [116] 进行比较时，ParMETIS 通常比 Sphynx 获得更好的切割效果——大约。对规则（或不规则）图实例的切割降低 20%（或 70%）。作者报告称，在不规则图上，在 24 个 GPU 上使用 Sphynx 与在四个计算节点上使用 168 个 MPI 进程的 ParMETIS 相比，速度显着提高了约 19 倍，尽管在规则图上 ParMETIS 比 Sphynx 快约 3 倍，即使 Sphynx 使用的每个 GPU 使用单个 CPU 核心也是如此。此外，宏碁等人。报告几种矩阵预处理器对不同类别图的影响。